Data Structures
Practical Answers

Complete C++ programs for Sets 1 - 40 and Sets 43 - 72

Arrays, Linked Lists, Stacks, Queues, Linear Search and Binary Search

- Every program is menu-driven (or single-run for the search sets) and written in standard C++17.
- Each program was compiled with `g++ -std=c++17 -Wall -Wextra -Wpedantic` with no warnings, run under AddressSanitizer / UBSan with no errors, and its output was checked line by line against the original C solution on the same inputs.
- Compile with: `g++ -std=c++17 setNN.cpp -o setNN` (no extra flags are needed, including the infix / postfix programs).
- Sets 33, 38, 41 and 42 are not present in the two problem PDFs, so they are not included.
- Where a problem lists only part of the requirements (Sets 29 and 39 stop at the input step), the full billing system (postfix conversion, evaluation, final amount, stack trace) is implemented.
- Assumptions: days in the linked-list sets are numbered 1, 2, 3... by position and renumbered after each insert or delete; "middle" insertion places the new node after the middle node; queues are simple linear array queues of size 5 (change `MAX` to resize); one comparison is counted per probe in binary search.

Index of Sets

Set	Topic	Page
1	Maintain N books records (Book ID, Title, Price) using an array	4
2	Daily book return status (R = Returned / N = Not returned) using a singly linked list	6
3	Daily book return status (R = Returned / N = Not returned) using a singly linked list	9
4	Maintain a list of library members sorted by Member ID using a doubly linked list	12
5	Store the number of books issued of N members using both an array and a linked list	14
6	Maintain N patients records (Patient ID, Name, Age) using an array	16
7	Daily patient check-up status (C = Completed / M = Missed) using a singly linked list	18
8	Daily patient check-up status (C = Completed / M = Missed) using a singly linked list	21
9	Maintain a list of doctors sorted by Doctor ID using a doubly linked list	24
10	Store the body temperature (F) of N patients using both an array and a linked list	26
11	Maintain N products records (Product ID, Name, Quantity) using an array	28
12	Daily product stock status (A = Available / O = Out of stock) using a singly linked list	30
13	Daily product stock status (A = Available / O = Out of stock) using a singly linked list	33
14	Maintain a list of suppliers sorted by Supplier ID using a doubly linked list	36
15	Store the monthly sales units of N products using both an array and a linked list	38
16	Maintain N customers records (Account No, Name, Balance) using an array	40
17	Daily EMI payment status (P = Paid / D = Due) using a singly linked list	42
18	Daily EMI payment status (P = Paid / D = Due) using a singly linked list	45
19	Maintain a list of loan applicants sorted by Application No using a doubly linked list	48
20	Store the account balance of N customers using both an array and a linked list	50
21	Simulate the page navigation history of an online library catalogue using a stack	52
22	Late-fine formula: infix to postfix and evaluation	54
23	Order return handling of an Amazon warehouse using a stack	58
24	Library Fine Billing System	60
25	Book issue counter queue of a library	64
26	Simulate the page navigation history of a hospital patient portal using a stack	66
27	Hospital bill formula: infix to postfix and evaluation	67
28	Flight lookup system of an airline using binary search	72
29	Hospital Billing System	73
30	OPD (outpatient department) queue of a hospital	78
31	Simulate the page navigation history of a warehouse management portal using a stack	80
32	Price formula: infix to postfix and evaluation	81
34	Purchase Billing System	86
35	Goods dispatch queue of a warehouse	90
36	Simulate the page navigation history of an internet banking portal using a stack	92
37	Interest formula: infix to postfix and evaluation	93
39	Loan Billing System	98
40	Teller counter queue of a bank	102
43	Passenger list of a flight - linear search by Name	104
44	Hospital patient records - linear search by Name (repeated names)	105
45	Bank transaction history - linear search by Transaction ID	106
46	Library catalogue - linear search by Title	107

Set	Topic	Page
47	Online store product list - linear search by Name	109
48	Student marks list - linear search by Roll No	110
49	Hotel guest register - linear search by Name	111
50	Railway reservation list - linear search by PNR	112
51	Phone directory - linear search by Phone Number	114
52	Warehouse stock list - linear search by Item Code (repeated codes)	115
53	Online store product catalogue - binary search by Product ID	116
54	Bank customer records - binary search by Account No	118
55	Student records (sorted input) - iterative binary search with low/mid/high trace	119
56	Library records - recursive binary search	121
57	Hospital patient records - binary search compared with linear search	122
58	Railway timetable - binary search by Train No	124
59	Employee records - binary search by Employee ID	126
60	Hotel rooms (sorted input) - binary search by Room No	127
61	Pin code list - binary search by Pin Code	129
62	Sorted student marks with duplicates - first / last occurrence by binary search	130
63	Ticket booking counter of a cinema (simple queue, array based)	132
64	Teller counter of a bank (simple queue)	134
65	Print queue of a college lab (simple queue)	135
66	OPD queue of a hospital (simple queue)	137
67	Check-in counter of an airline (simple queue)	139
68	Order processing of a food delivery app (simple queue)	141
69	Call queue of a customer care centre (simple queue)	143
70	Support ticket system of a software company (simple queue)	144
71	Railway reservation counter (simple queue)	146
72	Order packing line of an Amazon warehouse (simple queue)	148

Set 1 – Maintain N books records (Book ID, Title, Price) using an array

a. Insert a new book at a given position
b. Delete a book by Book ID
c. Search a book by Book ID and display the record
d. Display all books

Program (C++)

```
/* Set 1 - Maintain N books records (Book ID, Title, Price) using an array */
// Compile: g++ -std=c++17 -Wall -Wextra set1.cpp -o set1
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Book {
    int id;            // Book ID
    string name;       // Title
    float val;         // Price
};

Book records[MAX];
int n = 0;             // number of records currently stored

void readRecord(Book &r)
{
    cout << " Enter Book ID : ";
    cin >> r.id;
    cout << " Enter Title : ";
    cin >> ws;
    getline(cin, r.name);
    cout << " Enter Price : ";
    cin >> r.val;
}

void printHeader()
{
    cout << "\n" << left << setw(12) << "Book ID" << " " << setw(25) << "Title"
         << " " << "Price" << "\n";
    cout << "-------------------------------------------------------\n";
}

void printRecord(const Book &r)
{
    cout << left << setw(12) << r.id << " " << setw(25) << r.name << " " << r.val << "\n";
}

// returns the index of the record having the given ID, or -1
int findIndex(int id)
{
    for (int i = 0; i < n; i++) {
        if (records[i].id == id)
            return i;
    }
    return -1;
}

// a. Insert a new book at a given position (1-based)
void insertAt(int pos)
{
    Book r;
    if (n == MAX) {
        cout << "Array is full. Cannot insert.\n";
        return;
    }
    if (pos < 1 || pos > n + 1) {
        cout << "Invalid position! Valid range is 1 to " << n + 1 << ".\n";
        return;
    }
    readRecord(r);
    if (findIndex(r.id) != -1) {
        cout << "Book ID already exists. Insertion cancelled.\n";
```

```
        return;
    }
    for (int i = n - 1; i >= pos - 1; i--)      // shift right
        records[i + 1] = records[i];
    records[pos - 1] = r;
    n++;
    cout << "Book inserted at position " << pos << ".\n";
}

// b. Delete a book by Book ID
void deleteById(int id)
{
    int idx = findIndex(id);
    if (idx == -1) {
        cout << "Book with Book ID " << id << " not found.\n";
        return;
    }
    for (int i = idx; i < n - 1; i++)           // shift left
        records[i] = records[i + 1];
    n--;
    cout << "Book with Book ID " << id << " deleted.\n";
}

// c. Search a book by Book ID and display the record
void searchById(int id)
{
    int idx = findIndex(id);
    if (idx == -1) {
        cout << "Book with Book ID " << id << " not found.\n";
        return;
    }
    printHeader();
    printRecord(records[idx]);
}

// d. Display all books
void displayAll()
{
    if (n == 0) {
        cout << "No records to display.\n";
        return;
    }
    printHeader();
    for (int i = 0; i < n; i++)
        printRecord(records[i]);
}

int main()
{
    int choice, pos, id;
    cout << fixed << setprecision(2);
    cout << "Enter number of books (max " << MAX << "): ";
    cin >> n;
    if (n < 0 || n > MAX) {
        cout << "Invalid number of records.\n";
        return 1;
    }
    for (int i = 0; i < n; i++) {
        cout << "Book " << i + 1 << "\n";
        bool dup;
        do {
            readRecord(records[i]);
            dup = false;
            for (int j = 0; j < i; j++) {
                if (records[j].id == records[i].id)
                    dup = true;
            }
            if (dup)
                cout << " Book ID already used, enter again.\n";
        } while (dup);
    }
    do {
        cout << "\n===== Book RECORDS MENU =====\n";
        cout << "1. Insert a new book at a given position\n";
        cout << "2. Delete a book by Book ID\n";
```

```
        cout << "3. Search a book by Book ID\n";
        cout << "4. Display all books\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            cout << "Enter position (1 to " << n + 1 << "): ";
            cin >> pos;
            insertAt(pos);
            break;
        case 2:
            cout << "Enter Book ID to delete: ";
            cin >> id;
            deleteById(id);
            break;
        case 3:
            cout << "Enter Book ID to search: ";
            cin >> id;
            searchById(id);
            break;
        case 4:
            displayAll();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 2 – Daily book return status (R = Returned / N = Not returned) using a singly linked list

a. Insert a status (R/N) at the end of the list
b. Update the status of a given day
c. Calculate the total number of returned books
d. Display the complete status list

Program (C++)

```
/* Set 2 - Daily book return status (R = Returned / N = Not returned) using a singly
   linked list */
// Compile: g++ -std=c++17 -Wall -Wextra set2.cpp -o set2
#include <iostream>
#include <iomanip>
#include <cctype>
#include <cstdlib>
using namespace std;

struct Node {
    int day;           // day number = position in the list
    char status;       // 'R' = Returned, 'N' = Not returned
    Node *next;
};

Node *head = nullptr;

bool validStatus(char c)
{
    return c == 'R' || c == 'N';
}

char readStatus()
{
    char c;
    do {
        cout << "Enter status (R = Returned / N = Not returned): ";
        if (!(cin >> c)) {
            cout << "\nInput ended.\n";
```

```
            exit(0);
        }
        c = toupper((unsigned char)c);
        if (!validStatus(c))
            cout << "Invalid status! Enter only R or N.\n";
    } while (!validStatus(c));
    return c;
}

Node *newNode(char s)
{
    Node *t = new Node;
    t->status = s;
    t->day = 0;
    t->next = nullptr;
    return t;
}
// re-number the days 1,2,3,... after every insertion / deletion
void renumber()
{
    int d = 1;
    for (Node *p = head; p != nullptr; p = p->next)
        p->day = d++;
}

// Insert a status at the end of the list
void insertEnd(char s)
{
    Node *t = newNode(s);
    if (head == nullptr) {
        head = t;
    } else {
        Node *p = head;
        while (p->next != nullptr)
            p = p->next;
        p->next = t;
    }
    renumber();
    cout << "Status '" << s << "' inserted at the end (Day " << t->day << ").\n";
}

// Count the days having the given status
int countStatus(char s)
{
    int c = 0;
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->status == s)
            c++;
    }
    return c;
}

// Display the complete status list
void display()
{
    if (head == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    cout << "\nDay Status\n";
    cout << "----------------\n";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << left << setw(5) << p->day << " " << p->status << "\n";
    cout << "\nList : HEAD -> ";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << p->status << " -> ";
    cout << "NULL\n";
}

void freeList()
{
    while (head != nullptr) {
        Node *p = head;
        head = head->next;
        delete p;
```

```
    }
}
// Update the status of a given day
void updateStatus(int day, char s)
{
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->day == day) {
            p->status = s;
            cout << "Day " << day << " updated to '" << s << "'.\n";
            return;
        }
    }
    cout << "Day " << day << " not found in the list.\n";
}

int main()
{
    int choice, day;
    char s;
    do {
        cout << "\n===== BOOK RETURN STATUS MENU =====\n";
        cout << "1. Insert a status at the end of the list\n";
        cout << "2. Update the status of a given day\n";
        cout << "3. Calculate total number of returned books\n";
        cout << "4. Display the complete status list\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            s = readStatus();
            insertEnd(s);
            break;
        case 2:
            cout << "Enter day number: ";
            cin >> day;
            s = readStatus();
            updateStatus(day, s);
            break;
        case 3:
            cout << "Total number of returned books = "
                 << countStatus('R') << "\n";
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    freeList();
    return 0;
}
```

Set 3 – Daily book return status (R = Returned / N = Not returned) using a singly linked list

a. Insert a status at the middle of the list
b. Delete a status by day from the front
c. Calculate the total number of books not returned
d. Display the complete list

Program (C++)

```
/* Set 3 - Daily book return status (R = Returned / N = Not returned) using a singly
   linked list */
// Compile: g++ -std=c++17 -Wall -Wextra set3.cpp -o set3
#include <iostream>
#include <iomanip>
#include <cctype>
#include <cstdlib>
using namespace std;

struct Node {
    int day;           // day number = position in the list
    char status;       // 'R' = Returned, 'N' = Not returned
    Node *next;
};

Node *head = nullptr;

bool validStatus(char c)
{
    return c == 'R' || c == 'N';
}

char readStatus()
{
    char c;
    do {
        cout << "Enter status (R = Returned / N = Not returned): ";
        if (!(cin >> c)) {
            cout << "\nInput ended.\n";
            exit(0);
        }
        c = toupper((unsigned char)c);
        if (!validStatus(c))
            cout << "Invalid status! Enter only R or N.\n";
    } while (!validStatus(c));
    return c;
}

Node *newNode(char s)
{
    Node *t = new Node;
    t->status = s;
    t->day = 0;
    t->next = nullptr;
    return t;
}
int length()
{
    int c = 0;
    for (Node *p = head; p != nullptr; p = p->next)
        c++;
    return c;
}
// re-number the days 1,2,3,... after every insertion / deletion
void renumber()
{
    int d = 1;
    for (Node *p = head; p != nullptr; p = p->next)
        p->day = d++;
}

// Insert a status at the end of the list
void insertEnd(char s)
{
    Node *t = newNode(s);
    if (head == nullptr) {
```

```
        head = t;
    } else {
        Node *p = head;
        while (p->next != nullptr)
            p = p->next;
        p->next = t;
    }
    renumber();
    cout << "Status '" << s << "' inserted at the end (Day " << t->day << ").\n";
}

// Count the days having the given status
int countStatus(char s)
{
    int c = 0;
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->status == s)
            c++;
    }
    return c;
}

// Display the complete status list
void display()
{
    if (head == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    cout << "\nDay Status\n";
    cout << "----------------\n";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << left << setw(5) << p->day << " " << p->status << "\n";
    cout << "\nList : HEAD -> ";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << p->status << " -> ";
    cout << "NULL\n";
}

void freeList()
{
    while (head != nullptr) {
        Node *p = head;
        head = head->next;
        delete p;
    }
}
// Insert a status in the middle of the list (after the middle node)
void insertMiddle(char s)
{
    Node *t = newNode(s);
    int mid = (length() + 1) / 2;       // position of the middle node
    if (head == nullptr) {
        head = t;
    } else {
        Node *p = head;
        for (int i = 1; i < mid; i++)
            p = p->next;
        t->next = p->next;
        p->next = t;
    }
    renumber();
    cout << "Status '" << s << "' inserted in the middle (Day " << t->day << ").\n";
}

// Delete the status of a given day, searching from the front
void deleteByDay(int day)
{
    Node *p = head, *previous = nullptr;
    while (p != nullptr && p->day != day) {
        previous = p;
        p = p->next;
    }
    if (p == nullptr) {
        cout << "Day " << day << " not found in the list.\n";
```

```
        return;
    }
    if (previous == nullptr)
        head = p->next;
    else
        previous->next = p->next;
    delete p;
    renumber();
    cout << "Day " << day << " deleted.\n";
}

int main()
{
    int choice, day;
    char s;
    do {
        cout << "\n===== BOOK RETURN STATUS MENU =====\n";
        cout << "1. Insert a status at the end of the list (to build the list)\n";
        cout << "2. Insert a status at the middle of the list\n";
        cout << "3. Delete a status by day (searching from the front)\n";
        cout << "4. Calculate total number of books not returned\n";
        cout << "5. Display the complete list\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            s = readStatus();
            insertEnd(s);
            break;
        case 2:
            s = readStatus();
            insertMiddle(s);
            break;
        case 3:
            cout << "Enter day number to delete: ";
            cin >> day;
            deleteByDay(day);
            break;
        case 4:
            cout << "Total number of books not returned = "
                 << countStatus('N') << "\n";
            break;
        case 5:
            display();
            break;
        case 6:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 6);
    freeList();
    return 0;
}
```

Set 4 – Maintain a list of library members sorted by Member ID using a doubly linked list

a. Insert a member so that the list remains sorted
b. Delete a member by Member ID
c. Traverse forward and display all library members
d. Traverse backward and display all library members in reverse order

Program (C++)

```
/* Set 4 - Maintain a list of library members sorted by Member ID using a doubly linked
   list */
// Compile: g++ -std=c++17 -Wall -Wextra set4.cpp -o set4
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

struct Node {
    int id;            // Member ID
    string name;       // Member Name
    Node *prev;
    Node *next;
};

Node *head = nullptr;  // first node (smallest Member ID)
Node *tail = nullptr;  // last node (largest Member ID)

Node *findNode(int id)
{
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->id == id)
            return p;
    }
    return nullptr;
}

// a. Insert a member so that the list remains sorted by Member ID
void insertSorted(int id, const string &name)
{
    if (findNode(id) != nullptr) {
        cout << "Member ID " << id << " already exists. Insertion cancelled.\n";
        return;
    }
    Node *t = new Node;
    t->id = id;
    t->name = name;
    t->prev = t->next = nullptr;
    if (head == nullptr) {                  // empty list
        head = tail = t;
    } else if (id < head->id) {             // insert at beginning
        t->next = head;
        head->prev = t;
        head = t;
    } else {                                // middle or end
        Node *p = head;
        while (p->next != nullptr && p->next->id < id)
            p = p->next;
        t->next = p->next;
        t->prev = p;
        if (p->next != nullptr)
            p->next->prev = t;
        else
            tail = t;
        p->next = t;
    }
    cout << "Member " << id << " inserted.\n";
}

// b. Delete a member by Member ID
void deleteById(int id)
{
    Node *p = findNode(id);
    if (p == nullptr) {
        cout << "Member with Member ID " << id << " not found.\n";
        return;
```

```
    }
    if (p->prev != nullptr)
        p->prev->next = p->next;
    else
        head = p->next;                     // deleting first node
    if (p->next != nullptr)
        p->next->prev = p->prev;
    else
        tail = p->prev;                     // deleting last node
    delete p;
    cout << "Member with Member ID " << id << " deleted.\n";
}

void printHeader()
{
    cout << "\n" << left << setw(12) << "Member ID" << " " << "Member Name" << "\n";
    cout << "------------------------------\n";
}

// c. Traverse forward and display all library members
void displayForward()
{
    if (head == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    printHeader();
    for (Node *p = head; p != nullptr; p = p->next)
        cout << left << setw(12) << p->id << " " << p->name << "\n";
}

// d. Traverse backward and display all library members in reverse order
void displayBackward()
{
    if (tail == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    printHeader();
    for (Node *p = tail; p != nullptr; p = p->prev)
        cout << left << setw(12) << p->id << " " << p->name << "\n";
}

int main()
{
    int choice, id;
    string name;
    do {
        cout << "\n===== Member LIST MENU (DOUBLY LINKED LIST) =====\n";
        cout << "1. Insert a member (list stays sorted)\n";
        cout << "2. Delete a member by Member ID\n";
        cout << "3. Traverse forward and display all library members\n";
        cout << "4. Traverse backward and display all library members\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            cout << "Enter Member ID : ";
            cin >> id;
            cout << "Enter Member Name : ";
            cin >> ws;
            getline(cin, name);
            insertSorted(id, name);
            break;
        case 2:
            cout << "Enter Member ID to delete: ";
            cin >> id;
            deleteById(id);
            break;
        case 3:
            displayForward();
            break;
        case 4:
```

```
            displayBackward();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    while (head != nullptr) {              // release memory
        Node *p = head;
        head = head->next;
        delete p;
    }
    return 0;
}
```

Set 5 – Store the number of books issued of N members using both an array and a linked list

a. Insert the values in both structures (array and linked list)
b. Display the memory representation (index and nodes)
c. Count how many members have number of books issued above 5

Program (C++)

```
/* Set 5 - Store the number of books issued of N members using both an array and a linked
   list */
// Compile: g++ -std=c++17 -Wall -Wextra set5.cpp -o set5
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
using namespace std;

const int MAX = 50;

struct Node {
    int data;
    Node *next;
};

int arr[MAX];                // array representation
Node *head = nullptr;          // linked list representation
Node *tail = nullptr;
int n = 0;

// prints a pointer the way a C program would (null shown as (nil))
string ptrStr(const void *p)
{
    if (p == nullptr)
        return "(nil)";
    ostringstream os;
    os << p;
    return os.str();
}

// a. Insert the value in both the array and the linked list
void insertBoth(int x)
{
    if (n == MAX) {
        cout << "Storage full.\n";
        return;
    }
    arr[n] = x;                // array
    Node *t = new Node;        // linked list (append)
    t->data = x;
    t->next = nullptr;
    if (head == nullptr) {
        head = tail = t;
    } else {
        tail->next = t;
        tail = t;
    }
```

```
    n++;
}

// b. Display the memory representation (index and nodes)
void displayMemory()
{
    cout << "\n--- ARRAY (contiguous memory) ---\n";
    cout << left << setw(7) << "Index" << " " << setw(18) << "Address" << " "
         << "number of books issued" << "\n";
    for (int i = 0; i < n; i++)
        cout << left << setw(7) << i << " " << setw(18) << ptrStr(&arr[i]) << " "
             << arr[i] << "\n";

    cout << "\n--- LINKED LIST (nodes may be scattered in memory) ---\n";
    cout << left << setw(6) << "Node" << " " << setw(18) << "Address" << " " << setw(12) << "Data"
         << " " << "Next" << "\n";
    int i = 1;
    for (Node *p = head; p != nullptr; p = p->next, i++)
        cout << left << setw(6) << i << " " << setw(18) << ptrStr(p) << " " << setw(12) << p->data
             << " " << ptrStr(p->next) << "\n";
    cout << "HEAD -> ";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << "[" << p->data << "] -> ";
    cout << "NULL\n";
}

// c. Count how many members have number of books issued above 5
void countAbove()
{
    int ca = 0, cl = 0;
    for (int i = 0; i < n; i++) {                   // using the array
        if (arr[i] > 5)
            ca++;
    }
    for (Node *p = head; p != nullptr; p = p->next) {   // using the linked list
        if (p->data > 5)
            cl++;
    }
    cout << "Count using array = " << ca << "\n";
    cout << "Count using linked list = " << cl << "\n";
    cout << "Number of members with number of books issued above 5 = " << ca << "\n";
}

int main()
{
    int choice, cnt;
    int x;
    cout << fixed << setprecision(2);
    cout << "Enter number of members N (max " << MAX << "): ";
    cin >> cnt;
    if (cnt < 0 || cnt > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    for (int i = 0; i < cnt; i++) {
        cout << "Enter number of books issued of member " << i + 1 << ": ";
        cin >> x;
        insertBoth(x);
    }
    do {
        cout << "\n===== MENU =====\n";
        cout << "1. Display memory representation (array index and list nodes)\n";
        cout << "2. Count members with number of books issued above 5\n";
        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            displayMemory();
            break;
        case 2:
            countAbove();
            break;
        case 3:
```

```
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 3);
    while (head != nullptr) {
        Node *p = head;
        head = head->next;
        delete p;
    }
    return 0;
}
```

Set 6 – Maintain N patients records (Patient ID, Name, Age) using an array

a. Insert a new patient at a given position
b. Delete a patient by Patient ID
c. Search a patient by Patient ID and display the record
d. Display all patients

Program (C++)

```
/* Set 6 - Maintain N patients records (Patient ID, Name, Age) using an array */
// Compile: g++ -std=c++17 -Wall -Wextra set6.cpp -o set6
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Patient {
    int id;            // Patient ID
    string name;       // Name
    int val;         // Age
};

Patient records[MAX];
int n = 0;             // number of records currently stored

void readRecord(Patient &r)
{
    cout << " Enter Patient ID : ";
    cin >> r.id;
    cout << " Enter Name : ";
    cin >> ws;
    getline(cin, r.name);
    cout << " Enter Age : ";
    cin >> r.val;
}

void printHeader()
{
    cout << "\n" << left << setw(12) << "Patient ID" << " " << setw(25) << "Name"
         << " " << "Age" << "\n";
    cout << "-----------------------------------------------------------\n";
}

void printRecord(const Patient &r)
{
    cout << left << setw(12) << r.id << " " << setw(25) << r.name << " " << r.val << "\n";
}

// returns the index of the record having the given ID, or -1
int findIndex(int id)
{
    for (int i = 0; i < n; i++) {
        if (records[i].id == id)
            return i;
    }
    return -1;
}
```

```
// a. Insert a new patient at a given position (1-based)
void insertAt(int pos)
{
    Patient r;
    if (n == MAX) {
        cout << "Array is full. Cannot insert.\n";
        return;
    }
    if (pos < 1 || pos > n + 1) {
        cout << "Invalid position! Valid range is 1 to " << n + 1 << ".\n";
        return;
    }
    readRecord(r);
    if (findIndex(r.id) != -1) {
        cout << "Patient ID already exists. Insertion cancelled.\n";
        return;
    }
    for (int i = n - 1; i >= pos - 1; i--)      // shift right
        records[i + 1] = records[i];
    records[pos - 1] = r;
    n++;
    cout << "Patient inserted at position " << pos << ".\n";
}

// b. Delete a patient by Patient ID
void deleteById(int id)
{
    int idx = findIndex(id);
    if (idx == -1) {
        cout << "Patient with Patient ID " << id << " not found.\n";
        return;
    }
    for (int i = idx; i < n - 1; i++)           // shift left
        records[i] = records[i + 1];
    n--;
    cout << "Patient with Patient ID " << id << " deleted.\n";
}

// c. Search a patient by Patient ID and display the record
void searchById(int id)
{
    int idx = findIndex(id);
    if (idx == -1) {
        cout << "Patient with Patient ID " << id << " not found.\n";
        return;
    }
    printHeader();
    printRecord(records[idx]);
}

// d. Display all patients
void displayAll()
{
    if (n == 0) {
        cout << "No records to display.\n";
        return;
    }
    printHeader();
    for (int i = 0; i < n; i++)
        printRecord(records[i]);
}

int main()
{
    int choice, pos, id;
    cout << fixed << setprecision(2);
    cout << "Enter number of patients (max " << MAX << "): ";
    cin >> n;
    if (n < 0 || n > MAX) {
        cout << "Invalid number of records.\n";
        return 1;
    }
    for (int i = 0; i < n; i++) {
        cout << "Patient " << i + 1 << "\n";
        bool dup;
```

```
        do {
            readRecord(records[i]);
            dup = false;
            for (int j = 0; j < i; j++) {
                if (records[j].id == records[i].id)
                    dup = true;
            }
            if (dup)
                cout << " Patient ID already used, enter again.\n";
        } while (dup);
    }
    do {
        cout << "\n===== Patient RECORDS MENU =====\n";
        cout << "1. Insert a new patient at a given position\n";
        cout << "2. Delete a patient by Patient ID\n";
        cout << "3. Search a patient by Patient ID\n";
        cout << "4. Display all patients\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            cout << "Enter position (1 to " << n + 1 << "): ";
            cin >> pos;
            insertAt(pos);
            break;
        case 2:
            cout << "Enter Patient ID to delete: ";
            cin >> id;
            deleteById(id);
            break;
        case 3:
            cout << "Enter Patient ID to search: ";
            cin >> id;
            searchById(id);
            break;
        case 4:
            displayAll();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 7 – Daily patient check-up status (C = Completed / M = Missed) using a singly linked list

a. Insert a status (C/M) at the end of the list
b. Update the status of a given day
c. Calculate the total number of completed check-ups
d. Display the complete status list

Program (C++)

```
/* Set 7 - Daily patient check-up status (C = Completed / M = Missed) using a singly
   linked list */
// Compile: g++ -std=c++17 -Wall -Wextra set7.cpp -o set7
#include <iostream>
#include <iomanip>
#include <cctype>
#include <cstdlib>
using namespace std;

struct Node {
    int day;          // day number = position in the list
    char status;      // 'C' = Completed, 'M' = Missed
    Node *next;
};
```

```
Node *head = nullptr;

bool validStatus(char c)
{
    return c == 'C' || c == 'M';
}

char readStatus()
{
    char c;
    do {
        cout << "Enter status (C = Completed / M = Missed): ";
        if (!(cin >> c)) {
            cout << "\nInput ended.\n";
            exit(0);
        }
        c = toupper((unsigned char)c);
        if (!validStatus(c))
            cout << "Invalid status! Enter only C or M.\n";
    } while (!validStatus(c));
    return c;
}

Node *newNode(char s)
{
    Node *t = new Node;
    t->status = s;
    t->day = 0;
    t->next = nullptr;
    return t;
}
// re-number the days 1,2,3,... after every insertion / deletion
void renumber()
{
    int d = 1;
    for (Node *p = head; p != nullptr; p = p->next)
        p->day = d++;
}

// Insert a status at the end of the list
void insertEnd(char s)
{
    Node *t = newNode(s);
    if (head == nullptr) {
        head = t;
    } else {
        Node *p = head;
        while (p->next != nullptr)
            p = p->next;
        p->next = t;
    }
    renumber();
    cout << "Status '" << s << "' inserted at the end (Day " << t->day << ").\n";
}

// Count the days having the given status
int countStatus(char s)
{
    int c = 0;
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->status == s)
            c++;
    }
    return c;
}

// Display the complete status list
void display()
{
    if (head == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    cout << "\nDay Status\n";
    cout << "----------------\n";
```

```
    for (Node *p = head; p != nullptr; p = p->next)
        cout << left << setw(5) << p->day << " " << p->status << "\n";
    cout << "\nList : HEAD -> ";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << p->status << " -> ";
    cout << "NULL\n";
}

void freeList()
{
    while (head != nullptr) {
        Node *p = head;
        head = head->next;
        delete p;
    }
}
// Update the status of a given day
void updateStatus(int day, char s)
{
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->day == day) {
            p->status = s;
            cout << "Day " << day << " updated to '" << s << "'.\n";
            return;
        }
    }
    cout << "Day " << day << " not found in the list.\n";
}

int main()
{
    int choice, day;
    char s;
    do {
        cout << "\n===== PATIENT CHECK-UP STATUS MENU =====\n";
        cout << "1. Insert a status at the end of the list\n";
        cout << "2. Update the status of a given day\n";
        cout << "3. Calculate total number of completed check-ups\n";
        cout << "4. Display the complete status list\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            s = readStatus();
            insertEnd(s);
            break;
        case 2:
            cout << "Enter day number: ";
            cin >> day;
            s = readStatus();
            updateStatus(day, s);
            break;
        case 3:
            cout << "Total number of completed check-ups = "
                 << countStatus('C') << "\n";
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    freeList();
    return 0;
}
```

Set 8 – Daily patient check-up status (C = Completed / M = Missed) using a singly linked list

a. Insert a status at the middle of the list
b. Delete a status by day from the front
c. Calculate the total number of missed check-ups
d. Display the complete list

Program (C++)

```
/* Set 8 - Daily patient check-up status (C = Completed / M = Missed) using a singly
   linked list */
// Compile: g++ -std=c++17 -Wall -Wextra set8.cpp -o set8
#include <iostream>
#include <iomanip>
#include <cctype>
#include <cstdlib>
using namespace std;

struct Node {
    int day;           // day number = position in the list
    char status;       // 'C' = Completed, 'M' = Missed
    Node *next;
};

Node *head = nullptr;

bool validStatus(char c)
{
    return c == 'C' || c == 'M';
}

char readStatus()
{
    char c;
    do {
        cout << "Enter status (C = Completed / M = Missed): ";
        if (!(cin >> c)) {
            cout << "\nInput ended.\n";
            exit(0);
        }
        c = toupper((unsigned char)c);
        if (!validStatus(c))
            cout << "Invalid status! Enter only C or M.\n";
    } while (!validStatus(c));
    return c;
}

Node *newNode(char s)
{
    Node *t = new Node;
    t->status = s;
    t->day = 0;
    t->next = nullptr;
    return t;
}
int length()
{
    int c = 0;
    for (Node *p = head; p != nullptr; p = p->next)
        c++;
    return c;
}
// re-number the days 1,2,3,... after every insertion / deletion
void renumber()
{
    int d = 1;
    for (Node *p = head; p != nullptr; p = p->next)
        p->day = d++;
}

// Insert a status at the end of the list
void insertEnd(char s)
{
    Node *t = newNode(s);
    if (head == nullptr) {
```

```
        head = t;
    } else {
        Node *p = head;
        while (p->next != nullptr)
            p = p->next;
        p->next = t;
    }
    renumber();
    cout << "Status '" << s << "' inserted at the end (Day " << t->day << ").\n";
}

// Count the days having the given status
int countStatus(char s)
{
    int c = 0;
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->status == s)
            c++;
    }
    return c;
}

// Display the complete status list
void display()
{
    if (head == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    cout << "\nDay Status\n";
    cout << "----------------\n";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << left << setw(5) << p->day << " " << p->status << "\n";
    cout << "\nList : HEAD -> ";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << p->status << " -> ";
    cout << "NULL\n";
}

void freeList()
{
    while (head != nullptr) {
        Node *p = head;
        head = head->next;
        delete p;
    }
}
// Insert a status in the middle of the list (after the middle node)
void insertMiddle(char s)
{
    Node *t = newNode(s);
    int mid = (length() + 1) / 2;       // position of the middle node
    if (head == nullptr) {
        head = t;
    } else {
        Node *p = head;
        for (int i = 1; i < mid; i++)
            p = p->next;
        t->next = p->next;
        p->next = t;
    }
    renumber();
    cout << "Status '" << s << "' inserted in the middle (Day " << t->day << ").\n";
}

// Delete the status of a given day, searching from the front
void deleteByDay(int day)
{
    Node *p = head, *previous = nullptr;
    while (p != nullptr && p->day != day) {
        previous = p;
        p = p->next;
    }
    if (p == nullptr) {
        cout << "Day " << day << " not found in the list.\n";
```

```
        return;
    }
    if (previous == nullptr)
        head = p->next;
    else
        previous->next = p->next;
    delete p;
    renumber();
    cout << "Day " << day << " deleted.\n";
}

int main()
{
    int choice, day;
    char s;
    do {
        cout << "\n===== PATIENT CHECK-UP STATUS MENU =====\n";
        cout << "1. Insert a status at the end of the list (to build the list)\n";
        cout << "2. Insert a status at the middle of the list\n";
        cout << "3. Delete a status by day (searching from the front)\n";
        cout << "4. Calculate total number of missed check-ups\n";
        cout << "5. Display the complete list\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            s = readStatus();
            insertEnd(s);
            break;
        case 2:
            s = readStatus();
            insertMiddle(s);
            break;
        case 3:
            cout << "Enter day number to delete: ";
            cin >> day;
            deleteByDay(day);
            break;
        case 4:
            cout << "Total number of missed check-ups = "
                 << countStatus('M') << "\n";
            break;
        case 5:
            display();
            break;
        case 6:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 6);
    freeList();
    return 0;
}
```

Set 9 – Maintain a list of doctors sorted by Doctor ID using a doubly linked list

a. Insert a doctor so that the list remains sorted
b. Delete a doctor by Doctor ID
c. Traverse forward and display all doctors
d. Traverse backward and display all doctors in reverse order

Program (C++)

```
/* Set 9 - Maintain a list of doctors sorted by Doctor ID using a doubly linked list */
// Compile: g++ -std=c++17 -Wall -Wextra set9.cpp -o set9
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

struct Node {
    int id;            // Doctor ID
    string name;       // Doctor Name
    Node *prev;
    Node *next;
};

Node *head = nullptr;  // first node (smallest Doctor ID)
Node *tail = nullptr;  // last node (largest Doctor ID)

Node *findNode(int id)
{
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->id == id)
            return p;
    }
    return nullptr;
}

// a. Insert a doctor so that the list remains sorted by Doctor ID
void insertSorted(int id, const string &name)
{
    if (findNode(id) != nullptr) {
        cout << "Doctor ID " << id << " already exists. Insertion cancelled.\n";
        return;
    }
    Node *t = new Node;
    t->id = id;
    t->name = name;
    t->prev = t->next = nullptr;
    if (head == nullptr) {                  // empty list
        head = tail = t;
    } else if (id < head->id) {             // insert at beginning
        t->next = head;
        head->prev = t;
        head = t;
    } else {                                // middle or end
        Node *p = head;
        while (p->next != nullptr && p->next->id < id)
            p = p->next;
        t->next = p->next;
        t->prev = p;
        if (p->next != nullptr)
            p->next->prev = t;
        else
            tail = t;
        p->next = t;
    }
    cout << "Doctor " << id << " inserted.\n";
}

// b. Delete a doctor by Doctor ID
void deleteById(int id)
{
    Node *p = findNode(id);
    if (p == nullptr) {
        cout << "Doctor with Doctor ID " << id << " not found.\n";
        return;
    }
```

```
    if (p->prev != nullptr)
        p->prev->next = p->next;
    else
        head = p->next;                     // deleting first node
    if (p->next != nullptr)
        p->next->prev = p->prev;
    else
        tail = p->prev;                     // deleting last node
    delete p;
    cout << "Doctor with Doctor ID " << id << " deleted.\n";
}

void printHeader()
{
    cout << "\n" << left << setw(12) << "Doctor ID" << " " << "Doctor Name" << "\n";
    cout << "------------------------------\n";
}

// c. Traverse forward and display all doctors
void displayForward()
{
    if (head == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    printHeader();
    for (Node *p = head; p != nullptr; p = p->next)
        cout << left << setw(12) << p->id << " " << p->name << "\n";
}

// d. Traverse backward and display all doctors in reverse order
void displayBackward()
{
    if (tail == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    printHeader();
    for (Node *p = tail; p != nullptr; p = p->prev)
        cout << left << setw(12) << p->id << " " << p->name << "\n";
}

int main()
{
    int choice, id;
    string name;
    do {
        cout << "\n===== Doctor LIST MENU (DOUBLY LINKED LIST) =====\n";
        cout << "1. Insert a doctor (list stays sorted)\n";
        cout << "2. Delete a doctor by Doctor ID\n";
        cout << "3. Traverse forward and display all doctors\n";
        cout << "4. Traverse backward and display all doctors\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            cout << "Enter Doctor ID : ";
            cin >> id;
            cout << "Enter Doctor Name : ";
            cin >> ws;
            getline(cin, name);
            insertSorted(id, name);
            break;
        case 2:
            cout << "Enter Doctor ID to delete: ";
            cin >> id;
            deleteById(id);
            break;
        case 3:
            displayForward();
            break;
        case 4:
            displayBackward();
```

```
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    while (head != nullptr) {              // release memory
        Node *p = head;
        head = head->next;
        delete p;
    }
    return 0;
}
```

Set 10 – Store the body temperature (F) of N patients using both an array and a linked list

a. Insert the values in both structures (array and linked list)
b. Display the memory representation (index and nodes)
c. Count how many patients have body temperature (F) above 100 F

Program (C++)

```
/* Set 10 - Store the body temperature (F) of N patients using both an array and a linked
   list */
// Compile: g++ -std=c++17 -Wall -Wextra set10.cpp -o set10
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
using namespace std;

const int MAX = 50;

struct Node {
    float data;
    Node *next;
};

float arr[MAX];                // array representation
Node *head = nullptr;          // linked list representation
Node *tail = nullptr;
int n = 0;

// prints a pointer the way a C program would (null shown as (nil))
string ptrStr(const void *p)
{
    if (p == nullptr)
        return "(nil)";
    ostringstream os;
    os << p;
    return os.str();
}

// a. Insert the value in both the array and the linked list
void insertBoth(float x)
{
    if (n == MAX) {
        cout << "Storage full.\n";
        return;
    }
    arr[n] = x;                // array
    Node *t = new Node;        // linked list (append)
    t->data = x;
    t->next = nullptr;
    if (head == nullptr) {
        head = tail = t;
    } else {
        tail->next = t;
        tail = t;
    }
    n++;
```

```
}

// b. Display the memory representation (index and nodes)
void displayMemory()
{
    cout << "\n--- ARRAY (contiguous memory) ---\n";
    cout << left << setw(7) << "Index" << " " << setw(18) << "Address" << " "
         << "body temperature (F)" << "\n";
    for (int i = 0; i < n; i++)
        cout << left << setw(7) << i << " " << setw(18) << ptrStr(&arr[i]) << " "
             << arr[i] << "\n";

    cout << "\n--- LINKED LIST (nodes may be scattered in memory) ---\n";
    cout << left << setw(6) << "Node" << " " << setw(18) << "Address" << " " << setw(12) << "Data"
         << " " << "Next" << "\n";
    int i = 1;
    for (Node *p = head; p != nullptr; p = p->next, i++)
        cout << left << setw(6) << i << " " << setw(18) << ptrStr(p) << " " << setw(12) << p->data
             << " " << ptrStr(p->next) << "\n";
    cout << "HEAD -> ";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << "[" << p->data << "] -> ";
    cout << "NULL\n";
}

// c. Count how many patients have body temperature (F) above 100
void countAbove()
{
    int ca = 0, cl = 0;
    for (int i = 0; i < n; i++) {                   // using the array
        if (arr[i] > 100)
            ca++;
    }
    for (Node *p = head; p != nullptr; p = p->next) {   // using the linked list
        if (p->data > 100)
            cl++;
    }
    cout << "Count using array = " << ca << "\n";
    cout << "Count using linked list = " << cl << "\n";
    cout << "Number of patients with body temperature (F) above 100 = " << ca << "\n";
}

int main()
{
    int choice, cnt;
    float x;
    cout << fixed << setprecision(2);
    cout << "Enter number of patients N (max " << MAX << "): ";
    cin >> cnt;
    if (cnt < 0 || cnt > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    for (int i = 0; i < cnt; i++) {
        cout << "Enter body temperature (F) of patient " << i + 1 << ": ";
        cin >> x;
        insertBoth(x);
    }
    do {
        cout << "\n===== MENU =====\n";
        cout << "1. Display memory representation (array index and list nodes)\n";
        cout << "2. Count patients with body temperature (F) above 100\n";
        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            displayMemory();
            break;
        case 2:
            countAbove();
            break;
        case 3:
            cout << "Exiting...\n";
```

```
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 3);
    while (head != nullptr) {
        Node *p = head;
        head = head->next;
        delete p;
    }
    return 0;
}
```

Set 11 – Maintain N products records (Product ID, Name, Quantity) using an array

a. Insert a new product at a given position
b. Delete a product by Product ID
c. Search a product by Product ID and display the record
d. Display all products

Program (C++)

```
/* Set 11 - Maintain N products records (Product ID, Name, Quantity) using an array */
// Compile: g++ -std=c++17 -Wall -Wextra set11.cpp -o set11
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Product {
    int id;            // Product ID
    string name;       // Name
    int val;         // Quantity
};

Product records[MAX];
int n = 0;             // number of records currently stored

void readRecord(Product &r)
{
    cout << " Enter Product ID : ";
    cin >> r.id;
    cout << " Enter Name : ";
    cin >> ws;
    getline(cin, r.name);
    cout << " Enter Quantity : ";
    cin >> r.val;
}

void printHeader()
{
    cout << "\n" << left << setw(12) << "Product ID" << " " << setw(25) << "Name"
         << " " << "Quantity" << "\n";
    cout << "-----------------------------------------------------------\n";
}

void printRecord(const Product &r)
{
    cout << left << setw(12) << r.id << " " << setw(25) << r.name << " " << r.val << "\n";
}

// returns the index of the record having the given ID, or -1
int findIndex(int id)
{
    for (int i = 0; i < n; i++) {
        if (records[i].id == id)
            return i;
    }
    return -1;
}
// a. Insert a new product at a given position (1-based)
```

```
void insertAt(int pos)
{
    Product r;
    if (n == MAX) {
        cout << "Array is full. Cannot insert.\n";
        return;
    }
    if (pos < 1 || pos > n + 1) {
        cout << "Invalid position! Valid range is 1 to " << n + 1 << ".\n";
        return;
    }
    readRecord(r);
    if (findIndex(r.id) != -1) {
        cout << "Product ID already exists. Insertion cancelled.\n";
        return;
    }
    for (int i = n - 1; i >= pos - 1; i--)      // shift right
        records[i + 1] = records[i];
    records[pos - 1] = r;
    n++;
    cout << "Product inserted at position " << pos << ".\n";
}

// b. Delete a product by Product ID
void deleteById(int id)
{
    int idx = findIndex(id);
    if (idx == -1) {
        cout << "Product with Product ID " << id << " not found.\n";
        return;
    }
    for (int i = idx; i < n - 1; i++)           // shift left
        records[i] = records[i + 1];
    n--;
    cout << "Product with Product ID " << id << " deleted.\n";
}

// c. Search a product by Product ID and display the record
void searchById(int id)
{
    int idx = findIndex(id);
    if (idx == -1) {
        cout << "Product with Product ID " << id << " not found.\n";
        return;
    }
    printHeader();
    printRecord(records[idx]);
}

// d. Display all products
void displayAll()
{
    if (n == 0) {
        cout << "No records to display.\n";
        return;
    }
    printHeader();
    for (int i = 0; i < n; i++)
        printRecord(records[i]);
}

int main()
{
    int choice, pos, id;
    cout << fixed << setprecision(2);
    cout << "Enter number of products (max " << MAX << "): ";
    cin >> n;
    if (n < 0 || n > MAX) {
        cout << "Invalid number of records.\n";
        return 1;
    }
    for (int i = 0; i < n; i++) {
        cout << "Product " << i + 1 << "\n";
        bool dup;
        do {
```

```
            readRecord(records[i]);
            dup = false;
            for (int j = 0; j < i; j++) {
                if (records[j].id == records[i].id)
                    dup = true;
            }
            if (dup)
                cout << " Product ID already used, enter again.\n";
        } while (dup);
    }
    do {
        cout << "\n===== Product RECORDS MENU =====\n";
        cout << "1. Insert a new product at a given position\n";
        cout << "2. Delete a product by Product ID\n";
        cout << "3. Search a product by Product ID\n";
        cout << "4. Display all products\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            cout << "Enter position (1 to " << n + 1 << "): ";
            cin >> pos;
            insertAt(pos);
            break;
        case 2:
            cout << "Enter Product ID to delete: ";
            cin >> id;
            deleteById(id);
            break;
        case 3:
            cout << "Enter Product ID to search: ";
            cin >> id;
            searchById(id);
            break;
        case 4:
            displayAll();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 12 – Daily product stock status (A = Available / O = Out of stock) using a singly linked list

a. Insert a status (A/O) at the end of the list
b. Update the status of a given day
c. Calculate the total number of days the product was available
d. Display the complete status list

Program (C++)

```
/* Set 12 - Daily product stock status (A = Available / O = Out of stock) using a singly
   linked list */
// Compile: g++ -std=c++17 -Wall -Wextra set12.cpp -o set12
#include <iostream>
#include <iomanip>
#include <cctype>
#include <cstdlib>
using namespace std;

struct Node {
    int day;           // day number = position in the list
    char status;       // 'A' = Available, 'O' = Out of stock
    Node *next;
```

```
};

Node *head = nullptr;

bool validStatus(char c)
{
    return c == 'A' || c == 'O';
}

char readStatus()
{
    char c;
    do {
        cout << "Enter status (A = Available / O = Out of stock): ";
        if (!(cin >> c)) {
            cout << "\nInput ended.\n";
            exit(0);
        }
        c = toupper((unsigned char)c);
        if (!validStatus(c))
            cout << "Invalid status! Enter only A or O.\n";
    } while (!validStatus(c));
    return c;
}

Node *newNode(char s)
{
    Node *t = new Node;
    t->status = s;
    t->day = 0;
    t->next = nullptr;
    return t;
}
// re-number the days 1,2,3,... after every insertion / deletion
void renumber()
{
    int d = 1;
    for (Node *p = head; p != nullptr; p = p->next)
        p->day = d++;
}

// Insert a status at the end of the list
void insertEnd(char s)
{
    Node *t = newNode(s);
    if (head == nullptr) {
        head = t;
    } else {
        Node *p = head;
        while (p->next != nullptr)
            p = p->next;
        p->next = t;
    }
    renumber();
    cout << "Status '" << s << "' inserted at the end (Day " << t->day << ").\n";
}

// Count the days having the given status
int countStatus(char s)
{
    int c = 0;
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->status == s)
            c++;
    }
    return c;
}

// Display the complete status list
void display()
{
    if (head == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
```

```
    cout << "\nDay Status\n";
    cout << "-----------------\n";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << left << setw(5) << p->day << " " << p->status << "\n";
    cout << "\nList : HEAD -> ";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << p->status << " -> ";
    cout << "NULL\n";
}

void freeList()
{
    while (head != nullptr) {
        Node *p = head;
        head = head->next;
        delete p;
    }
}
// Update the status of a given day
void updateStatus(int day, char s)
{
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->day == day) {
            p->status = s;
            cout << "Day " << day << " updated to '" << s << "'.\n";
            return;
        }
    }
    cout << "Day " << day << " not found in the list.\n";
}

int main()
{
    int choice, day;
    char s;
    do {
        cout << "\n===== PRODUCT STOCK STATUS MENU =====\n";
        cout << "1. Insert a status at the end of the list\n";
        cout << "2. Update the status of a given day\n";
        cout << "3. Calculate total number of days the product was available\n";
        cout << "4. Display the complete status list\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            s = readStatus();
            insertEnd(s);
            break;
        case 2:
            cout << "Enter day number: ";
            cin >> day;
            s = readStatus();
            updateStatus(day, s);
            break;
        case 3:
            cout << "Total number of days the product was available = "
                 << countStatus('A') << "\n";
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    freeList();
    return 0;
}
```

Set 13 – Daily product stock status (A = Available / O = Out of stock) using a singly linked list

a. Insert a status at the middle of the list
b. Delete a status by day from the front
c. Calculate the total number of out-of-stock days
d. Display the complete list

Program (C++)

```
/* Set 13 - Daily product stock status (A = Available / O = Out of stock) using a singly
   linked list */
// Compile: g++ -std=c++17 -Wall -Wextra set13.cpp -o set13
#include <iostream>
#include <iomanip>
#include <cctype>
#include <cstdlib>
using namespace std;

struct Node {
    int day;           // day number = position in the list
    char status;       // 'A' = Available, 'O' = Out of stock
    Node *next;
};

Node *head = nullptr;

bool validStatus(char c)
{
    return c == 'A' || c == 'O';
}

char readStatus()
{
    char c;
    do {
        cout << "Enter status (A = Available / O = Out of stock): ";
        if (!(cin >> c)) {
            cout << "\nInput ended.\n";
            exit(0);
        }
        c = toupper((unsigned char)c);
        if (!validStatus(c))
            cout << "Invalid status! Enter only A or O.\n";
    } while (!validStatus(c));
    return c;
}

Node *newNode(char s)
{
    Node *t = new Node;
    t->status = s;
    t->day = 0;
    t->next = nullptr;
    return t;
}
int length()
{
    int c = 0;
    for (Node *p = head; p != nullptr; p = p->next)
        c++;
    return c;
}
// re-number the days 1,2,3,... after every insertion / deletion
void renumber()
{
    int d = 1;
    for (Node *p = head; p != nullptr; p = p->next)
        p->day = d++;
}
// Insert a status at the end of the list
```

```
void insertEnd(char s)
{
    Node *t = newNode(s);
    if (head == nullptr) {
        head = t;
    } else {
        Node *p = head;
        while (p->next != nullptr)
            p = p->next;
        p->next = t;
    }
    renumber();
    cout << "Status '" << s << "' inserted at the end (Day " << t->day << ").\n";
}

// Count the days having the given status
int countStatus(char s)
{
    int c = 0;
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->status == s)
            c++;
    }
    return c;
}

// Display the complete status list
void display()
{
    if (head == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    cout << "\nDay Status\n";
    cout << "----------------\n";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << left << setw(5) << p->day << " " << p->status << "\n";
    cout << "\nList : HEAD -> ";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << p->status << " -> ";
    cout << "NULL\n";
}

void freeList()
{
    while (head != nullptr) {
        Node *p = head;
        head = head->next;
        delete p;
    }
}
// Insert a status in the middle of the list (after the middle node)
void insertMiddle(char s)
{
    Node *t = newNode(s);
    int mid = (length() + 1) / 2;       // position of the middle node
    if (head == nullptr) {
        head = t;
    } else {
        Node *p = head;
        for (int i = 1; i < mid; i++)
            p = p->next;
        t->next = p->next;
        p->next = t;
    }
    renumber();
    cout << "Status '" << s << "' inserted in the middle (Day " << t->day << ").\n";
}

// Delete the status of a given day, searching from the front
void deleteByDay(int day)
{
    Node *p = head, *previous = nullptr;
    while (p != nullptr && p->day != day) {
        previous = p;
```

```
        p = p->next;
    }
    if (p == nullptr) {
        cout << "Day " << day << " not found in the list.\n";
        return;
    }
    if (previous == nullptr)
        head = p->next;
    else
        previous->next = p->next;
    delete p;
    renumber();
    cout << "Day " << day << " deleted.\n";
}

int main()
{
    int choice, day;
    char s;
    do {
        cout << "\n===== PRODUCT STOCK STATUS MENU =====\n";
        cout << "1. Insert a status at the end of the list (to build the list)\n";
        cout << "2. Insert a status at the middle of the list\n";
        cout << "3. Delete a status by day (searching from the front)\n";
        cout << "4. Calculate total number of out-of-stock days\n";
        cout << "5. Display the complete list\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            s = readStatus();
            insertEnd(s);
            break;
        case 2:
            s = readStatus();
            insertMiddle(s);
            break;
        case 3:
            cout << "Enter day number to delete: ";
            cin >> day;
            deleteByDay(day);
            break;
        case 4:
            cout << "Total number of out-of-stock days = "
                 << countStatus('O') << "\n";
            break;
        case 5:
            display();
            break;
        case 6:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 6);
    freeList();
    return 0;
}
```

Set 14 – Maintain a list of suppliers sorted by Supplier ID using a doubly linked list

a. Insert a supplier so that the list remains sorted
b. Delete a supplier by Supplier ID
c. Traverse forward and display all suppliers
d. Traverse backward and display all suppliers in reverse order

Program (C++)

```
/* Set 14 - Maintain a list of suppliers sorted by Supplier ID using a doubly linked list */
// Compile: g++ -std=c++17 -Wall -Wextra set14.cpp -o set14
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

struct Node {
    int id;            // Supplier ID
    string name;       // Supplier Name
    Node *prev;
    Node *next;
};

Node *head = nullptr;  // first node (smallest Supplier ID)
Node *tail = nullptr;  // last node (largest Supplier ID)

Node *findNode(int id)
{
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->id == id)
            return p;
    }
    return nullptr;
}

// a. Insert a supplier so that the list remains sorted by Supplier ID
void insertSorted(int id, const string &name)
{
    if (findNode(id) != nullptr) {
        cout << "Supplier ID " << id << " already exists. Insertion cancelled.\n";
        return;
    }
    Node *t = new Node;
    t->id = id;
    t->name = name;
    t->prev = t->next = nullptr;
    if (head == nullptr) {                  // empty list
        head = tail = t;
    } else if (id < head->id) {             // insert at beginning
        t->next = head;
        head->prev = t;
        head = t;
    } else {                                // middle or end
        Node *p = head;
        while (p->next != nullptr && p->next->id < id)
            p = p->next;
        t->next = p->next;
        t->prev = p;
        if (p->next != nullptr)
            p->next->prev = t;
        else
            tail = t;
        p->next = t;
    }
    cout << "Supplier " << id << " inserted.\n";
}

// b. Delete a supplier by Supplier ID
void deleteById(int id)
{
    Node *p = findNode(id);
    if (p == nullptr) {
        cout << "Supplier with Supplier ID " << id << " not found.\n";
        return;
    }
```

```
    if (p->prev != nullptr)
        p->prev->next = p->next;
    else
        head = p->next;                     // deleting first node
    if (p->next != nullptr)
        p->next->prev = p->prev;
    else
        tail = p->prev;                     // deleting last node
    delete p;
    cout << "Supplier with Supplier ID " << id << " deleted.\n";
}

void printHeader()
{
    cout << "\n" << left << setw(12) << "Supplier ID" << " " << "Supplier Name" << "\n";
    cout << "------------------------------\n";
}

// c. Traverse forward and display all suppliers
void displayForward()
{
    if (head == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    printHeader();
    for (Node *p = head; p != nullptr; p = p->next)
        cout << left << setw(12) << p->id << " " << p->name << "\n";
}

// d. Traverse backward and display all suppliers in reverse order
void displayBackward()
{
    if (tail == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    printHeader();
    for (Node *p = tail; p != nullptr; p = p->prev)
        cout << left << setw(12) << p->id << " " << p->name << "\n";
}

int main()
{
    int choice, id;
    string name;
    do {
        cout << "\n===== Supplier LIST MENU (DOUBLY LINKED LIST) =====\n";
        cout << "1. Insert a supplier (list stays sorted)\n";
        cout << "2. Delete a supplier by Supplier ID\n";
        cout << "3. Traverse forward and display all suppliers\n";
        cout << "4. Traverse backward and display all suppliers\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            cout << "Enter Supplier ID : ";
            cin >> id;
            cout << "Enter Supplier Name : ";
            cin >> ws;
            getline(cin, name);
            insertSorted(id, name);
            break;
        case 2:
            cout << "Enter Supplier ID to delete: ";
            cin >> id;
            deleteById(id);
            break;
        case 3:
            displayForward();
            break;
        case 4:
            displayBackward();
```

```
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    while (head != nullptr) {              // release memory
        Node *p = head;
        head = head->next;
        delete p;
    }
    return 0;
}
```

Set 15 – Store the monthly sales units of N products using both an array and a linked list

a. Insert the values in both structures (array and linked list)
b. Display the memory representation (index and nodes)
c. Count how many products have monthly sales units above 500 units

Program (C++)

```
/* Set 15 - Store the monthly sales units of N products using both an array and a linked
   list */
// Compile: g++ -std=c++17 -Wall -Wextra set15.cpp -o set15
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
using namespace std;

const int MAX = 50;

struct Node {
    int data;
    Node *next;
};

int arr[MAX];                // array representation
Node *head = nullptr;          // linked list representation
Node *tail = nullptr;
int n = 0;

// prints a pointer the way a C program would (null shown as (nil))
string ptrStr(const void *p)
{
    if (p == nullptr)
        return "(nil)";
    ostringstream os;
    os << p;
    return os.str();
}

// a. Insert the value in both the array and the linked list
void insertBoth(int x)
{
    if (n == MAX) {
        cout << "Storage full.\n";
        return;
    }
    arr[n] = x;                // array
    Node *t = new Node;        // linked list (append)
    t->data = x;
    t->next = nullptr;
    if (head == nullptr) {
        head = tail = t;
    } else {
        tail->next = t;
        tail = t;
    }
    n++;
```

```
}

// b. Display the memory representation (index and nodes)
void displayMemory()
{
    cout << "\n--- ARRAY (contiguous memory) ---\n";
    cout << left << setw(7) << "Index" << " " << setw(18) << "Address" << " "
         << "monthly sales units" << "\n";
    for (int i = 0; i < n; i++)
        cout << left << setw(7) << i << " " << setw(18) << ptrStr(&arr[i]) << " "
             << arr[i] << "\n";

    cout << "\n--- LINKED LIST (nodes may be scattered in memory) ---\n";
    cout << left << setw(6) << "Node" << " " << setw(18) << "Address" << " " << setw(12) << "Data"
         << " " << "Next" << "\n";
    int i = 1;
    for (Node *p = head; p != nullptr; p = p->next, i++)
        cout << left << setw(6) << i << " " << setw(18) << ptrStr(p) << " " << setw(12) << p->data
             << " " << ptrStr(p->next) << "\n";
    cout << "HEAD -> ";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << "[" << p->data << "] -> ";
    cout << "NULL\n";
}

// c. Count how many products have monthly sales units above 500
void countAbove()
{
    int ca = 0, cl = 0;
    for (int i = 0; i < n; i++) {                   // using the array
        if (arr[i] > 500)
            ca++;
    }
    for (Node *p = head; p != nullptr; p = p->next) {   // using the linked list
        if (p->data > 500)
            cl++;
    }
    cout << "Count using array = " << ca << "\n";
    cout << "Count using linked list = " << cl << "\n";
    cout << "Number of products with monthly sales units above 500 = " << ca << "\n";
}

int main()
{
    int choice, cnt;
    int x;
    cout << fixed << setprecision(2);
    cout << "Enter number of products N (max " << MAX << "): ";
    cin >> cnt;
    if (cnt < 0 || cnt > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    for (int i = 0; i < cnt; i++) {
        cout << "Enter monthly sales units of product " << i + 1 << ": ";
        cin >> x;
        insertBoth(x);
    }
    do {
        cout << "\n===== MENU =====\n";
        cout << "1. Display memory representation (array index and list nodes)\n";
        cout << "2. Count products with monthly sales units above 500\n";
        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            displayMemory();
            break;
        case 2:
            countAbove();
            break;
        case 3:
            cout << "Exiting...\n";
```

```
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 3);
    while (head != nullptr) {
        Node *p = head;
        head = head->next;
        delete p;
    }
    return 0;
}
```

Set 16 – Maintain N customers records (Account No, Name, Balance) using an array

a. Insert a new customer at a given position
b. Delete a customer by Account No
c. Search a customer by Account No and display the record
d. Display all customers

Program (C++)

```
/* Set 16 - Maintain N customers records (Account No, Name, Balance) using an array */
// Compile: g++ -std=c++17 -Wall -Wextra set16.cpp -o set16
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Customer {
    int id;            // Account No
    string name;       // Name
    float val;         // Balance
};

Customer records[MAX];
int n = 0;             // number of records currently stored

void readRecord(Customer &r)
{
    cout << " Enter Account No : ";
    cin >> r.id;
    cout << " Enter Name : ";
    cin >> ws;
    getline(cin, r.name);
    cout << " Enter Balance : ";
    cin >> r.val;
}

void printHeader()
{
    cout << "\n" << left << setw(12) << "Account No" << " " << setw(25) << "Name"
         << " " << "Balance" << "\n";
    cout << "-------------------------------------------------------\n";
}

void printRecord(const Customer &r)
{
    cout << left << setw(12) << r.id << " " << setw(25) << r.name << " " << r.val << "\n";
}

// returns the index of the record having the given ID, or -1
int findIndex(int id)
{
    for (int i = 0; i < n; i++) {
        if (records[i].id == id)
            return i;
    }
    return -1;
}
// a. Insert a new customer at a given position (1-based)
```

```
void insertAt(int pos)
{
    Customer r;
    if (n == MAX) {
        cout << "Array is full. Cannot insert.\n";
        return;
    }
    if (pos < 1 || pos > n + 1) {
        cout << "Invalid position! Valid range is 1 to " << n + 1 << ".\n";
        return;
    }
    readRecord(r);
    if (findIndex(r.id) != -1) {
        cout << "Account No already exists. Insertion cancelled.\n";
        return;
    }
    for (int i = n - 1; i >= pos - 1; i--)      // shift right
        records[i + 1] = records[i];
    records[pos - 1] = r;
    n++;
    cout << "Customer inserted at position " << pos << ".\n";
}

// b. Delete a customer by Account No
void deleteById(int id)
{
    int idx = findIndex(id);
    if (idx == -1) {
        cout << "Customer with Account No " << id << " not found.\n";
        return;
    }
    for (int i = idx; i < n - 1; i++)           // shift left
        records[i] = records[i + 1];
    n--;
    cout << "Customer with Account No " << id << " deleted.\n";
}

// c. Search a customer by Account No and display the record
void searchById(int id)
{
    int idx = findIndex(id);
    if (idx == -1) {
        cout << "Customer with Account No " << id << " not found.\n";
        return;
    }
    printHeader();
    printRecord(records[idx]);
}

// d. Display all customers
void displayAll()
{
    if (n == 0) {
        cout << "No records to display.\n";
        return;
    }
    printHeader();
    for (int i = 0; i < n; i++)
        printRecord(records[i]);
}

int main()
{
    int choice, pos, id;
    cout << fixed << setprecision(2);
    cout << "Enter number of customers (max " << MAX << "): ";
    cin >> n;
    if (n < 0 || n > MAX) {
        cout << "Invalid number of records.\n";
        return 1;
    }
    for (int i = 0; i < n; i++) {
        cout << "Customer " << i + 1 << "\n";
        bool dup;
        do {
```

```
            readRecord(records[i]);
            dup = false;
            for (int j = 0; j < i; j++) {
                if (records[j].id == records[i].id)
                    dup = true;
            }
            if (dup)
                cout << " Account No already used, enter again.\n";
        } while (dup);
    }
    do {
        cout << "\n===== Customer RECORDS MENU =====\n";
        cout << "1. Insert a new customer at a given position\n";
        cout << "2. Delete a customer by Account No\n";
        cout << "3. Search a customer by Account No\n";
        cout << "4. Display all customers\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            cout << "Enter position (1 to " << n + 1 << "): ";
            cin >> pos;
            insertAt(pos);
            break;
        case 2:
            cout << "Enter Account No to delete: ";
            cin >> id;
            deleteById(id);
            break;
        case 3:
            cout << "Enter Account No to search: ";
            cin >> id;
            searchById(id);
            break;
        case 4:
            displayAll();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 17 – Daily EMI payment status (P = Paid / D = Due) using a singly linked list

a. Insert a status (P/D) at the end of the list
b. Update the status of a given day
c. Calculate the total number of paid EMIs
d. Display the complete status list

Program (C++)

```
/* Set 17 - Daily EMI payment status (P = Paid / D = Due) using a singly linked list */
// Compile: g++ -std=c++17 -Wall -Wextra set17.cpp -o set17
#include <iostream>
#include <iomanip>
#include <cctype>
#include <cstdlib>
using namespace std;

struct Node {
    int day;           // day number = position in the list
    char status;       // 'P' = Paid, 'D' = Due
    Node *next;
};

Node *head = nullptr;
```

```
bool validStatus(char c)
{
    return c == 'P' || c == 'D';
}

char readStatus()
{
    char c;
    do {
        cout << "Enter status (P = Paid / D = Due): ";
        if (!(cin >> c)) {
            cout << "\nInput ended.\n";
            exit(0);
        }
        c = toupper((unsigned char)c);
        if (!validStatus(c))
            cout << "Invalid status! Enter only P or D.\n";
    } while (!validStatus(c));
    return c;
}

Node *newNode(char s)
{
    Node *t = new Node;
    t->status = s;
    t->day = 0;
    t->next = nullptr;
    return t;
}
// re-number the days 1,2,3,... after every insertion / deletion
void renumber()
{
    int d = 1;
    for (Node *p = head; p != nullptr; p = p->next)
        p->day = d++;
}

// Insert a status at the end of the list
void insertEnd(char s)
{
    Node *t = newNode(s);
    if (head == nullptr) {
        head = t;
    } else {
        Node *p = head;
        while (p->next != nullptr)
            p = p->next;
        p->next = t;
    }
    renumber();
    cout << "Status '" << s << "' inserted at the end (Day " << t->day << ").\n";
}

// Count the days having the given status
int countStatus(char s)
{
    int c = 0;
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->status == s)
            c++;
    }
    return c;
}

// Display the complete status list
void display()
{
    if (head == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    cout << "\nDay Status\n";
    cout << "----------------\n";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << left << setw(5) << p->day << " " << p->status << "\n";
```

```
    cout << "\nList : HEAD -> ";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << p->status << " -> ";
    cout << "NULL\n";
}

void freeList()
{
    while (head != nullptr) {
        Node *p = head;
        head = head->next;
        delete p;
    }
}
// Update the status of a given day
void updateStatus(int day, char s)
{
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->day == day) {
            p->status = s;
            cout << "Day " << day << " updated to '" << s << "'.\n";
            return;
        }
    }
    cout << "Day " << day << " not found in the list.\n";
}

int main()
{
    int choice, day;
    char s;
    do {
        cout << "\n===== EMI PAYMENT STATUS MENU =====\n";
        cout << "1. Insert a status at the end of the list\n";
        cout << "2. Update the status of a given day\n";
        cout << "3. Calculate total number of paid EMIs\n";
        cout << "4. Display the complete status list\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            s = readStatus();
            insertEnd(s);
            break;
        case 2:
            cout << "Enter day number: ";
            cin >> day;
            s = readStatus();
            updateStatus(day, s);
            break;
        case 3:
            cout << "Total number of paid EMIs = "
                 << countStatus('P') << "\n";
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    freeList();
    return 0;
}
```

Set 18 – Daily EMI payment status (P = Paid / D = Due) using a singly linked list

a. Insert a status at the middle of the list
b. Delete a status by day from the front
c. Calculate the total number of due EMIs
d. Display the complete list

Program (C++)

```
/* Set 18 - Daily EMI payment status (P = Paid / D = Due) using a singly linked list */
// Compile: g++ -std=c++17 -Wall -Wextra set18.cpp -o set18
#include <iostream>
#include <iomanip>
#include <cctype>
#include <cstdlib>
using namespace std;

struct Node {
    int day;           // day number = position in the list
    char status;       // 'P' = Paid, 'D' = Due
    Node *next;
};

Node *head = nullptr;

bool validStatus(char c)
{
    return c == 'P' || c == 'D';
}

char readStatus()
{
    char c;
    do {
        cout << "Enter status (P = Paid / D = Due): ";
        if (!(cin >> c)) {
            cout << "\nInput ended.\n";
            exit(0);
        }
        c = toupper((unsigned char)c);
        if (!validStatus(c))
            cout << "Invalid status! Enter only P or D.\n";
    } while (!validStatus(c));
    return c;
}

Node *newNode(char s)
{
    Node *t = new Node;
    t->status = s;
    t->day = 0;
    t->next = nullptr;
    return t;
}
int length()
{
    int c = 0;
    for (Node *p = head; p != nullptr; p = p->next)
        c++;
    return c;
}
// re-number the days 1,2,3,... after every insertion / deletion
void renumber()
{
    int d = 1;
    for (Node *p = head; p != nullptr; p = p->next)
        p->day = d++;
}

// Insert a status at the end of the list
void insertEnd(char s)
{
    Node *t = newNode(s);
    if (head == nullptr) {
        head = t;
```

```
    } else {
        Node *p = head;
        while (p->next != nullptr)
            p = p->next;
        p->next = t;
    }
    renumber();
    cout << "Status '" << s << "' inserted at the end (Day " << t->day << ").\n";
}

// Count the days having the given status
int countStatus(char s)
{
    int c = 0;
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->status == s)
            c++;
    }
    return c;
}

// Display the complete status list
void display()
{
    if (head == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    cout << "\nDay Status\n";
    cout << "----------------\n";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << left << setw(5) << p->day << " " << p->status << "\n";
    cout << "\nList : HEAD -> ";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << p->status << " -> ";
    cout << "NULL\n";
}

void freeList()
{
    while (head != nullptr) {
        Node *p = head;
        head = head->next;
        delete p;
    }
}
// Insert a status in the middle of the list (after the middle node)
void insertMiddle(char s)
{
    Node *t = newNode(s);
    int mid = (length() + 1) / 2;       // position of the middle node
    if (head == nullptr) {
        head = t;
    } else {
        Node *p = head;
        for (int i = 1; i < mid; i++)
            p = p->next;
        t->next = p->next;
        p->next = t;
    }
    renumber();
    cout << "Status '" << s << "' inserted in the middle (Day " << t->day << ").\n";
}

// Delete the status of a given day, searching from the front
void deleteByDay(int day)
{
    Node *p = head, *previous = nullptr;
    while (p != nullptr && p->day != day) {
        previous = p;
        p = p->next;
    }
    if (p == nullptr) {
        cout << "Day " << day << " not found in the list.\n";
        return;
```

```
    }
    if (previous == nullptr)
        head = p->next;
    else
        previous->next = p->next;
    delete p;
    renumber();
    cout << "Day " << day << " deleted.\n";
}

int main()
{
    int choice, day;
    char s;
    do {
        cout << "\n===== EMI PAYMENT STATUS MENU =====\n";
        cout << "1. Insert a status at the end of the list (to build the list)\n";
        cout << "2. Insert a status at the middle of the list\n";
        cout << "3. Delete a status by day (searching from the front)\n";
        cout << "4. Calculate total number of due EMIs\n";
        cout << "5. Display the complete list\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            s = readStatus();
            insertEnd(s);
            break;
        case 2:
            s = readStatus();
            insertMiddle(s);
            break;
        case 3:
            cout << "Enter day number to delete: ";
            cin >> day;
            deleteByDay(day);
            break;
        case 4:
            cout << "Total number of due EMIs = "
                 << countStatus('D') << "\n";
            break;
        case 5:
            display();
            break;
        case 6:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 6);
    freeList();
    return 0;
}
```

Set 19 – Maintain a list of loan applicants sorted by Application No using a doubly linked list

a. Insert a applicant so that the list remains sorted
b. Delete a applicant by Application No
c. Traverse forward and display all loan applicants
d. Traverse backward and display all loan applicants in reverse order

Program (C++)

```
/* Set 19 - Maintain a list of loan applicants sorted by Application No using a doubly
   linked list */
// Compile: g++ -std=c++17 -Wall -Wextra set19.cpp -o set19
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

struct Node {
    int id;            // Application No
    string name;       // Applicant Name
    Node *prev;
    Node *next;
};

Node *head = nullptr;  // first node (smallest Application No)
Node *tail = nullptr;  // last node (largest Application No)

Node *findNode(int id)
{
    for (Node *p = head; p != nullptr; p = p->next) {
        if (p->id == id)
            return p;
    }
    return nullptr;
}

// a. Insert an applicant so that the list remains sorted by Application No
void insertSorted(int id, const string &name)
{
    if (findNode(id) != nullptr) {
        cout << "Application No " << id << " already exists. Insertion cancelled.\n";
        return;
    }
    Node *t = new Node;
    t->id = id;
    t->name = name;
    t->prev = t->next = nullptr;
    if (head == nullptr) {                  // empty list
        head = tail = t;
    } else if (id < head->id) {             // insert at beginning
        t->next = head;
        head->prev = t;
        head = t;
    } else {                                // middle or end
        Node *p = head;
        while (p->next != nullptr && p->next->id < id)
            p = p->next;
        t->next = p->next;
        t->prev = p;
        if (p->next != nullptr)
            p->next->prev = t;
        else
            tail = t;
        p->next = t;
    }
    cout << "Applicant " << id << " inserted.\n";
}

// b. Delete an applicant by Application No
void deleteById(int id)
{
    Node *p = findNode(id);
    if (p == nullptr) {
        cout << "Applicant with Application No " << id << " not found.\n";
        return;
```

```
    }
    if (p->prev != nullptr)
        p->prev->next = p->next;
    else
        head = p->next;                    // deleting first node
    if (p->next != nullptr)
        p->next->prev = p->prev;
    else
        tail = p->prev;                    // deleting last node
    delete p;
    cout << "Applicant with Application No " << id << " deleted.\n";
}

void printHeader()
{
    cout << "\n" << left << setw(12) << "Application No" << " " << "Applicant Name" << "\n";
    cout << "------------------------------\n";
}

// c. Traverse forward and display all loan applicants
void displayForward()
{
    if (head == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    printHeader();
    for (Node *p = head; p != nullptr; p = p->next)
        cout << left << setw(12) << p->id << " " << p->name << "\n";
}

// d. Traverse backward and display all loan applicants in reverse order
void displayBackward()
{
    if (tail == nullptr) {
        cout << "The list is empty.\n";
        return;
    }
    printHeader();
    for (Node *p = tail; p != nullptr; p = p->prev)
        cout << left << setw(12) << p->id << " " << p->name << "\n";
}

int main()
{
    int choice, id;
    string name;
    do {
        cout << "\n===== Applicant LIST MENU (DOUBLY LINKED LIST) =====\n";
        cout << "1. Insert an applicant (list stays sorted)\n";
        cout << "2. Delete an applicant by Application No\n";
        cout << "3. Traverse forward and display all loan applicants\n";
        cout << "4. Traverse backward and display all loan applicants\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            cout << "Enter Application No : ";
            cin >> id;
            cout << "Enter Applicant Name : ";
            cin >> ws;
            getline(cin, name);
            insertSorted(id, name);
            break;
        case 2:
            cout << "Enter Application No to delete: ";
            cin >> id;
            deleteById(id);
            break;
        case 3:
            displayForward();
            break;
        case 4:
```

```
            displayBackward();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    while (head != nullptr) {              // release memory
        Node *p = head;
        head = head->next;
        delete p;
    }
    return 0;
}
```

Set 20 – Store the account balance of N customers using both an array and a linked list

a. Insert the values in both structures (array and linked list)
b. Display the memory representation (index and nodes)
c. Count how many customers have account balance above Rs. 50000

Program (C++)

```
/* Set 20 - Store the account balance of N customers using both an array and a linked
   list */
// Compile: g++ -std=c++17 -Wall -Wextra set20.cpp -o set20
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
using namespace std;

const int MAX = 50;

struct Node {
    float data;
    Node *next;
};

float arr[MAX];                // array representation
Node *head = nullptr;          // linked list representation
Node *tail = nullptr;
int n = 0;

// prints a pointer the way a C program would (null shown as (nil))
string ptrStr(const void *p)
{
    if (p == nullptr)
        return "(nil)";
    ostringstream os;
    os << p;
    return os.str();
}

// a. Insert the value in both the array and the linked list
void insertBoth(float x)
{
    if (n == MAX) {
        cout << "Storage full.\n";
        return;
    }
    arr[n] = x;                // array
    Node *t = new Node;        // linked list (append)
    t->data = x;
    t->next = nullptr;
    if (head == nullptr) {
        head = tail = t;
    } else {
        tail->next = t;
        tail = t;
    }
```

```
    n++;
}

// b. Display the memory representation (index and nodes)
void displayMemory()
{
    cout << "\n--- ARRAY (contiguous memory) ---\n";
    cout << left << setw(7) << "Index" << " " << setw(18) << "Address" << " "
         << "account balance" << "\n";
    for (int i = 0; i < n; i++)
        cout << left << setw(7) << i << " " << setw(18) << ptrStr(&arr[i]) << " "
             << arr[i] << "\n";

    cout << "\n--- LINKED LIST (nodes may be scattered in memory) ---\n";
    cout << left << setw(6) << "Node" << " " << setw(18) << "Address" << " " << setw(12) << "Data"
         << " " << "Next" << "\n";
    int i = 1;
    for (Node *p = head; p != nullptr; p = p->next, i++)
        cout << left << setw(6) << i << " " << setw(18) << ptrStr(p) << " " << setw(12) << p->data
             << " " << ptrStr(p->next) << "\n";
    cout << "HEAD -> ";
    for (Node *p = head; p != nullptr; p = p->next)
        cout << "[" << p->data << "] -> ";
    cout << "NULL\n";
}

// c. Count how many customers have account balance above 50000
void countAbove()
{
    int ca = 0, cl = 0;
    for (int i = 0; i < n; i++) {                   // using the array
        if (arr[i] > 50000)
            ca++;
    }
    for (Node *p = head; p != nullptr; p = p->next) {   // using the linked list
        if (p->data > 50000)
            cl++;
    }
    cout << "Count using array = " << ca << "\n";
    cout << "Count using linked list = " << cl << "\n";
    cout << "Number of customers with account balance above 50000 = " << ca << "\n";
}

int main()
{
    int choice, cnt;
    float x;
    cout << fixed << setprecision(2);
    cout << "Enter number of customers N (max " << MAX << "): ";
    cin >> cnt;
    if (cnt < 0 || cnt > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    for (int i = 0; i < cnt; i++) {
        cout << "Enter account balance of customer " << i + 1 << ": ";
        cin >> x;
        insertBoth(x);
    }
    do {
        cout << "\n===== MENU =====\n";
        cout << "1. Display memory representation (array index and list nodes)\n";
        cout << "2. Count customers with account balance above 50000\n";
        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            displayMemory();
            break;
        case 2:
            countAbove();
            break;
        case 3:
```

```
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 3);
    while (head != nullptr) {
        Node *p = head;
        head = head->next;
        delete p;
    }
    return 0;
}
```

Set 21 – Simulate the page navigation history of an online library catalogue using a stack

a. Visit a new page (push URL)
b. Go back to the previous page (pop URL)
c. Display the current page
d. Show the entire browsing history stack

Program (C++)

```
/* Set 21 - Simulate the page navigation history of an online library catalogue using a
   stack */
// Compile: g++ -std=c++17 -Wall -Wextra set21.cpp -o set21
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 10;               // maximum pages kept in the history

string history[MAX];
int top = -1;

bool isEmpty() { return top == -1; }
bool isFull()  { return top == MAX - 1; }

// a. Visit a new page (push URL)
void visit(const string &url)
{
    if (isFull()) {
        cout << "Stack Overflow! History is full.\n";
        return;
    }
    history[++top] = url;
    cout << "Visited: " << url << "\n";
}

// b. Go back to the previous page (pop URL)
void goBack()
{
    if (isEmpty()) {
        cout << "Stack Underflow! No page in history.\n";
        return;
    }
    if (top == 0) {
        cout << "This is the first page. Cannot go back.\n";
        return;
    }
    cout << "Left page: " << history[top] << "\n";
    top--;
    cout << "Went back to: " << history[top] << "\n";
}

// c. Display the current page
void currentPage()
{
    if (isEmpty())
        cout << "No page has been visited yet.\n";
    else
        cout << "Current page: " << history[top] << "\n";
```

```
}

// d. Show the entire browsing history stack (top to bottom)
void showHistory()
{
    if (isEmpty()) {
        cout << "Browsing history is empty.\n";
        return;
    }
    cout << "\n----- Browsing history (top to bottom) -----\n";
    for (int i = top; i >= 0; i--)
        cout << right << setw(2) << i + 1 << ". " << history[i]
             << (i == top ? " <-- current" : "") << "\n";
}

int main()
{
    int choice;
    string url;
    do {
        cout << "\n===== AN ONLINE LIBRARY CATALOGUE - BROWSING HISTORY =====\n";
        cout << "1. Visit a new page (push URL)\n";
        cout << "2. Go back to the previous page (pop URL)\n";
        cout << "3. Display the current page\n";
        cout << "4. Show the entire browsing history\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            cout << "Enter URL: ";
            cin >> url;
            visit(url);
            break;
        case 2:
            goBack();
            break;
        case 3:
            currentPage();
            break;
        case 4:
            showHistory();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 22 – Late-fine formula: infix to postfix and evaluation

a. Convert the infix expression to postfix
b. Evaluate the postfix expression for given inputs

Program (C++)

```
/* Set 22 - Late-fine formula: infix to postfix and evaluation */
// Compile: g++ -std=c++17 -Wall -Wextra set22.cpp -o set22
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <cctype>
#include <cmath>
#include <cstdlib>
using namespace std;

const int MAX = 100;
const bool TRACE = false;       // true = show intermediate stack contents

// variables the user must supply
const vector<string> required = {};

// ---------- stack of operators (used for infix -> postfix) ----------
char opStack[MAX];
int opTop = -1;

void opPush(char c)
{
    if (opTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    opStack[++opTop] = c;
}

char opPop()
{
    if (opTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return opStack[opTop--];
}

char opPeek()
{
    return (opTop == -1) ? '\0' : opStack[opTop];
}

// ---------- stack of values (used for postfix evaluation) ----------
double valStack[MAX];
int valTop = -1;

void valPush(double v)
{
    if (valTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    valStack[++valTop] = v;
}

double valPop()
{
    if (valTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return valStack[valTop--];
}

// ---------- postfix expression and variable table ----------
string post[MAX];
```

```
int pn = 0;

struct Var {
    string name;
    double value;
};

Var vars[MAX];
int vn = 0;

void emit(const string &t)
{
    if (pn == MAX) {
        cout << "Expression too long.\n";
        exit(1);
    }
    post[pn++] = t;
}

bool isOperand(const string &t)
{
    return isalnum((unsigned char)t[0]) || t[0] == '_' || t[0] == '.';
}

bool isNumber(const string &t)
{
    return isdigit((unsigned char)t[0]) || t[0] == '.';
}

int prec(char c)
{
    switch (c) {
    case '^': return 3;
    case '*':
    case '/': return 2;
    case '+':
    case '-': return 1;
    }
    return 0;
}

void showOpStack()
{
    cout << "[";
    for (int i = 0; i <= opTop; i++)
        cout << " " << opStack[i];
    cout << " ]";
}

void showPostfix()
{
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
}

void traceConv(const string &tok)
{
    if (!TRACE)
        return;
    cout << left << setw(14) << tok << "| ";
    showOpStack();
    cout << " | ";
    showPostfix();
    cout << "\n";
}

void showValStack()
{
    cout << "[";
    for (int i = 0; i <= valTop; i++)
        cout << " " << valStack[i];
    cout << " ]";
}

// Convert infix expression e to postfix (returns true on success)
```

```
bool infixToPostfix(const string &e)
{
    size_t i = 0;
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << setw(16) << "Operator stack"
             << "| " << "Postfix so far" << "\n";
    while (i < e.size()) {
        char ch = e[i];
        if (isspace((unsigned char)ch)) {
            i++;
        } else if (isalnum((unsigned char)ch) || ch == '_' || ch == '.') {
            string tok;
            while (i < e.size() && (isalnum((unsigned char)e[i]) || e[i] == '_' || e[i] == '.'))
                tok += e[i++];
            emit(tok);
            traceConv(tok);
        } else if (ch == '(') {
            opPush('(');
            traceConv("(");
            i++;
        } else if (ch == ')') {
            while (opTop != -1 && opPeek() != '(')
                emit(string(1, opPop()));
            if (opTop == -1) {
                cout << "Error: unbalanced ')'\n";
                return false;
            }
            opPop();                        // discard '('
            traceConv(")");
            i++;
        } else if (string("+-*/^").find(ch) != string::npos) {
            while (opTop != -1 && opPeek() != '(' &&
                   (prec(opPeek()) > prec(ch) || (prec(opPeek()) == prec(ch) && ch != '^')))
                emit(string(1, opPop()));
            opPush(ch);
            traceConv(string(1, ch));
            i++;
        } else {
            cout << "Error: invalid character '" << ch << "' in expression\n";
            return false;
        }
    }
    while (opTop != -1) {
        char c = opPop();
        if (c == '(') {
            cout << "Error: unbalanced '('\n";
            return false;
        }
        emit(string(1, c));
    }
    if (TRACE)
        traceConv("(end)");
    return true;
}

int findVar(const string &name)
{
    for (int i = 0; i < vn; i++) {
        if (vars[i].name == name)
            return i;
    }
    return -1;
}

void askVar(const string &name)
{
    double v;
    if (findVar(name) != -1)
        return;
    cout << "Enter value of " << name << " : ";
    if (!(cin >> v)) {
        cout << "\nInvalid input.\n";
        exit(1);
    }
    vars[vn].name = name;
```

```
    vars[vn].value = v;
    vn++;
}

// accept the required variables first, then any other variable used
void readValues()
{
    for (size_t i = 0; i < required.size(); i++)
        askVar(required[i]);
    for (int i = 0; i < pn; i++) {
        if (isOperand(post[i]) && !isNumber(post[i]))
            askVar(post[i]);
    }
}

double valueOf(const string &t)
{
    if (isNumber(t))
        return atof(t.c_str());
    return vars[findVar(t)].value;
}

// Evaluate the postfix expression using a stack (returns true on success)
bool evaluate(double &result)
{
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << "Value stack" << "\n";
    for (int i = 0; i < pn; i++) {
        const string &t = post[i];
        if (isOperand(t)) {
            valPush(valueOf(t));
        } else {
            if (valTop < 1) {
                cout << "Error: invalid expression (operand missing)\n";
                return false;
            }
            double b = valPop();
            double a = valPop();
            double r = 0;
            switch (t[0]) {
            case '+': r = a + b; break;
            case '-': r = a - b; break;
            case '*': r = a * b; break;
            case '/':
                if (b == 0) {
                    cout << "Error: division by zero\n";
                    return false;
                }
                r = a / b;
                break;
            case '^': r = pow(a, b); break;
            }
            valPush(r);
        }
        if (TRACE) {
            cout << left << setw(14) << t << "| ";
            showValStack();
            cout << "\n";
        }
    }
    if (valTop != 0) {
        cout << "Error: invalid expression\n";
        return false;
    }
    result = valPop();
    return true;
}

int main()
{
    string expr;
    double result;
    cout << "Enter the late-fine formula in infix form\n";
    cout << "(e.g. (Days * Rate) + Fixed - Waiver): ";
    if (!getline(cin, expr))
```

```
        return 1;
    if (!infixToPostfix(expr))
        return 1;
    cout << "\nPostfix expression : ";
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
    cout << "\n\n";
    readValues();
    if (!evaluate(result))
        return 1;
    cout << "\nLate fine = " << fixed << setprecision(2) << result << "\n";
    return 0;
}
```

Set 23 – Order return handling of an Amazon warehouse using a stack

a. Push a newly returned parcel (display "Stack Overflow" if full)
b. Pop the top parcel and display its details (display "Stack Underflow" if empty)
c. Peek the parcel on top without removing it
d. Display all parcels from top to bottom

Program (C++)

```
/* Set 23 - Order return handling of an Amazon warehouse using a stack */
// Compile: g++ -std=c++17 -Wall -Wextra set23.cpp -o set23
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the returns stack

struct Parcel {
    int orderId;
    string product;
    float price;
};

Parcel parcels[MAX];
int top = -1;

void printParcel(const Parcel &p)
{
    cout << "Order ID : " << p.orderId << "\n";
    cout << "Product : " << p.product << "\n";
    cout << "Price : " << p.price << "\n";
}

// a. Push a newly returned parcel onto the stack
void push()
{
    Parcel p;
    if (top == MAX - 1) {
        cout << "Stack Overflow\n";
        return;
    }
    cout << "Enter Order ID : ";
    cin >> p.orderId;
    cout << "Enter Product Name : ";
    cin >> ws;
    getline(cin, p.product);
    cout << "Enter Price : ";
    cin >> p.price;
    parcels[++top] = p;
    cout << "Parcel pushed onto the stack.\n";
}

// b. Pop the top parcel when it is processed and display its details
void pop()
{
    if (top == -1) {
        cout << "Stack Underflow\n";
        return;
```

```
    }
    cout << "Processing parcel:\n";
    printParcel(parcels[top]);
    top--;
}

// c. Peek at the parcel currently on top without removing it
void peek()
{
    if (top == -1) {
        cout << "Stack is empty.\n";
        return;
    }
    cout << "Parcel on top of the stack:\n";
    printParcel(parcels[top]);
}

// d. Display all parcels in the stack from top to bottom
void display()
{
    if (top == -1) {
        cout << "Stack is empty.\n";
        return;
    }
    cout << "\n" << left << setw(10) << "Order ID" << " " << setw(25) << "Product Name"
         << " " << "Price" << "\n";
    cout << "-------------------------------------------------\n";
    for (int i = top; i >= 0; i--)
        cout << left << setw(10) << parcels[i].orderId << " " << setw(25) << parcels[i].product
             << " " << parcels[i].price << "\n";
}

int main()
{
    int choice;
    cout << fixed << setprecision(2);
    do {
        cout << "\n===== RETURNED PARCELS (STACK) =====\n";
        cout << "1. Push a returned parcel\n";
        cout << "2. Pop (process) the top parcel\n";
        cout << "3. Peek the top parcel\n";
        cout << "4. Display all parcels (top to bottom)\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1: push(); break;
        case 2: pop(); break;
        case 3: peek(); break;
        case 4: display(); break;
        case 5: cout << "Exiting...\n"; break;
        default: cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 24 – Library Fine Billing System

a. Read the fine formula in infix form
b. Convert the infix expression to postfix using a stack
c. Display the postfix expression after conversion
d. Accept values for LateDays, DailyRate, DamageCharge, Waiver
e. Evaluate the postfix expression using a stack
f. Display the final result
g. Display the intermediate stack contents during conversion and evaluation

Program (C++)

```
/* Set 24 - Library Fine Billing System */
// Compile: g++ -std=c++17 -Wall -Wextra set24.cpp -o set24
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <cctype>
#include <cmath>
#include <cstdlib>
using namespace std;

const int MAX = 100;
const bool TRACE = true;       // true = show intermediate stack contents

// variables the user must supply
const vector<string> required = {"LateDays", "DailyRate", "DamageCharge", "Waiver"};

// ---------- stack of operators (used for infix -> postfix) ----------
char opStack[MAX];
int opTop = -1;

void opPush(char c)
{
    if (opTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    opStack[++opTop] = c;
}

char opPop()
{
    if (opTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return opStack[opTop--];
}

char opPeek()
{
    return (opTop == -1) ? '\0' : opStack[opTop];
}

// ---------- stack of values (used for postfix evaluation) ----------
double valStack[MAX];
int valTop = -1;

void valPush(double v)
{
    if (valTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    valStack[++valTop] = v;
}

double valPop()
{
    if (valTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
```

```
    }
    return valStack[valTop--];
}

// ---------- postfix expression and variable table ----------
string post[MAX];
int pn = 0;

struct Var {
    string name;
    double value;
};

Var vars[MAX];
int vn = 0;

void emit(const string &t)
{
    if (pn == MAX) {
        cout << "Expression too long.\n";
        exit(1);
    }
    post[pn++] = t;
}

bool isOperand(const string &t)
{
    return isalnum((unsigned char)t[0]) || t[0] == '_' || t[0] == '.';
}

bool isNumber(const string &t)
{
    return isdigit((unsigned char)t[0]) || t[0] == '.';
}

int prec(char c)
{
    switch (c) {
    case '^': return 3;
    case '*':
    case '/': return 2;
    case '+':
    case '-': return 1;
    }
    return 0;
}

void showOpStack()
{
    cout << "[";
    for (int i = 0; i <= opTop; i++)
        cout << " " << opStack[i];
    cout << " ]";
}

void showPostfix()
{
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
}

void traceConv(const string &tok)
{
    if (!TRACE)
        return;
    cout << left << setw(14) << tok << "| ";
    showOpStack();
    cout << " | ";
    showPostfix();
    cout << "\n";
}

void showValStack()
{
    cout << "[";
```

```
    for (int i = 0; i <= valTop; i++)
        cout << " " << valStack[i];
    cout << " ]";
}

// Convert infix expression e to postfix (returns true on success)
bool infixToPostfix(const string &e)
{
    size_t i = 0;
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << setw(16) << "Operator stack"
             << "| " << "Postfix so far" << "\n";
    while (i < e.size()) {
        char ch = e[i];
        if (isspace((unsigned char)ch)) {
            i++;
        } else if (isalnum((unsigned char)ch) || ch == '_' || ch == '.') {
            string tok;
            while (i < e.size() && (isalnum((unsigned char)e[i]) || e[i] == '_' || e[i] == '.'))
                tok += e[i++];
            emit(tok);
            traceConv(tok);
        } else if (ch == '(') {
            opPush('(');
            traceConv("(");
            i++;
        } else if (ch == ')') {
            while (opTop != -1 && opPeek() != '(')
                emit(string(1, opPop()));
            if (opTop == -1) {
                cout << "Error: unbalanced ')'\n";
                return false;
            }
            opPop();                        // discard '('
            traceConv(")");
            i++;
        } else if (string("+-*/^").find(ch) != string::npos) {
            while (opTop != -1 && opPeek() != '(' &&
                   (prec(opPeek()) > prec(ch) || (prec(opPeek()) == prec(ch) && ch != '^')))
                emit(string(1, opPop()));
            opPush(ch);
            traceConv(string(1, ch));
            i++;
        } else {
            cout << "Error: invalid character '" << ch << "' in expression\n";
            return false;
        }
    }
    while (opTop != -1) {
        char c = opPop();
        if (c == '(') {
            cout << "Error: unbalanced '('\n";
            return false;
        }
        emit(string(1, c));
    }
    if (TRACE)
        traceConv("(end)");
    return true;
}

int findVar(const string &name)
{
    for (int i = 0; i < vn; i++) {
        if (vars[i].name == name)
            return i;
    }
    return -1;
}

void askVar(const string &name)
{
    double v;
    if (findVar(name) != -1)
        return;
```

```
    cout << "Enter value of " << name << " : ";
    if (!(cin >> v)) {
        cout << "\nInvalid input.\n";
        exit(1);
    }
    vars[vn].name = name;
    vars[vn].value = v;
    vn++;
}

// accept the required variables first, then any other variable used
void readValues()
{
    for (size_t i = 0; i < required.size(); i++)
        askVar(required[i]);
    for (int i = 0; i < pn; i++) {
        if (isOperand(post[i]) && !isNumber(post[i]))
            askVar(post[i]);
    }
}

double valueOf(const string &t)
{
    if (isNumber(t))
        return atof(t.c_str());
    return vars[findVar(t)].value;
}

// Evaluate the postfix expression using a stack (returns true on success)
bool evaluate(double &result)
{
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << "Value stack" << "\n";
    for (int i = 0; i < pn; i++) {
        const string &t = post[i];
        if (isOperand(t)) {
            valPush(valueOf(t));
        } else {
            if (valTop < 1) {
                cout << "Error: invalid expression (operand missing)\n";
                return false;
            }
            double b = valPop();
            double a = valPop();
            double r = 0;
            switch (t[0]) {
            case '+': r = a + b; break;
            case '-': r = a - b; break;
            case '*': r = a * b; break;
            case '/':
                if (b == 0) {
                    cout << "Error: division by zero\n";
                    return false;
                }
                r = a / b;
                break;
            case '^': r = pow(a, b); break;
            }
            valPush(r);
        }
        if (TRACE) {
            cout << left << setw(14) << t << "| ";
            showValStack();
            cout << "\n";
        }
    }
    if (valTop != 0) {
        cout << "Error: invalid expression\n";
        return false;
    }
    result = valPop();
    return true;
}

int main()
```

```
{
    string expr;
    double result;
    cout << "Enter the fine formula in infix form\n";
    cout << "(e.g. (LateDays * DailyRate) + DamageCharge - Waiver): ";
    if (!getline(cin, expr))
        return 1;
    if (!infixToPostfix(expr))
        return 1;
    cout << "\nPostfix expression : ";
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
    cout << "\n\n";
    readValues();
    if (!evaluate(result))
        return 1;
    cout << "\nFinal payable fine = " << fixed << setprecision(2) << result << "\n";
    return 0;
}
```

Set 25 – Book issue counter queue of a library

a. Enqueue a member's token number
b. Dequeue and display the member being served
c. Display the front and rear token numbers
d. Display the complete queue

Program (C++)

```
/* Set 25 - Book issue counter queue of a library */
// Compile: g++ -std=c++17 -Wall -Wextra set25.cpp -o set25
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Token No
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Token No : ";
    cin >> x.id;
}

void printItem(const Item &x)
{
    cout << "Token No " << x.id;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
    cout << "Token No " << x.id << " added to the queue.\n";
}
```

```
// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;         // queue became empty - reset
    else
        front++;
    cout << "Member being served: ";
    printItem(x);
    cout << "\n";
}

void showFrontRear()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "Front token : ";
    printItem(q[front]);
    cout << "\nRear token : ";
    printItem(q[rear]);
    cout << "\n";
}

void display()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "FRONT -> ";
    for (int i = front; i <= rear; i++) {
        printItem(q[i]);
        if (i < rear)
            cout << " | ";
    }
    cout << " <- REAR\n";
}

int main()
{
    int choice;
    do {
        cout << "\n===== LIBRARY ISSUE COUNTER QUEUE =====\n";
        cout << "1. Enqueue a member's token number\n";
        cout << "2. Dequeue and display the member being served\n";
        cout << "3. Display the front and rear token numbers\n";
        cout << "4. Display the complete queue\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            showFrontRear();
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
```

```
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 26 – Simulate the page navigation history of a hospital patient portal using a stack

a. Visit a new page (push URL)
b. Go back to the previous page (pop URL)
c. Display the current page
d. Show the entire browsing history stack

Program (C++)

```
/* Set 26 - Simulate the page navigation history of a hospital patient portal using a
   stack */
// Compile: g++ -std=c++17 -Wall -Wextra set26.cpp -o set26
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 10;               // maximum pages kept in the history

string history[MAX];
int top = -1;

bool isEmpty() { return top == -1; }
bool isFull()  { return top == MAX - 1; }

// a. Visit a new page (push URL)
void visit(const string &url)
{
    if (isFull()) {
        cout << "Stack Overflow! History is full.\n";
        return;
    }
    history[++top] = url;
    cout << "Visited: " << url << "\n";
}

// b. Go back to the previous page (pop URL)
void goBack()
{
    if (isEmpty()) {
        cout << "Stack Underflow! No page in history.\n";
        return;
    }
    if (top == 0) {
        cout << "This is the first page. Cannot go back.\n";
        return;
    }
    cout << "Left page: " << history[top] << "\n";
    top--;
    cout << "Went back to: " << history[top] << "\n";
}

// c. Display the current page
void currentPage()
{
    if (isEmpty())
        cout << "No page has been visited yet.\n";
    else
        cout << "Current page: " << history[top] << "\n";
}

// d. Show the entire browsing history stack (top to bottom)
void showHistory()
{
    if (isEmpty()) {
        cout << "Browsing history is empty.\n";
        return;
```

```
    }
    cout << "\n----- Browsing history (top to bottom) -----\n";
    for (int i = top; i >= 0; i--)
        cout << right << setw(2) << i + 1 << ". " << history[i]
             << (i == top ? " <-- current" : "") << "\n";
}

int main()
{
    int choice;
    string url;
    do {
        cout << "\n===== A HOSPITAL PATIENT PORTAL - BROWSING HISTORY =====\n";
        cout << "1. Visit a new page (push URL)\n";
        cout << "2. Go back to the previous page (pop URL)\n";
        cout << "3. Display the current page\n";
        cout << "4. Show the entire browsing history\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            cout << "Enter URL: ";
            cin >> url;
            visit(url);
            break;
        case 2:
            goBack();
            break;
        case 3:
            currentPage();
            break;
        case 4:
            showHistory();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 27 – Hospital bill formula: infix to postfix and evaluation

a. Convert the infix expression to postfix

b. Evaluate the postfix expression for given inputs

Program (C++)

```
/* Set 27 - Hospital bill formula: infix to postfix and evaluation */
// Compile: g++ -std=c++17 -Wall -Wextra set27.cpp -o set27
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <cctype>
#include <cmath>
#include <cstdlib>
using namespace std;

const int MAX = 100;
const bool TRACE = false;       // true = show intermediate stack contents

// variables the user must supply
const vector<string> required = {};

// ---------- stack of operators (used for infix -> postfix) ----------
char opStack[MAX];
int opTop = -1;
void opPush(char c)
```

```
{
    if (opTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    opStack[++opTop] = c;
}

char opPop()
{
    if (opTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return opStack[opTop--];
}

char opPeek()
{
    return (opTop == -1) ? '\0' : opStack[opTop];
}

// ---------- stack of values (used for postfix evaluation) ----------
double valStack[MAX];
int valTop = -1;

void valPush(double v)
{
    if (valTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    valStack[++valTop] = v;
}

double valPop()
{
    if (valTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return valStack[valTop--];
}

// ---------- postfix expression and variable table ----------
string post[MAX];
int pn = 0;

struct Var {
    string name;
    double value;
};

Var vars[MAX];
int vn = 0;

void emit(const string &t)
{
    if (pn == MAX) {
        cout << "Expression too long.\n";
        exit(1);
    }
    post[pn++] = t;
}

bool isOperand(const string &t)
{
    return isalnum((unsigned char)t[0]) || t[0] == '_' || t[0] == '.';
}

bool isNumber(const string &t)
{
    return isdigit((unsigned char)t[0]) || t[0] == '.';
}
int prec(char c)
```

```
{
    switch (c) {
    case '^': return 3;
    case '*':
    case '/': return 2;
    case '+':
    case '-': return 1;
    }
    return 0;
}

void showOpStack()
{
    cout << "[";
    for (int i = 0; i <= opTop; i++)
        cout << " " << opStack[i];
    cout << " ]";
}

void showPostfix()
{
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
}

void traceConv(const string &tok)
{
    if (!TRACE)
        return;
    cout << left << setw(14) << tok << "| ";
    showOpStack();
    cout << " | ";
    showPostfix();
    cout << "\n";
}

void showValStack()
{
    cout << "[";
    for (int i = 0; i <= valTop; i++)
        cout << " " << valStack[i];
    cout << " ]";
}

// Convert infix expression e to postfix (returns true on success)
bool infixToPostfix(const string &e)
{
    size_t i = 0;
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << setw(16) << "Operator stack"
             << "| " << "Postfix so far" << "\n";
    while (i < e.size()) {
        char ch = e[i];
        if (isspace((unsigned char)ch)) {
            i++;
        } else if (isalnum((unsigned char)ch) || ch == '_' || ch == '.') {
            string tok;
            while (i < e.size() && (isalnum((unsigned char)e[i]) || e[i] == '_' || e[i] == '.'))
                tok += e[i++];
            emit(tok);
            traceConv(tok);
        } else if (ch == '(') {
            opPush('(');
            traceConv("(");
            i++;
        } else if (ch == ')') {
            while (opTop != -1 && opPeek() != '(')
                emit(string(1, opPop()));
            if (opTop == -1) {
                cout << "Error: unbalanced ')'\n";
                return false;
            }
            opPop();                        // discard '('
            traceConv(")");
            i++;
```

```
        } else if (string("+-*/^").find(ch) != string::npos) {
            while (opTop != -1 && opPeek() != '(' &&
                   (prec(opPeek()) > prec(ch) || (prec(opPeek()) == prec(ch) && ch != '^')))
                emit(string(1, opPop()));
            opPush(ch);
            traceConv(string(1, ch));
            i++;
        } else {
            cout << "Error: invalid character '" << ch << "' in expression\n";
            return false;
        }
    }
    while (opTop != -1) {
        char c = opPop();
        if (c == '(') {
            cout << "Error: unbalanced '('\n";
            return false;
        }
        emit(string(1, c));
    }
    if (TRACE)
        traceConv("(end)");
    return true;
}

int findVar(const string &name)
{
    for (int i = 0; i < vn; i++) {
        if (vars[i].name == name)
            return i;
    }
    return -1;
}

void askVar(const string &name)
{
    double v;
    if (findVar(name) != -1)
        return;
    cout << "Enter value of " << name << " : ";
    if (!(cin >> v)) {
        cout << "\nInvalid input.\n";
        exit(1);
    }
    vars[vn].name = name;
    vars[vn].value = v;
    vn++;
}

// accept the required variables first, then any other variable used
void readValues()
{
    for (size_t i = 0; i < required.size(); i++)
        askVar(required[i]);
    for (int i = 0; i < pn; i++) {
        if (isOperand(post[i]) && !isNumber(post[i]))
            askVar(post[i]);
    }
}

double valueOf(const string &t)
{
    if (isNumber(t))
        return atof(t.c_str());
    return vars[findVar(t)].value;
}

// Evaluate the postfix expression using a stack (returns true on success)
bool evaluate(double &result)
{
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << "Value stack" << "\n";
    for (int i = 0; i < pn; i++) {
        const string &t = post[i];
        if (isOperand(t)) {
```

```
            valPush(valueOf(t));
        } else {
            if (valTop < 1) {
                cout << "Error: invalid expression (operand missing)\n";
                return false;
            }
            double b = valPop();
            double a = valPop();
            double r = 0;
            switch (t[0]) {
            case '+': r = a + b; break;
            case '-': r = a - b; break;
            case '*': r = a * b; break;
            case '/':
                if (b == 0) {
                    cout << "Error: division by zero\n";
                    return false;
                }
                r = a / b;
                break;
            case '^': r = pow(a, b); break;
            }
            valPush(r);
        }
        if (TRACE) {
            cout << left << setw(14) << t << "| ";
            showValStack();
            cout << "\n";
        }
    }
    if (valTop != 0) {
        cout << "Error: invalid expression\n";
        return false;
    }
    result = valPop();
    return true;
}

int main()
{
    string expr;
    double result;
    cout << "Enter the bill formula in infix form\n";
    cout << "(e.g. (Consultation + Medicine) * Days - Insurance): ";
    if (!getline(cin, expr))
        return 1;
    if (!infixToPostfix(expr))
        return 1;
    cout << "\nPostfix expression : ";
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
    cout << "\n\n";
    readValues();
    if (!evaluate(result))
        return 1;
    cout << "\nBill amount = " << fixed << setprecision(2) << result << "\n";
    return 0;
}
```

Set 28 – Flight lookup system of an airline using binary search

a. Read the records of N flights and sort them by Flight Number in ascending order
b. Search for a flight by Flight Number using binary search
c. Display the full flight record if found, otherwise "Flight not found"
d. Display the number of comparisons made during the search

Program (C++)

```
/* Set 28 - Flight lookup system of an airline using binary search */
// Compile: g++ -std=c++17 -Wall -Wextra set28.cpp -o set28
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Flight {
    string flight;   // Flight Number
    string dest;     // Destination
    string dep;      // Departure Time
};

Flight rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Flight Number : ";
        cin >> ws;
        getline(cin, rec[i].flight);
        cout << " Enter Destination : ";
        cin >> ws;
        getline(cin, rec[i].dest);
        cout << " Enter Departure Time : ";
        cin >> ws;
        getline(cin, rec[i].dep);
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Flight Number" << ": " << rec[i].flight << "\n";
    cout << " " << left << setw(16) << "Destination" << ": " << rec[i].dest << "\n";
    cout << " " << left << setw(16) << "Departure Time" << ": " << rec[i].dep << "\n";
}

void printAll()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        printRecord(i);
    }
}

// insertion sort : ascending order of Flight Number
void sortRecords()
{
    for (int i = 1; i < n; i++) {
        Flight temp = rec[i];
        int j = i - 1;
        while (j >= 0 && rec[j].flight > temp.flight) {
            rec[j + 1] = rec[j];
            j--;
        }
        rec[j + 1] = temp;
    }
}

// iterative binary search : returns index or -1.
// One comparison (three-way) is counted for every probe of the middle element.
int binarySearch(const string &key, int &comparisons)
```

```
{
    int low = 0, high = n - 1, mid;
    comparisons = 0;
    while (low <= high) {
        mid = low + (high - low) / 2;
        comparisons++;
        int c = rec[mid].flight.compare(key);
        if (c == 0) {
            return mid;
        } else if (c < 0) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main()
{
    int idx, count = 0;
    string key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    sortRecords();
    cout << "\nRecords after sorting by Flight Number:\n";
    printAll();
    cout << "Enter Flight Number to search: ";
    cin >> ws;
    getline(cin, key);
    idx = binarySearch(key, count);
    if (idx == -1) {
        cout << "\nFlight not found\n";
    } else {
        cout << "\nRecord found at index " << idx << ":\n";
        printRecord(idx);
    }
    cout << "Number of comparisons made = " << count << "\n";
    return 0;
}
```

Set 29 – Hospital Billing System

a. Read the bill formula in infix form
b. Convert the infix expression to postfix using a stack
c. Display the postfix expression after conversion
d. Accept values for Consultation, RoomCharge, Medicine, Insurance
e. Evaluate the postfix expression using a stack
f. Display the final result
g. Display the intermediate stack contents during conversion and evaluation

Program (C++)

```
/* Set 29 - Hospital Billing System */
// Compile: g++ -std=c++17 -Wall -Wextra set29.cpp -o set29
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <cctype>
#include <cmath>
#include <cstdlib>
using namespace std;

const int MAX = 100;
const bool TRACE = true;       // true = show intermediate stack contents
```

```
// variables the user must supply
const vector<string> required = {"Consultation", "RoomCharge", "Medicine", "Insurance"};

// ---------- stack of operators (used for infix -> postfix) ----------
char opStack[MAX];
int opTop = -1;

void opPush(char c)
{
    if (opTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    opStack[++opTop] = c;
}

char opPop()
{
    if (opTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return opStack[opTop--];
}

char opPeek()
{
    return (opTop == -1) ? '\0' : opStack[opTop];
}

// ---------- stack of values (used for postfix evaluation) ----------
double valStack[MAX];
int valTop = -1;

void valPush(double v)
{
    if (valTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    valStack[++valTop] = v;
}

double valPop()
{
    if (valTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return valStack[valTop--];
}

// ---------- postfix expression and variable table ----------
string post[MAX];
int pn = 0;

struct Var {
    string name;
    double value;
};

Var vars[MAX];
int vn = 0;

void emit(const string &t)
{
    if (pn == MAX) {
        cout << "Expression too long.\n";
        exit(1);
    }
    post[pn++] = t;
}

bool isOperand(const string &t)
{
```

```
    return isalnum((unsigned char)t[0]) || t[0] == '_' || t[0] == '.';
}

bool isNumber(const string &t)
{
    return isdigit((unsigned char)t[0]) || t[0] == '.';
}

int prec(char c)
{
    switch (c) {
    case '^': return 3;
    case '*':
    case '/': return 2;
    case '+':
    case '-': return 1;
    }
    return 0;
}

void showOpStack()
{
    cout << "[";
    for (int i = 0; i <= opTop; i++)
        cout << " " << opStack[i];
    cout << " ]";
}

void showPostfix()
{
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
}

void traceConv(const string &tok)
{
    if (!TRACE)
        return;
    cout << left << setw(14) << tok << "| ";
    showOpStack();
    cout << " | ";
    showPostfix();
    cout << "\n";
}

void showValStack()
{
    cout << "[";
    for (int i = 0; i <= valTop; i++)
        cout << " " << valStack[i];
    cout << " ]";
}

// Convert infix expression e to postfix (returns true on success)
bool infixToPostfix(const string &e)
{
    size_t i = 0;
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << setw(16) << "Operator stack"
             << "| " << "Postfix so far" << "\n";
    while (i < e.size()) {
        char ch = e[i];
        if (isspace((unsigned char)ch)) {
            i++;
        } else if (isalnum((unsigned char)ch) || ch == '_' || ch == '.') {
            string tok;
            while (i < e.size() && (isalnum((unsigned char)e[i]) || e[i] == '_' || e[i] == '.'))
                tok += e[i++];
            emit(tok);
            traceConv(tok);
        } else if (ch == '(') {
            opPush('(');
            traceConv("(");
            i++;
        } else if (ch == ')') {
```

```
            while (opTop != -1 && opPeek() != '(')
                emit(string(1, opPop()));
            if (opTop == -1) {
                cout << "Error: unbalanced ')'\n";
                return false;
            }
            opPop();                        // discard '('
            traceConv(")");
            i++;
        } else if (string("+-*/^").find(ch) != string::npos) {
            while (opTop != -1 && opPeek() != '(' &&
                   (prec(opPeek()) > prec(ch) || (prec(opPeek()) == prec(ch) && ch != '^')))
                emit(string(1, opPop()));
            opPush(ch);
            traceConv(string(1, ch));
            i++;
        } else {
            cout << "Error: invalid character '" << ch << "' in expression\n";
            return false;
        }
    }
    while (opTop != -1) {
        char c = opPop();
        if (c == '(') {
            cout << "Error: unbalanced '('\n";
            return false;
        }
        emit(string(1, c));
    }
    if (TRACE)
        traceConv("(end)");
    return true;
}

int findVar(const string &name)
{
    for (int i = 0; i < vn; i++) {
        if (vars[i].name == name)
            return i;
    }
    return -1;
}

void askVar(const string &name)
{
    double v;
    if (findVar(name) != -1)
        return;
    cout << "Enter value of " << name << " : ";
    if (!(cin >> v)) {
        cout << "\nInvalid input.\n";
        exit(1);
    }
    vars[vn].name = name;
    vars[vn].value = v;
    vn++;
}

// accept the required variables first, then any other variable used
void readValues()
{
    for (size_t i = 0; i < required.size(); i++)
        askVar(required[i]);
    for (int i = 0; i < pn; i++) {
        if (isOperand(post[i]) && !isNumber(post[i]))
            askVar(post[i]);
    }
}

double valueOf(const string &t)
{
    if (isNumber(t))
        return atof(t.c_str());
    return vars[findVar(t)].value;
}
```

```
// Evaluate the postfix expression using a stack (returns true on success)
bool evaluate(double &result)
{
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << "Value stack" << "\n";
    for (int i = 0; i < pn; i++) {
        const string &t = post[i];
        if (isOperand(t)) {
            valPush(valueOf(t));
        } else {
            if (valTop < 1) {
                cout << "Error: invalid expression (operand missing)\n";
                return false;
            }
            double b = valPop();
            double a = valPop();
            double r = 0;
            switch (t[0]) {
            case '+': r = a + b; break;
            case '-': r = a - b; break;
            case '*': r = a * b; break;
            case '/':
                if (b == 0) {
                    cout << "Error: division by zero\n";
                    return false;
                }
                r = a / b;
                break;
            case '^': r = pow(a, b); break;
            }
            valPush(r);
        }
        if (TRACE) {
            cout << left << setw(14) << t << "| ";
            showValStack();
            cout << "\n";
        }
    }
    if (valTop != 0) {
        cout << "Error: invalid expression\n";
        return false;
    }
    result = valPop();
    return true;
}

int main()
{
    string expr;
    double result;
    cout << "Enter the bill formula in infix form\n";
    cout << "(e.g. (Consultation + RoomCharge + Medicine) - Insurance): ";
    if (!getline(cin, expr))
        return 1;
    if (!infixToPostfix(expr))
        return 1;
    cout << "\nPostfix expression : ";
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
    cout << "\n\n";
    readValues();
    if (!evaluate(result))
        return 1;
    cout << "\nFinal payable bill = " << fixed << setprecision(2) << result << "\n";
    return 0;
}
```

Set 30 – OPD (outpatient department) queue of a hospital

a. Enqueue a patient's token number
b. Dequeue and display the patient being called
c. Display the front and rear token numbers
d. Display the complete queue

Program (C++)

```
/* Set 30 - OPD (outpatient department) queue of a hospital */
// Compile: g++ -std=c++17 -Wall -Wextra set30.cpp -o set30
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Token No
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Token No : ";
    cin >> x.id;
}

void printItem(const Item &x)
{
    cout << "Token No " << x.id;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
    cout << "Token No " << x.id << " added to the queue.\n";
}

// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;        // queue became empty - reset
    else
        front++;
    cout << "Patient being called: ";
    printItem(x);
    cout << "\n";
}

void showFrontRear()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
```

```
        return;
    }
    cout << "Front token : ";
    printItem(q[front]);
    cout << "\nRear token : ";
    printItem(q[rear]);
    cout << "\n";
}

void display()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "FRONT -> ";
    for (int i = front; i <= rear; i++) {
        printItem(q[i]);
        if (i < rear)
            cout << " | ";
    }
    cout << " <- REAR\n";
}

int main()
{
    int choice;
    do {
        cout << "\n===== OPD QUEUE =====\n";
        cout << "1. Enqueue a patient's token number\n";
        cout << "2. Dequeue and display the patient being called\n";
        cout << "3. Display the front and rear token numbers\n";
        cout << "4. Display the complete queue\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            showFrontRear();
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 31 – Simulate the page navigation history of a warehouse management portal using a stack

a. Visit a new page (push URL)
b. Go back to the previous page (pop URL)
c. Display the current page
d. Show the entire browsing history stack

Program (C++)

```
/* Set 31 - Simulate the page navigation history of a warehouse management portal using a
   stack */
// Compile: g++ -std=c++17 -Wall -Wextra set31.cpp -o set31
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 10;               // maximum pages kept in the history

string history[MAX];
int top = -1;

bool isEmpty() { return top == -1; }
bool isFull()  { return top == MAX - 1; }

// a. Visit a new page (push URL)
void visit(const string &url)
{
    if (isFull()) {
        cout << "Stack Overflow! History is full.\n";
        return;
    }
    history[++top] = url;
    cout << "Visited: " << url << "\n";
}

// b. Go back to the previous page (pop URL)
void goBack()
{
    if (isEmpty()) {
        cout << "Stack Underflow! No page in history.\n";
        return;
    }
    if (top == 0) {
        cout << "This is the first page. Cannot go back.\n";
        return;
    }
    cout << "Left page: " << history[top] << "\n";
    top--;
    cout << "Went back to: " << history[top] << "\n";
}

// c. Display the current page
void currentPage()
{
    if (isEmpty())
        cout << "No page has been visited yet.\n";
    else
        cout << "Current page: " << history[top] << "\n";
}

// d. Show the entire browsing history stack (top to bottom)
void showHistory()
{
    if (isEmpty()) {
        cout << "Browsing history is empty.\n";
        return;
    }
    cout << "\n----- Browsing history (top to bottom) -----\n";
    for (int i = top; i >= 0; i--)
        cout << right << setw(2) << i + 1 << ". " << history[i]
             << (i == top ? " <-- current" : "") << "\n";
}
```

```
int main()
{
    int choice;
    string url;
    do {
        cout << "\n===== A WAREHOUSE MANAGEMENT PORTAL - BROWSING HISTORY =====\n";
        cout << "1. Visit a new page (push URL)\n";
        cout << "2. Go back to the previous page (pop URL)\n";
        cout << "3. Display the current page\n";
        cout << "4. Show the entire browsing history\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            cout << "Enter URL: ";
            cin >> url;
            visit(url);
            break;
        case 2:
            goBack();
            break;
        case 3:
            currentPage();
            break;
        case 4:
            showHistory();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 32 – Price formula: infix to postfix and evaluation

a. Convert the infix expression to postfix
b. Evaluate the postfix expression for given inputs

Program (C++)

```
/* Set 32 - Price formula: infix to postfix and evaluation */
// Compile: g++ -std=c++17 -Wall -Wextra set32.cpp -o set32
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <cctype>
#include <cmath>
#include <cstdlib>
using namespace std;

const int MAX = 100;
const bool TRACE = false;       // true = show intermediate stack contents

// variables the user must supply
const vector<string> required = {};

// ---------- stack of operators (used for infix -> postfix) ----------
char opStack[MAX];
int opTop = -1;

void opPush(char c)
{
    if (opTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    opStack[++opTop] = c;
```

```
}

char opPop()
{
    if (opTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return opStack[opTop--];
}

char opPeek()
{
    return (opTop == -1) ? '\0' : opStack[opTop];
}

// ---------- stack of values (used for postfix evaluation) ----------
double valStack[MAX];
int valTop = -1;

void valPush(double v)
{
    if (valTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    valStack[++valTop] = v;
}

double valPop()
{
    if (valTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return valStack[valTop--];
}

// ---------- postfix expression and variable table ----------
string post[MAX];
int pn = 0;

struct Var {
    string name;
    double value;
};

Var vars[MAX];
int vn = 0;

void emit(const string &t)
{
    if (pn == MAX) {
        cout << "Expression too long.\n";
        exit(1);
    }
    post[pn++] = t;
}

bool isOperand(const string &t)
{
    return isalnum((unsigned char)t[0]) || t[0] == '_' || t[0] == '.';
}

bool isNumber(const string &t)
{
    return isdigit((unsigned char)t[0]) || t[0] == '.';
}

int prec(char c)
{
    switch (c) {
    case '^': return 3;
    case '*':
    case '/': return 2;
```

```
    case '+':
    case '-': return 1;
    }
    return 0;
}

void showOpStack()
{
    cout << "[";
    for (int i = 0; i <= opTop; i++)
        cout << " " << opStack[i];
    cout << " ]";
}

void showPostfix()
{
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
}

void traceConv(const string &tok)
{
    if (!TRACE)
        return;
    cout << left << setw(14) << tok << "| ";
    showOpStack();
    cout << " | ";
    showPostfix();
    cout << "\n";
}

void showValStack()
{
    cout << "[";
    for (int i = 0; i <= valTop; i++)
        cout << " " << valStack[i];
    cout << " ]";
}

// Convert infix expression e to postfix (returns true on success)
bool infixToPostfix(const string &e)
{
    size_t i = 0;
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << setw(16) << "Operator stack"
             << "| " << "Postfix so far" << "\n";
    while (i < e.size()) {
        char ch = e[i];
        if (isspace((unsigned char)ch)) {
            i++;
        } else if (isalnum((unsigned char)ch) || ch == '_' || ch == '.') {
            string tok;
            while (i < e.size() && (isalnum((unsigned char)e[i]) || e[i] == '_' || e[i] == '.'))
                tok += e[i++];
            emit(tok);
            traceConv(tok);
        } else if (ch == '(') {
            opPush('(');
            traceConv("(");
            i++;
        } else if (ch == ')') {
            while (opTop != -1 && opPeek() != '(')
                emit(string(1, opPop()));
            if (opTop == -1) {
                cout << "Error: unbalanced ')'\n";
                return false;
            }
            opPop();                        // discard '('
            traceConv(")");
            i++;
        } else if (string("+-*/^").find(ch) != string::npos) {
            while (opTop != -1 && opPeek() != '(' &&
                   (prec(opPeek()) > prec(ch) || (prec(opPeek()) == prec(ch) && ch != '^')))
                emit(string(1, opPop()));
            opPush(ch);
```

```
            traceConv(string(1, ch));
            i++;
        } else {
            cout << "Error: invalid character '" << ch << "' in expression\n";
            return false;
        }
    }
    while (opTop != -1) {
        char c = opPop();
        if (c == '(') {
            cout << "Error: unbalanced '('\n";
            return false;
        }
        emit(string(1, c));
    }
    if (TRACE)
        traceConv("(end)");
    return true;
}

int findVar(const string &name)
{
    for (int i = 0; i < vn; i++) {
        if (vars[i].name == name)
            return i;
    }
    return -1;
}

void askVar(const string &name)
{
    double v;
    if (findVar(name) != -1)
        return;
    cout << "Enter value of " << name << " : ";
    if (!(cin >> v)) {
        cout << "\nInvalid input.\n";
        exit(1);
    }
    vars[vn].name = name;
    vars[vn].value = v;
    vn++;
}

// accept the required variables first, then any other variable used
void readValues()
{
    for (size_t i = 0; i < required.size(); i++)
        askVar(required[i]);
    for (int i = 0; i < pn; i++) {
        if (isOperand(post[i]) && !isNumber(post[i]))
            askVar(post[i]);
    }
}

double valueOf(const string &t)
{
    if (isNumber(t))
        return atof(t.c_str());
    return vars[findVar(t)].value;
}

// Evaluate the postfix expression using a stack (returns true on success)
bool evaluate(double &result)
{
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << "Value stack" << "\n";
    for (int i = 0; i < pn; i++) {
        const string &t = post[i];
        if (isOperand(t)) {
            valPush(valueOf(t));
        } else {
            if (valTop < 1) {
                cout << "Error: invalid expression (operand missing)\n";
                return false;
```

```
            }
            double b = valPop();
            double a = valPop();
            double r = 0;
            switch (t[0]) {
            case '+': r = a + b; break;
            case '-': r = a - b; break;
            case '*': r = a * b; break;
            case '/':
                if (b == 0) {
                    cout << "Error: division by zero\n";
                    return false;
                }
                r = a / b;
                break;
            case '^': r = pow(a, b); break;
            }
            valPush(r);
        }
        if (TRACE) {
            cout << left << setw(14) << t << "| ";
            showValStack();
            cout << "\n";
        }
    }
    if (valTop != 0) {
        cout << "Error: invalid expression\n";
        return false;
    }
    result = valPop();
    return true;
}

int main()
{
    string expr;
    double result;
    cout << "Enter the price formula in infix form\n";
    cout << "(e.g. (Cost * Qty) + Shipping - Discount): ";
    if (!getline(cin, expr))
        return 1;
    if (!infixToPostfix(expr))
        return 1;
    cout << "\nPostfix expression : ";
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
    cout << "\n\n";
    readValues();
    if (!evaluate(result))
        return 1;
    cout << "\nPrice = " << fixed << setprecision(2) << result << "\n";
    return 0;
}
```

Set 34 – Purchase Billing System

a. Read the price formula in infix form
b. Convert the infix expression to postfix using a stack
c. Display the postfix expression after conversion
d. Accept values for UnitCost, Quantity, Shipping, Discount
e. Evaluate the postfix expression using a stack
f. Display the final result
g. Display the intermediate stack contents during conversion and evaluation

Program (C++)

```
/* Set 34 - Purchase Billing System */
// Compile: g++ -std=c++17 -Wall -Wextra set34.cpp -o set34
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <cctype>
#include <cmath>
#include <cstdlib>
using namespace std;

const int MAX = 100;
const bool TRACE = true;       // true = show intermediate stack contents

// variables the user must supply
const vector<string> required = {"UnitCost", "Quantity", "Shipping", "Discount"};

// ---------- stack of operators (used for infix -> postfix) ----------
char opStack[MAX];
int opTop = -1;

void opPush(char c)
{
    if (opTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    opStack[++opTop] = c;
}

char opPop()
{
    if (opTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return opStack[opTop--];
}

char opPeek()
{
    return (opTop == -1) ? '\0' : opStack[opTop];
}

// ---------- stack of values (used for postfix evaluation) ----------
double valStack[MAX];
int valTop = -1;

void valPush(double v)
{
    if (valTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    valStack[++valTop] = v;
}

double valPop()
{
    if (valTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
```

```
    }
    return valStack[valTop--];
}

// ---------- postfix expression and variable table ----------
string post[MAX];
int pn = 0;

struct Var {
    string name;
    double value;
};

Var vars[MAX];
int vn = 0;

void emit(const string &t)
{
    if (pn == MAX) {
        cout << "Expression too long.\n";
        exit(1);
    }
    post[pn++] = t;
}

bool isOperand(const string &t)
{
    return isalnum((unsigned char)t[0]) || t[0] == '_' || t[0] == '.';
}

bool isNumber(const string &t)
{
    return isdigit((unsigned char)t[0]) || t[0] == '.';
}

int prec(char c)
{
    switch (c) {
    case '^': return 3;
    case '*':
    case '/': return 2;
    case '+':
    case '-': return 1;
    }
    return 0;
}

void showOpStack()
{
    cout << "[";
    for (int i = 0; i <= opTop; i++)
        cout << " " << opStack[i];
    cout << " ]";
}

void showPostfix()
{
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
}

void traceConv(const string &tok)
{
    if (!TRACE)
        return;
    cout << left << setw(14) << tok << "| ";
    showOpStack();
    cout << " | ";
    showPostfix();
    cout << "\n";
}

void showValStack()
{
    cout << "[";
```

```
    for (int i = 0; i <= valTop; i++)
        cout << " " << valStack[i];
    cout << " ]";
}

// Convert infix expression e to postfix (returns true on success)
bool infixToPostfix(const string &e)
{
    size_t i = 0;
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << setw(16) << "Operator stack"
             << "| " << "Postfix so far" << "\n";
    while (i < e.size()) {
        char ch = e[i];
        if (isspace((unsigned char)ch)) {
            i++;
        } else if (isalnum((unsigned char)ch) || ch == '_' || ch == '.') {
            string tok;
            while (i < e.size() && (isalnum((unsigned char)e[i]) || e[i] == '_' || e[i] == '.'))
                tok += e[i++];
            emit(tok);
            traceConv(tok);
        } else if (ch == '(') {
            opPush('(');
            traceConv("(");
            i++;
        } else if (ch == ')') {
            while (opTop != -1 && opPeek() != '(')
                emit(string(1, opPop()));
            if (opTop == -1) {
                cout << "Error: unbalanced ')'\n";
                return false;
            }
            opPop();                        // discard '('
            traceConv(")");
            i++;
        } else if (string("+-*/^").find(ch) != string::npos) {
            while (opTop != -1 && opPeek() != '(' &&
                   (prec(opPeek()) > prec(ch) || (prec(opPeek()) == prec(ch) && ch != '^')))
                emit(string(1, opPop()));
            opPush(ch);
            traceConv(string(1, ch));
            i++;
        } else {
            cout << "Error: invalid character '" << ch << "' in expression\n";
            return false;
        }
    }
    while (opTop != -1) {
        char c = opPop();
        if (c == '(') {
            cout << "Error: unbalanced '('\n";
            return false;
        }
        emit(string(1, c));
    }
    if (TRACE)
        traceConv("(end)");
    return true;
}

int findVar(const string &name)
{
    for (int i = 0; i < vn; i++) {
        if (vars[i].name == name)
            return i;
    }
    return -1;
}

void askVar(const string &name)
{
    double v;
    if (findVar(name) != -1)
        return;
```

```
    cout << "Enter value of " << name << " : ";
    if (!(cin >> v)) {
        cout << "\nInvalid input.\n";
        exit(1);
    }
    vars[vn].name = name;
    vars[vn].value = v;
    vn++;
}

// accept the required variables first, then any other variable used
void readValues()
{
    for (size_t i = 0; i < required.size(); i++)
        askVar(required[i]);
    for (int i = 0; i < pn; i++) {
        if (isOperand(post[i]) && !isNumber(post[i]))
            askVar(post[i]);
    }
}

double valueOf(const string &t)
{
    if (isNumber(t))
        return atof(t.c_str());
    return vars[findVar(t)].value;
}

// Evaluate the postfix expression using a stack (returns true on success)
bool evaluate(double &result)
{
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << "Value stack" << "\n";
    for (int i = 0; i < pn; i++) {
        const string &t = post[i];
        if (isOperand(t)) {
            valPush(valueOf(t));
        } else {
            if (valTop < 1) {
                cout << "Error: invalid expression (operand missing)\n";
                return false;
            }
            double b = valPop();
            double a = valPop();
            double r = 0;
            switch (t[0]) {
            case '+': r = a + b; break;
            case '-': r = a - b; break;
            case '*': r = a * b; break;
            case '/':
                if (b == 0) {
                    cout << "Error: division by zero\n";
                    return false;
                }
                r = a / b;
                break;
            case '^': r = pow(a, b); break;
            }
            valPush(r);
        }
        if (TRACE) {
            cout << left << setw(14) << t << "| ";
            showValStack();
            cout << "\n";
        }
    }
    if (valTop != 0) {
        cout << "Error: invalid expression\n";
        return false;
    }
    result = valPop();
    return true;
}

int main()
```

```
{
    string expr;
    double result;
    cout << "Enter the price formula in infix form\n";
    cout << "(e.g. (UnitCost * Quantity) + Shipping - Discount): ";
    if (!getline(cin, expr))
        return 1;
    if (!infixToPostfix(expr))
        return 1;
    cout << "\nPostfix expression : ";
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
    cout << "\n\n";
    readValues();
    if (!evaluate(result))
        return 1;
    cout << "\nFinal payable amount = " << fixed << setprecision(2) << result << "\n";
    return 0;
}
```

Set 35 – Goods dispatch queue of a warehouse

a. Enqueue an order number
b. Dequeue and display the order being dispatched
c. Display the front and rear order numbers
d. Display the complete queue

Program (C++)

```
/* Set 35 - Goods dispatch queue of a warehouse */
// Compile: g++ -std=c++17 -Wall -Wextra set35.cpp -o set35
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Order No
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Order No : ";
    cin >> x.id;
}

void printItem(const Item &x)
{
    cout << "Order No " << x.id;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
    cout << "Order No " << x.id << " added to the queue.\n";
}
```

```
// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;         // queue became empty - reset
    else
        front++;
    cout << "Order being dispatched: ";
    printItem(x);
    cout << "\n";
}

void showFrontRear()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "Front order : ";
    printItem(q[front]);
    cout << "\nRear order : ";
    printItem(q[rear]);
    cout << "\n";
}

void display()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "FRONT -> ";
    for (int i = front; i <= rear; i++) {
        printItem(q[i]);
        if (i < rear)
            cout << " | ";
    }
    cout << " <- REAR\n";
}

int main()
{
    int choice;
    do {
        cout << "\n===== GOODS DISPATCH QUEUE =====\n";
        cout << "1. Enqueue an order number\n";
        cout << "2. Dequeue and display the order being dispatched\n";
        cout << "3. Display the front and rear order numbers\n";
        cout << "4. Display the complete queue\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            showFrontRear();
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
```

```
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 36 – Simulate the page navigation history of an internet banking portal using a stack

a. Visit a new page (push URL)
b. Go back to the previous page (pop URL)
c. Display the current page
d. Show the entire browsing history stack

Program (C++)

```
/* Set 36 - Simulate the page navigation history of an internet banking portal using a
   stack */
// Compile: g++ -std=c++17 -Wall -Wextra set36.cpp -o set36
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 10;               // maximum pages kept in the history

string history[MAX];
int top = -1;

bool isEmpty() { return top == -1; }
bool isFull()  { return top == MAX - 1; }

// a. Visit a new page (push URL)
void visit(const string &url)
{
    if (isFull()) {
        cout << "Stack Overflow! History is full.\n";
        return;
    }
    history[++top] = url;
    cout << "Visited: " << url << "\n";
}

// b. Go back to the previous page (pop URL)
void goBack()
{
    if (isEmpty()) {
        cout << "Stack Underflow! No page in history.\n";
        return;
    }
    if (top == 0) {
        cout << "This is the first page. Cannot go back.\n";
        return;
    }
    cout << "Left page: " << history[top] << "\n";
    top--;
    cout << "Went back to: " << history[top] << "\n";
}

// c. Display the current page
void currentPage()
{
    if (isEmpty())
        cout << "No page has been visited yet.\n";
    else
        cout << "Current page: " << history[top] << "\n";
}

// d. Show the entire browsing history stack (top to bottom)
void showHistory()
{
    if (isEmpty()) {
        cout << "Browsing history is empty.\n";
        return;
```

```
    }
    cout << "\n----- Browsing history (top to bottom) -----\n";
    for (int i = top; i >= 0; i--)
        cout << right << setw(2) << i + 1 << ". " << history[i]
             << (i == top ? " <-- current" : "") << "\n";
}

int main()
{
    int choice;
    string url;
    do {
        cout << "\n===== AN INTERNET BANKING PORTAL - BROWSING HISTORY =====\n";
        cout << "1. Visit a new page (push URL)\n";
        cout << "2. Go back to the previous page (pop URL)\n";
        cout << "3. Display the current page\n";
        cout << "4. Show the entire browsing history\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            cout << "Enter URL: ";
            cin >> url;
            visit(url);
            break;
        case 2:
            goBack();
            break;
        case 3:
            currentPage();
            break;
        case 4:
            showHistory();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 37 – Interest formula: infix to postfix and evaluation

a. Convert the infix expression to postfix

b. Evaluate the postfix expression for given inputs

Program (C++)

```
/* Set 37 - Interest formula: infix to postfix and evaluation */
// Compile: g++ -std=c++17 -Wall -Wextra set37.cpp -o set37
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <cctype>
#include <cmath>
#include <cstdlib>
using namespace std;

const int MAX = 100;
const bool TRACE = false;       // true = show intermediate stack contents

// variables the user must supply
const vector<string> required = {};

// ---------- stack of operators (used for infix -> postfix) ----------
char opStack[MAX];
int opTop = -1;
void opPush(char c)
```

```
{
    if (opTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    opStack[++opTop] = c;
}

char opPop()
{
    if (opTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return opStack[opTop--];
}

char opPeek()
{
    return (opTop == -1) ? '\0' : opStack[opTop];
}

// ---------- stack of values (used for postfix evaluation) ----------
double valStack[MAX];
int valTop = -1;

void valPush(double v)
{
    if (valTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    valStack[++valTop] = v;
}

double valPop()
{
    if (valTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return valStack[valTop--];
}

// ---------- postfix expression and variable table ----------
string post[MAX];
int pn = 0;

struct Var {
    string name;
    double value;
};

Var vars[MAX];
int vn = 0;

void emit(const string &t)
{
    if (pn == MAX) {
        cout << "Expression too long.\n";
        exit(1);
    }
    post[pn++] = t;
}

bool isOperand(const string &t)
{
    return isalnum((unsigned char)t[0]) || t[0] == '_' || t[0] == '.';
}

bool isNumber(const string &t)
{
    return isdigit((unsigned char)t[0]) || t[0] == '.';
}
int prec(char c)
```

```
{
    switch (c) {
    case '^': return 3;
    case '*':
    case '/': return 2;
    case '+':
    case '-': return 1;
    }
    return 0;
}

void showOpStack()
{
    cout << "[";
    for (int i = 0; i <= opTop; i++)
        cout << " " << opStack[i];
    cout << " ]";
}

void showPostfix()
{
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
}

void traceConv(const string &tok)
{
    if (!TRACE)
        return;
    cout << left << setw(14) << tok << "| ";
    showOpStack();
    cout << " | ";
    showPostfix();
    cout << "\n";
}

void showValStack()
{
    cout << "[";
    for (int i = 0; i <= valTop; i++)
        cout << " " << valStack[i];
    cout << " ]";
}

// Convert infix expression e to postfix (returns true on success)
bool infixToPostfix(const string &e)
{
    size_t i = 0;
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << setw(16) << "Operator stack"
             << "| " << "Postfix so far" << "\n";
    while (i < e.size()) {
        char ch = e[i];
        if (isspace((unsigned char)ch)) {
            i++;
        } else if (isalnum((unsigned char)ch) || ch == '_' || ch == '.') {
            string tok;
            while (i < e.size() && (isalnum((unsigned char)e[i]) || e[i] == '_' || e[i] == '.'))
                tok += e[i++];
            emit(tok);
            traceConv(tok);
        } else if (ch == '(') {
            opPush('(');
            traceConv("(");
            i++;
        } else if (ch == ')') {
            while (opTop != -1 && opPeek() != '(')
                emit(string(1, opPop()));
            if (opTop == -1) {
                cout << "Error: unbalanced ')'\n";
                return false;
            }
            opPop();                        // discard '('
            traceConv(")");
            i++;
```

```
        } else if (string("+-*/^").find(ch) != string::npos) {
            while (opTop != -1 && opPeek() != '(' &&
                   (prec(opPeek()) > prec(ch) || (prec(opPeek()) == prec(ch) && ch != '^')))
                emit(string(1, opPop()));
            opPush(ch);
            traceConv(string(1, ch));
            i++;
        } else {
            cout << "Error: invalid character '" << ch << "' in expression\n";
            return false;
        }
    }
    while (opTop != -1) {
        char c = opPop();
        if (c == '(') {
            cout << "Error: unbalanced '('\n";
            return false;
        }
        emit(string(1, c));
    }
    if (TRACE)
        traceConv("(end)");
    return true;
}

int findVar(const string &name)
{
    for (int i = 0; i < vn; i++) {
        if (vars[i].name == name)
            return i;
    }
    return -1;
}

void askVar(const string &name)
{
    double v;
    if (findVar(name) != -1)
        return;
    cout << "Enter value of " << name << " : ";
    if (!(cin >> v)) {
        cout << "\nInvalid input.\n";
        exit(1);
    }
    vars[vn].name = name;
    vars[vn].value = v;
    vn++;
}

// accept the required variables first, then any other variable used
void readValues()
{
    for (size_t i = 0; i < required.size(); i++)
        askVar(required[i]);
    for (int i = 0; i < pn; i++) {
        if (isOperand(post[i]) && !isNumber(post[i]))
            askVar(post[i]);
    }
}

double valueOf(const string &t)
{
    if (isNumber(t))
        return atof(t.c_str());
    return vars[findVar(t)].value;
}

// Evaluate the postfix expression using a stack (returns true on success)
bool evaluate(double &result)
{
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << "Value stack" << "\n";
    for (int i = 0; i < pn; i++) {
        const string &t = post[i];
        if (isOperand(t)) {
```

```
            valPush(valueOf(t));
        } else {
            if (valTop < 1) {
                cout << "Error: invalid expression (operand missing)\n";
                return false;
            }
            double b = valPop();
            double a = valPop();
            double r = 0;
            switch (t[0]) {
            case '+': r = a + b; break;
            case '-': r = a - b; break;
            case '*': r = a * b; break;
            case '/':
                if (b == 0) {
                    cout << "Error: division by zero\n";
                    return false;
                }
                r = a / b;
                break;
            case '^': r = pow(a, b); break;
            }
            valPush(r);
        }
        if (TRACE) {
            cout << left << setw(14) << t << "| ";
            showValStack();
            cout << "\n";
        }
    }
    if (valTop != 0) {
        cout << "Error: invalid expression\n";
        return false;
    }
    result = valPop();
    return true;
}

int main()
{
    string expr;
    double result;
    cout << "Enter the interest formula in infix form\n";
    cout << "(e.g. (Principal * Rate * Time) / 100): ";
    if (!getline(cin, expr))
        return 1;
    if (!infixToPostfix(expr))
        return 1;
    cout << "\nPostfix expression : ";
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
    cout << "\n\n";
    readValues();
    if (!evaluate(result))
        return 1;
    cout << "\nInterest = " << fixed << setprecision(2) << result << "\n";
    return 0;
}
```

Set 39 – Loan Billing System

a. Read the loan formula in infix form
b. Convert the infix expression to postfix using a stack
c. Display the postfix expression after conversion
d. Accept values for Principal, Interest, ProcessingFee, Rebate
e. Evaluate the postfix expression using a stack
f. Display the final result
g. Display the intermediate stack contents during conversion and evaluation

Program (C++)

```
/* Set 39 - Loan Billing System */
// Compile: g++ -std=c++17 -Wall -Wextra set39.cpp -o set39
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <cctype>
#include <cmath>
#include <cstdlib>
using namespace std;

const int MAX = 100;
const bool TRACE = true;       // true = show intermediate stack contents

// variables the user must supply
const vector<string> required = {"Principal", "Interest", "ProcessingFee", "Rebate"};

// ---------- stack of operators (used for infix -> postfix) ----------
char opStack[MAX];
int opTop = -1;

void opPush(char c)
{
    if (opTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    opStack[++opTop] = c;
}

char opPop()
{
    if (opTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
    }
    return opStack[opTop--];
}

char opPeek()
{
    return (opTop == -1) ? '\0' : opStack[opTop];
}

// ---------- stack of values (used for postfix evaluation) ----------
double valStack[MAX];
int valTop = -1;

void valPush(double v)
{
    if (valTop == MAX - 1) {
        cout << "Stack Overflow\n";
        exit(1);
    }
    valStack[++valTop] = v;
}

double valPop()
{
    if (valTop == -1) {
        cout << "Stack Underflow\n";
        exit(1);
```

```
    }
    return valStack[valTop--];
}

// ---------- postfix expression and variable table ----------
string post[MAX];
int pn = 0;

struct Var {
    string name;
    double value;
};

Var vars[MAX];
int vn = 0;

void emit(const string &t)
{
    if (pn == MAX) {
        cout << "Expression too long.\n";
        exit(1);
    }
    post[pn++] = t;
}

bool isOperand(const string &t)
{
    return isalnum((unsigned char)t[0]) || t[0] == '_' || t[0] == '.';
}

bool isNumber(const string &t)
{
    return isdigit((unsigned char)t[0]) || t[0] == '.';
}

int prec(char c)
{
    switch (c) {
    case '^': return 3;
    case '*':
    case '/': return 2;
    case '+':
    case '-': return 1;
    }
    return 0;
}

void showOpStack()
{
    cout << "[";
    for (int i = 0; i <= opTop; i++)
        cout << " " << opStack[i];
    cout << " ]";
}

void showPostfix()
{
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
}

void traceConv(const string &tok)
{
    if (!TRACE)
        return;
    cout << left << setw(14) << tok << "| ";
    showOpStack();
    cout << " | ";
    showPostfix();
    cout << "\n";
}

void showValStack()
{
    cout << "[";
```

```
    for (int i = 0; i <= valTop; i++)
        cout << " " << valStack[i];
    cout << " ]";
}

// Convert infix expression e to postfix (returns true on success)
bool infixToPostfix(const string &e)
{
    size_t i = 0;
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << setw(16) << "Operator stack"
             << "| " << "Postfix so far" << "\n";
    while (i < e.size()) {
        char ch = e[i];
        if (isspace((unsigned char)ch)) {
            i++;
        } else if (isalnum((unsigned char)ch) || ch == '_' || ch == '.') {
            string tok;
            while (i < e.size() && (isalnum((unsigned char)e[i]) || e[i] == '_' || e[i] == '.'))
                tok += e[i++];
            emit(tok);
            traceConv(tok);
        } else if (ch == '(') {
            opPush('(');
            traceConv("(");
            i++;
        } else if (ch == ')') {
            while (opTop != -1 && opPeek() != '(')
                emit(string(1, opPop()));
            if (opTop == -1) {
                cout << "Error: unbalanced ')'\n";
                return false;
            }
            opPop();                        // discard '('
            traceConv(")");
            i++;
        } else if (string("+-*/^").find(ch) != string::npos) {
            while (opTop != -1 && opPeek() != '(' &&
                   (prec(opPeek()) > prec(ch) || (prec(opPeek()) == prec(ch) && ch != '^')))
                emit(string(1, opPop()));
            opPush(ch);
            traceConv(string(1, ch));
            i++;
        } else {
            cout << "Error: invalid character '" << ch << "' in expression\n";
            return false;
        }
    }
    while (opTop != -1) {
        char c = opPop();
        if (c == '(') {
            cout << "Error: unbalanced '('\n";
            return false;
        }
        emit(string(1, c));
    }
    if (TRACE)
        traceConv("(end)");
    return true;
}

int findVar(const string &name)
{
    for (int i = 0; i < vn; i++) {
        if (vars[i].name == name)
            return i;
    }
    return -1;
}

void askVar(const string &name)
{
    double v;
    if (findVar(name) != -1)
        return;
```

```
    cout << "Enter value of " << name << " : ";
    if (!(cin >> v)) {
        cout << "\nInvalid input.\n";
        exit(1);
    }
    vars[vn].name = name;
    vars[vn].value = v;
    vn++;
}

// accept the required variables first, then any other variable used
void readValues()
{
    for (size_t i = 0; i < required.size(); i++)
        askVar(required[i]);
    for (int i = 0; i < pn; i++) {
        if (isOperand(post[i]) && !isNumber(post[i]))
            askVar(post[i]);
    }
}

double valueOf(const string &t)
{
    if (isNumber(t))
        return atof(t.c_str());
    return vars[findVar(t)].value;
}

// Evaluate the postfix expression using a stack (returns true on success)
bool evaluate(double &result)
{
    if (TRACE)
        cout << "\n" << left << setw(14) << "Token" << "| " << "Value stack" << "\n";
    for (int i = 0; i < pn; i++) {
        const string &t = post[i];
        if (isOperand(t)) {
            valPush(valueOf(t));
        } else {
            if (valTop < 1) {
                cout << "Error: invalid expression (operand missing)\n";
                return false;
            }
            double b = valPop();
            double a = valPop();
            double r = 0;
            switch (t[0]) {
            case '+': r = a + b; break;
            case '-': r = a - b; break;
            case '*': r = a * b; break;
            case '/':
                if (b == 0) {
                    cout << "Error: division by zero\n";
                    return false;
                }
                r = a / b;
                break;
            case '^': r = pow(a, b); break;
            }
            valPush(r);
        }
        if (TRACE) {
            cout << left << setw(14) << t << "| ";
            showValStack();
            cout << "\n";
        }
    }
    if (valTop != 0) {
        cout << "Error: invalid expression\n";
        return false;
    }
    result = valPop();
    return true;
}

int main()
```

```
{
    string expr;
    double result;
    cout << "Enter the loan formula in infix form\n";
    cout << "(e.g. (Principal + Interest + ProcessingFee) - Rebate): ";
    if (!getline(cin, expr))
        return 1;
    if (!infixToPostfix(expr))
        return 1;
    cout << "\nPostfix expression : ";
    for (int i = 0; i < pn; i++)
        cout << post[i] << " ";
    cout << "\n\n";
    readValues();
    if (!evaluate(result))
        return 1;
    cout << "\nFinal payable loan amount = " << fixed << setprecision(2) << result << "\n";
    return 0;
}
```

Set 40 – Teller counter queue of a bank

a. Enqueue a customer's token number
b. Dequeue and display the customer being served
c. Display the front and rear token numbers
d. Display the complete queue

Program (C++)

```
/* Set 40 - Teller counter queue of a bank */
// Compile: g++ -std=c++17 -Wall -Wextra set40.cpp -o set40
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Token No
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Token No : ";
    cin >> x.id;
}

void printItem(const Item &x)
{
    cout << "Token No " << x.id;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
    cout << "Token No " << x.id << " added to the queue.\n";
}
```

```
// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;         // queue became empty - reset
    else
        front++;
    cout << "Customer being served: ";
    printItem(x);
    cout << "\n";
}

void showFrontRear()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "Front token : ";
    printItem(q[front]);
    cout << "\nRear token : ";
    printItem(q[rear]);
    cout << "\n";
}

void display()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "FRONT -> ";
    for (int i = front; i <= rear; i++) {
        printItem(q[i]);
        if (i < rear)
            cout << " | ";
    }
    cout << " <- REAR\n";
}

int main()
{
    int choice;
    do {
        cout << "\n===== BANK TELLER QUEUE =====\n";
        cout << "1. Enqueue a customer's token number\n";
        cout << "2. Dequeue and display the customer being served\n";
        cout << "3. Display the front and rear token numbers\n";
        cout << "4. Display the complete queue\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            showFrontRear();
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
```

```
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 43 – Passenger list of a flight - linear search by Name

a. Read the records of N passengers
b. Search a passenger by Name using linear search
c. Display the record if found, otherwise display "Passenger not found"
d. Display the number of comparisons made

Program (C++)

```
/* Set 43 - Passenger list of a flight - linear search by Name */
// Compile: g++ -std=c++17 -Wall -Wextra set43.cpp -o set43
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Passenger {
    int ticket;    // Ticket No
    string name;   // Name
    string seat;   // Seat No
};

Passenger rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Ticket No : ";
        cin >> rec[i].ticket;
        cout << " Enter Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Seat No : ";
        cin >> ws;
        getline(cin, rec[i].seat);
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Ticket No" << ": " << rec[i].ticket << "\n";
    cout << " " << left << setw(16) << "Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "Seat No" << ": " << rec[i].seat << "\n";
}

// linear search : returns index of the first match, or -1
int linearSearch(const string &key, int &comparisons)
{
    comparisons = 0;
    for (int i = 0; i < n; i++) {
        comparisons++;
        if (rec[i].name == key)
            return i;
    }
    return -1;
}

int main()
{
    int idx, comparisons;
    string key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
```

```
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    cout << "Enter Name to search: ";
    cin >> ws;
    getline(cin, key);
    idx = linearSearch(key, comparisons);
    if (idx == -1) {
        cout << "\nPassenger not found\n";
        cout << "Number of comparisons made = " << comparisons << "\n";
    } else {
        cout << "\nRecord found:\n";
        printRecord(idx);
        cout << "Number of comparisons made = " << comparisons << "\n";
    }
    return 0;
}
```

Set 44 – Hospital patient records - linear search by Name (repeated names)

a. Read the patient records into an array
b. Search a patient by Name using linear search
c. Display all patients matching the given Name
d. Display the total number of matches found

Program (C++)

```
/* Set 44 - Hospital patient records - linear search by Name (repeated names) */
// Compile: g++ -std=c++17 -Wall -Wextra set44.cpp -o set44
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Patient {
    int id;        // Patient ID
    string name;   // Name
    string ward;   // Ward
};

Patient rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Patient ID : ";
        cin >> rec[i].id;
        cout << " Enter Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Ward : ";
        cin >> ws;
        getline(cin, rec[i].ward);
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Patient ID" << ": " << rec[i].id << "\n";
    cout << " " << left << setw(16) << "Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "Ward" << ": " << rec[i].ward << "\n";
}

int main()
{
    int matches = 0;
```

```
    string key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    cout << "Enter Name to search: ";
    cin >> ws;
    getline(cin, key);
    cout << "\nMatching records:\n";
    for (int i = 0; i < n; i++) {               // values may repeat, so scan all
        if (rec[i].name == key) {
            matches++;
            cout << "Match " << matches << " (position " << i + 1 << ")\n";
            printRecord(i);
        }
    }
    if (matches == 0) {
        cout << "Patient not found\n";
    } else {
        cout << "\nTotal number of matches found = " << matches << "\n";
    }
    return 0;
}
```

Set 45 – Bank transaction history - linear search by Transaction ID

a. Read N transactions
b. Search for a transaction by Transaction ID using linear search
c. Display the position (index) and details of the transaction
d. Display "Transaction not found" if absent

Program (C++)

```
/* Set 45 - Bank transaction history - linear search by Transaction ID */
// Compile: g++ -std=c++17 -Wall -Wextra set45.cpp -o set45
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Transaction {
    int id;         // Transaction ID
    string type;    // Type
    float amount;   // Amount
};

Transaction rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Transaction ID : ";
        cin >> rec[i].id;
        cout << " Enter Type : ";
        cin >> ws;
        getline(cin, rec[i].type);
        cout << " Enter Amount : ";
        cin >> rec[i].amount;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Transaction ID" << ": " << rec[i].id << "\n";
    cout << " " << left << setw(16) << "Type" << ": " << rec[i].type << "\n";
```

```
    cout << " " << left << setw(16) << "Amount" << ": " << rec[i].amount << "\n";
}

// linear search : returns index of the first match, or -1
int linearSearch(int key, int &comparisons)
{
    comparisons = 0;
    for (int i = 0; i < n; i++) {
        comparisons++;
        if (rec[i].id == key)
            return i;
    }
    return -1;
}

int main()
{
    int idx, comparisons;
    int key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    cout << "Enter Transaction ID to search: ";
    cin >> key;
    idx = linearSearch(key, comparisons);
    if (idx == -1) {
        cout << "\nTransaction not found\n";
    } else {
        cout << "\nRecord found:\n";
        cout << " " << left << setw(16) << "Index" << ": " << idx << "\n";
        cout << " " << left << setw(16) << "Position" << ": " << idx + 1 << "\n";
        printRecord(idx);
    }
    return 0;
}
```

Set 46 – Library catalogue - linear search by Title

a. Read the records of N books
b. Search a book by Title using linear search
c. Display the record if found
d. Display the number of comparisons made

Program (C++)

```
/* Set 46 - Library catalogue - linear search by Title */
// Compile: g++ -std=c++17 -Wall -Wextra set46.cpp -o set46
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Book {
    int id;          // Book ID
    string title;    // Title
    string author;   // Author
};

Book rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Book ID : ";
```

```
        cin >> rec[i].id;
        cout << " Enter Title : ";
        cin >> ws;
        getline(cin, rec[i].title);
        cout << " Enter Author : ";
        cin >> ws;
        getline(cin, rec[i].author);
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Book ID" << ": " << rec[i].id << "\n";
    cout << " " << left << setw(16) << "Title" << ": " << rec[i].title << "\n";
    cout << " " << left << setw(16) << "Author" << ": " << rec[i].author << "\n";
}

// linear search : returns index of the first match, or -1
int linearSearch(const string &key, int &comparisons)
{
    comparisons = 0;
    for (int i = 0; i < n; i++) {
        comparisons++;
        if (rec[i].title == key)
            return i;
    }
    return -1;
}

int main()
{
    int idx, comparisons;
    string key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    cout << "Enter Title to search: ";
    cin >> ws;
    getline(cin, key);
    idx = linearSearch(key, comparisons);
    if (idx == -1) {
        cout << "\nBook not found\n";
        cout << "Number of comparisons made = " << comparisons << "\n";
    } else {
        cout << "\nRecord found:\n";
        printRecord(idx);
        cout << "Number of comparisons made = " << comparisons << "\n";
    }
    return 0;
}
```

Set 47 – Online store product list - linear search by Name

a. Read N products
b. Search a product by Name using linear search
c. Display the product details if found
d. Display "Out of catalogue" otherwise

Program (C++)

```
/* Set 47 - Online store product list - linear search by Name */
// Compile: g++ -std=c++17 -Wall -Wextra set47.cpp -o set47
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Product {
    int id;        // Product ID
    string name;   // Name
    float price;   // Price
};

Product rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Product ID : ";
        cin >> rec[i].id;
        cout << " Enter Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Price : ";
        cin >> rec[i].price;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Product ID" << ": " << rec[i].id << "\n";
    cout << " " << left << setw(16) << "Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "Price" << ": " << rec[i].price << "\n";
}

// linear search : returns index of the first match, or -1
int linearSearch(const string &key, int &comparisons)
{
    comparisons = 0;
    for (int i = 0; i < n; i++) {
        comparisons++;
        if (rec[i].name == key)
            return i;
    }
    return -1;
}

int main()
{
    int idx, comparisons;
    string key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    cout << "Enter Name to search: ";
    cin >> ws;
    getline(cin, key);
```

```
    idx = linearSearch(key, comparisons);
    if (idx == -1) {
        cout << "\nOut of catalogue\n";
    } else {
        cout << "\nRecord found:\n";
        printRecord(idx);
    }
    return 0;
}
```

Set 48 – Student marks list - linear search by Roll No

a. Read the student records
b. Search a student by Roll No using linear search
c. Display the student's record
d. Count and display how many students scored the same marks as the student found

Program (C++)

```
/* Set 48 - Student marks list - linear search by Roll No */
// Compile: g++ -std=c++17 -Wall -Wextra set48.cpp -o set48
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Student {
    int roll;      // Roll No
    string name;   // Name
    int marks;     // Marks
};

Student rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Roll No : ";
        cin >> rec[i].roll;
        cout << " Enter Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Marks : ";
        cin >> rec[i].marks;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Roll No" << ": " << rec[i].roll << "\n";
    cout << " " << left << setw(16) << "Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "Marks" << ": " << rec[i].marks << "\n";
}

// linear search : returns index of the first match, or -1
int linearSearch(int key, int &comparisons)
{
    comparisons = 0;
    for (int i = 0; i < n; i++) {
        comparisons++;
        if (rec[i].roll == key)
            return i;
    }
    return -1;
}

int main()
{
    int idx, comparisons;
```

```
    int key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    cout << "Enter Roll No to search: ";
    cin >> key;
    idx = linearSearch(key, comparisons);
    if (idx == -1) {
        cout << "\nStudent not found\n";
    } else {
        cout << "\nRecord found:\n";
        printRecord(idx);
        int same = 0;
        for (int i = 0; i < n; i++) {
            if (i != idx && rec[i].marks == rec[idx].marks)
                same++;
        }
        cout << "\nNumber of other students with the same marks (" << rec[idx].marks
             << ") = " << same << "\n";
    }
    return 0;
}
```

Set 49 – Hotel guest register - linear search by Name

a. Read N guest records
b. Search for a guest by Name using linear search
c. Display the room number and days stayed
d. Display the position of the match in the register

Program (C++)

```
/* Set 49 - Hotel guest register - linear search by Name */
// Compile: g++ -std=c++17 -Wall -Wextra set49.cpp -o set49
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Guest {
    int room;      // Room No
    string name;   // Guest Name
    int days;      // Days Stayed
};

Guest rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Room No : ";
        cin >> rec[i].room;
        cout << " Enter Guest Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Days Stayed : ";
        cin >> rec[i].days;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Room No" << ": " << rec[i].room << "\n";
    cout << " " << left << setw(16) << "Guest Name" << ": " << rec[i].name << "\n";
```

```
    cout << " " << left << setw(16) << "Days Stayed" << ": " << rec[i].days << "\n";
}

// linear search : returns index of the first match, or -1
int linearSearch(const string &key, int &comparisons)
{
    comparisons = 0;
    for (int i = 0; i < n; i++) {
        comparisons++;
        if (rec[i].name == key)
            return i;
    }
    return -1;
}

int main()
{
    int idx, comparisons;
    string key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    cout << "Enter Guest Name to search: ";
    cin >> ws;
    getline(cin, key);
    idx = linearSearch(key, comparisons);
    if (idx == -1) {
        cout << "\nGuest not found\n";
    } else {
        cout << "\nRecord found:\n";
        cout << " " << left << setw(16) << "Room No" << ": " << rec[idx].room << "\n";
        cout << " " << left << setw(16) << "Days Stayed" << ": " << rec[idx].days << "\n";
        cout << " " << left << setw(16) << "Position" << ": " << idx + 1 << "\n";
    }
    return 0;
}
```

Set 50 – Railway reservation list - linear search by PNR

a. Read N reservations
b. Search by PNR using linear search
c. Display the booking details if found
d. Display the number of comparisons made

Program (C++)

```
/* Set 50 - Railway reservation list - linear search by PNR */
// Compile: g++ -std=c++17 -Wall -Wextra set50.cpp -o set50
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Reservation {
    string pnr;    // PNR
    string name;   // Passenger Name
    int train;     // Train No
};

Reservation rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
```

```
        cout << " Enter PNR : ";
        cin >> ws;
        getline(cin, rec[i].pnr);
        cout << " Enter Passenger Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Train No : ";
        cin >> rec[i].train;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "PNR" << ": " << rec[i].pnr << "\n";
    cout << " " << left << setw(16) << "Passenger Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "Train No" << ": " << rec[i].train << "\n";
}

// linear search : returns index of the first match, or -1
int linearSearch(const string &key, int &comparisons)
{
    comparisons = 0;
    for (int i = 0; i < n; i++) {
        comparisons++;
        if (rec[i].pnr == key)
            return i;
    }
    return -1;
}

int main()
{
    int idx, comparisons;
    string key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    cout << "Enter PNR to search: ";
    cin >> ws;
    getline(cin, key);
    idx = linearSearch(key, comparisons);
    if (idx == -1) {
        cout << "\nPNR not found\n";
        cout << "Number of comparisons made = " << comparisons << "\n";
    } else {
        cout << "\nRecord found:\n";
        printRecord(idx);
        cout << "Number of comparisons made = " << comparisons << "\n";
    }
    return 0;
}
```

Set 51 – Phone directory - linear search by Phone Number

a. Read N contacts
b. Search a contact by Phone Number using linear search
c. Display the contact details
d. Display "Number not saved" if not found

Program (C++)

```
/* Set 51 - Phone directory - linear search by Phone Number */
// Compile: g++ -std=c++17 -Wall -Wextra set51.cpp -o set51
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Contact {
    string name;    // Name
    string phone;   // Phone Number
    string city;    // City
};

Contact rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Phone Number : ";
        cin >> ws;
        getline(cin, rec[i].phone);
        cout << " Enter City : ";
        cin >> ws;
        getline(cin, rec[i].city);
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "Phone Number" << ": " << rec[i].phone << "\n";
    cout << " " << left << setw(16) << "City" << ": " << rec[i].city << "\n";
}

// linear search : returns index of the first match, or -1
int linearSearch(const string &key, int &comparisons)
{
    comparisons = 0;
    for (int i = 0; i < n; i++) {
        comparisons++;
        if (rec[i].phone == key)
            return i;
    }
    return -1;
}

int main()
{
    int idx, comparisons;
    string key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    cout << "Enter Phone Number to search: ";
```

```
    cin >> ws;
    getline(cin, key);
    idx = linearSearch(key, comparisons);
    if (idx == -1) {
        cout << "\nNumber not saved\n";
    } else {
        cout << "\nRecord found:\n";
        printRecord(idx);
    }
    return 0;
}
```

Set 52 – Warehouse stock list - linear search by Item Code (repeated codes)

a. Read N items (item codes may repeat across batches)
b. Search by Item Code using linear search
c. Display all batches with that Item Code
d. Display the total quantity available for that Item Code

Program (C++)

```
/* Set 52 - Warehouse stock list - linear search by Item Code (repeated codes) */
// Compile: g++ -std=c++17 -Wall -Wextra set52.cpp -o set52
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Item {
    int code;      // Item Code
    string name;   // Item Name
    int qty;       // Quantity
};

Item rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Item Code : ";
        cin >> rec[i].code;
        cout << " Enter Item Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Quantity : ";
        cin >> rec[i].qty;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Item Code" << ": " << rec[i].code << "\n";
    cout << " " << left << setw(16) << "Item Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "Quantity" << ": " << rec[i].qty << "\n";
}

int main()
{
    int matches = 0;
    int key;
    int total = 0;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
```

```
    cout << "Enter Item Code to search: ";
    cin >> key;
    cout << "\nMatching records:\n";
    for (int i = 0; i < n; i++) {               // values may repeat, so scan all
        if (rec[i].code == key) {
            matches++;
            cout << "Match " << matches << " (position " << i + 1 << ")\n";
            printRecord(i);
            total += rec[i].qty;
        }
    }
    if (matches == 0) {
        cout << "Item code not found\n";
    } else {
        cout << "\nTotal number of matches found = " << matches << "\n";
        cout << "Total quantity available = " << total << "\n";
    }
    return 0;
}
```

Set 53 – Online store product catalogue - binary search by Product ID

a. Read N products and sort them by Product ID
b. Search a product by Product ID using binary search
c. Display the product details if found
d. Display the number of comparisons made

Program (C++)

```
/* Set 53 - Online store product catalogue - binary search by Product ID */
// Compile: g++ -std=c++17 -Wall -Wextra set53.cpp -o set53
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Product {
    int id;        // Product ID
    string name;   // Name
    float price;   // Price
};

Product rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Product ID : ";
        cin >> rec[i].id;
        cout << " Enter Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Price : ";
        cin >> rec[i].price;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Product ID" << ": " << rec[i].id << "\n";
    cout << " " << left << setw(16) << "Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "Price" << ": " << rec[i].price << "\n";
}

void printAll()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        printRecord(i);
```

```
    }
}

// insertion sort : ascending order of Product ID
void sortRecords()
{
    for (int i = 1; i < n; i++) {
        Product temp = rec[i];
        int j = i - 1;
        while (j >= 0 && rec[j].id > temp.id) {
            rec[j + 1] = rec[j];
            j--;
        }
        rec[j + 1] = temp;
    }
}

// iterative binary search : returns index or -1.
// One comparison (three-way) is counted for every probe of the middle element.
int binarySearch(int key, int &comparisons)
{
    int low = 0, high = n - 1, mid;
    comparisons = 0;
    while (low <= high) {
        mid = low + (high - low) / 2;
        comparisons++;
        if (rec[mid].id == key) {
            return mid;
        } else if (rec[mid].id < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main()
{
    int idx, count = 0;
    int key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    sortRecords();
    cout << "\nRecords after sorting by Product ID:\n";
    printAll();
    cout << "Enter Product ID to search: ";
    cin >> key;
    idx = binarySearch(key, count);
    if (idx == -1) {
        cout << "\nProduct not found\n";
    } else {
        cout << "\nRecord found at index " << idx << ":\n";
        printRecord(idx);
    }
    cout << "Number of comparisons made = " << count << "\n";
    return 0;
}
```

Set 54 – Bank customer records - binary search by Account No

a. Read N records and sort them by Account No
b. Search an account using binary search
c. Display the customer's record
d. Display "Account not found" if absent

Program (C++)

```
/* Set 54 - Bank customer records - binary search by Account No */
// Compile: g++ -std=c++17 -Wall -Wextra set54.cpp -o set54
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Customer {
    int acc;         // Account No
    string name;     // Name
    float balance;   // Balance
};

Customer rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Account No : ";
        cin >> rec[i].acc;
        cout << " Enter Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Balance : ";
        cin >> rec[i].balance;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Account No" << ": " << rec[i].acc << "\n";
    cout << " " << left << setw(16) << "Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "Balance" << ": " << rec[i].balance << "\n";
}

void printAll()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        printRecord(i);
    }
}

// insertion sort : ascending order of Account No
void sortRecords()
{
    for (int i = 1; i < n; i++) {
        Customer temp = rec[i];
        int j = i - 1;
        while (j >= 0 && rec[j].acc > temp.acc) {
            rec[j + 1] = rec[j];
            j--;
        }
        rec[j + 1] = temp;
    }
}

// iterative binary search : returns index or -1.
// One comparison (three-way) is counted for every probe of the middle element.
int binarySearch(int key, int &comparisons)
{
    int low = 0, high = n - 1, mid;
```

```
    comparisons = 0;
    while (low <= high) {
        mid = low + (high - low) / 2;
        comparisons++;
        if (rec[mid].acc == key) {
            return mid;
        } else if (rec[mid].acc < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main()
{
    int idx, count = 0;
    int key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    sortRecords();
    cout << "\nRecords after sorting by Account No:\n";
    printAll();
    cout << "Enter Account No to search: ";
    cin >> key;
    idx = binarySearch(key, count);
    if (idx == -1) {
        cout << "\nAccount not found\n";
    } else {
        cout << "\nRecord found at index " << idx << ":\n";
        printRecord(idx);
    }
    return 0;
}
```

Set 55 – Student records (sorted input) - iterative binary search with low/mid/high trace

a. Read N records already sorted by Roll No
b. Search a student by Roll No using binary search (iterative)
c. Display the record if found
d. Display the low, mid and high index at every step

Program (C++)

```
/* Set 55 - Student records (sorted input) - iterative binary search with low/mid/high
   trace */
// Compile: g++ -std=c++17 -Wall -Wextra set55.cpp -o set55
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Student {
    int roll;      // Roll No
    string name;   // Name
    float cgpa;    // CGPA
};

Student rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
```

```
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Roll No : ";
        cin >> rec[i].roll;
        cout << " Enter Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter CGPA : ";
        cin >> rec[i].cgpa;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Roll No" << ": " << rec[i].roll << "\n";
    cout << " " << left << setw(16) << "Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "CGPA" << ": " << rec[i].cgpa << "\n";
}

// the records must be entered in ascending order of Roll No
bool isSorted()
{
    for (int i = 1; i < n; i++) {
        if (rec[i].roll < rec[i - 1].roll)
            return false;
    }
    return true;
}

// iterative binary search : returns index or -1.
// One comparison (three-way) is counted for every probe of the middle element.
int binarySearch(int key, int &comparisons)
{
    int low = 0, high = n - 1, mid, step = 0;
    comparisons = 0;
    while (low <= high) {
        mid = low + (high - low) / 2;
        cout << "Step " << ++step << " : low = " << low << ", mid = " << mid
             << ", high = " << high << "\n";
        comparisons++;
        if (rec[mid].roll == key) {
            return mid;
        } else if (rec[mid].roll < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main()
{
    int idx, count = 0;
    int key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    if (!isSorted()) {
        cout << "Records are not in ascending order of Roll No. Please enter them sorted.\n";
        return 1;
    }
    cout << "Enter Roll No to search: ";
    cin >> key;
    idx = binarySearch(key, count);
    if (idx == -1) {
        cout << "\nStudent not found\n";
    } else {
        cout << "\nRecord found at index " << idx << ":\n";
        printRecord(idx);
    }
```

```
    return 0;
}
```

Set 56 – Library records - recursive binary search

a. Read N books and sort them by Book ID
b. Search a book using binary search (recursive)
c. Display the record if found
d. Display the number of recursive calls made

Program (C++)

```
/* Set 56 - Library records - recursive binary search */
// Compile: g++ -std=c++17 -Wall -Wextra set56.cpp -o set56
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Book {
    int id;         // Book ID
    string title;   // Title
    int copies;     // Copies
};

Book rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Book ID : ";
        cin >> rec[i].id;
        cout << " Enter Title : ";
        cin >> ws;
        getline(cin, rec[i].title);
        cout << " Enter Copies : ";
        cin >> rec[i].copies;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Book ID" << ": " << rec[i].id << "\n";
    cout << " " << left << setw(16) << "Title" << ": " << rec[i].title << "\n";
    cout << " " << left << setw(16) << "Copies" << ": " << rec[i].copies << "\n";
}

void printAll()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        printRecord(i);
    }
}

// insertion sort : ascending order of Book ID
void sortRecords()
{
    for (int i = 1; i < n; i++) {
        Book temp = rec[i];
        int j = i - 1;
        while (j >= 0 && rec[j].id > temp.id) {
            rec[j + 1] = rec[j];
            j--;
        }
        rec[j + 1] = temp;
    }
}
// recursive binary search : returns index or -1, counts every call in calls
```

```
int binarySearch(int key, int low, int high, int &calls)
{
    calls++;
    if (low > high)
        return -1;
    int mid = low + (high - low) / 2;
    if (rec[mid].id == key)
        return mid;
    if (key < rec[mid].id)
        return binarySearch(key, low, mid - 1, calls);
    return binarySearch(key, mid + 1, high, calls);
}

int main()
{
    int idx, count = 0;
    int key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    sortRecords();
    cout << "\nRecords after sorting by Book ID:\n";
    printAll();
    cout << "Enter Book ID to search: ";
    cin >> key;
    idx = binarySearch(key, 0, n - 1, count);
    if (idx == -1) {
        cout << "\nBook not found\n";
    } else {
        cout << "\nRecord found at index " << idx << ":\n";
        printRecord(idx);
    }
    cout << "Number of recursive calls made = " << count << "\n";
    return 0;
}
```

Set 57 – Hospital patient records - binary search compared with linear search

a. Read N patients and sort them by Patient ID
b. Search a patient using binary search
c. Display the record if found
d. Display the number of comparisons and compare it with linear search

Program (C++)

```
/* Set 57 - Hospital patient records - binary search compared with linear search */
// Compile: g++ -std=c++17 -Wall -Wextra set57.cpp -o set57
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Patient {
    int id;        // Patient ID
    string name;   // Name
    int age;       // Age
};

Patient rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Patient ID : ";
```

```
        cin >> rec[i].id;
        cout << " Enter Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Age : ";
        cin >> rec[i].age;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Patient ID" << ": " << rec[i].id << "\n";
    cout << " " << left << setw(16) << "Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "Age" << ": " << rec[i].age << "\n";
}

void printAll()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        printRecord(i);
    }
}

// insertion sort : ascending order of Patient ID
void sortRecords()
{
    for (int i = 1; i < n; i++) {
        Patient temp = rec[i];
        int j = i - 1;
        while (j >= 0 && rec[j].id > temp.id) {
            rec[j + 1] = rec[j];
            j--;
        }
        rec[j + 1] = temp;
    }
}

// iterative binary search : returns index or -1.
// One comparison (three-way) is counted for every probe of the middle element.
int binarySearch(int key, int &comparisons)
{
    int low = 0, high = n - 1, mid;
    comparisons = 0;
    while (low <= high) {
        mid = low + (high - low) / 2;
        comparisons++;
        if (rec[mid].id == key) {
            return mid;
        } else if (rec[mid].id < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

// linear search, used only to compare the number of comparisons
int linearSearch(int key, int &comparisons)
{
    comparisons = 0;
    for (int i = 0; i < n; i++) {
        comparisons++;
        if (rec[i].id == key)
            return i;
    }
    return -1;
}

int main()
{
    int idx, count = 0;
    int lcount;
    int key;
```

```
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    sortRecords();
    cout << "\nRecords after sorting by Patient ID:\n";
    printAll();
    cout << "Enter Patient ID to search: ";
    cin >> key;
    idx = binarySearch(key, count);
    linearSearch(key, lcount);          // only the comparison count is needed
    if (idx == -1) {
        cout << "\nPatient not found\n";
    } else {
        cout << "\nRecord found at index " << idx << ":\n";
        printRecord(idx);
    }
    cout << "Comparisons in binary search = " << count << "\n";
    cout << "Comparisons in linear search = " << lcount << "\n";
    return 0;
}
```

Set 58 – Railway timetable - binary search by Train No

a. Read N trains and sort them by Train No
b. Search a train using binary search
c. Display the train details if found
d. Display "Train not available" otherwise

Program (C++)

```
/* Set 58 - Railway timetable - binary search by Train No */
// Compile: g++ -std=c++17 -Wall -Wextra set58.cpp -o set58
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Train {
    int no;        // Train No
    string name;   // Train Name
    string dep;    // Departure Time
};

Train rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Train No : ";
        cin >> rec[i].no;
        cout << " Enter Train Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Departure Time : ";
        cin >> ws;
        getline(cin, rec[i].dep);
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Train No" << ": " << rec[i].no << "\n";
    cout << " " << left << setw(16) << "Train Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "Departure Time" << ": " << rec[i].dep << "\n";
```

```
}

void printAll()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        printRecord(i);
    }
}

// insertion sort : ascending order of Train No
void sortRecords()
{
    for (int i = 1; i < n; i++) {
        Train temp = rec[i];
        int j = i - 1;
        while (j >= 0 && rec[j].no > temp.no) {
            rec[j + 1] = rec[j];
            j--;
        }
        rec[j + 1] = temp;
    }
}

// iterative binary search : returns index or -1.
// One comparison (three-way) is counted for every probe of the middle element.
int binarySearch(int key, int &comparisons)
{
    int low = 0, high = n - 1, mid;
    comparisons = 0;
    while (low <= high) {
        mid = low + (high - low) / 2;
        comparisons++;
        if (rec[mid].no == key) {
            return mid;
        } else if (rec[mid].no < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main()
{
    int idx, count = 0;
    int key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    sortRecords();
    cout << "\nRecords after sorting by Train No:\n";
    printAll();
    cout << "Enter Train No to search: ";
    cin >> key;
    idx = binarySearch(key, count);
    if (idx == -1) {
        cout << "\nTrain not available\n";
    } else {
        cout << "\nRecord found at index " << idx << ":\n";
        printRecord(idx);
    }
    return 0;
}
```

Set 59 – Employee records - binary search by Employee ID

a. Read N employees and sort them by Employee ID
b. Search an employee using binary search
c. Display the record if found
d. Display the number of steps taken

Program (C++)

```
/* Set 59 - Employee records - binary search by Employee ID */
// Compile: g++ -std=c++17 -Wall -Wextra set59.cpp -o set59
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Employee {
    int id;         // Employee ID
    string name;    // Name
    float salary;   // Salary
};

Employee rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Employee ID : ";
        cin >> rec[i].id;
        cout << " Enter Name : ";
        cin >> ws;
        getline(cin, rec[i].name);
        cout << " Enter Salary : ";
        cin >> rec[i].salary;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Employee ID" << ": " << rec[i].id << "\n";
    cout << " " << left << setw(16) << "Name" << ": " << rec[i].name << "\n";
    cout << " " << left << setw(16) << "Salary" << ": " << rec[i].salary << "\n";
}

void printAll()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        printRecord(i);
    }
}

// insertion sort : ascending order of Employee ID
void sortRecords()
{
    for (int i = 1; i < n; i++) {
        Employee temp = rec[i];
        int j = i - 1;
        while (j >= 0 && rec[j].id > temp.id) {
            rec[j + 1] = rec[j];
            j--;
        }
        rec[j + 1] = temp;
    }
}

// iterative binary search : returns index or -1.
// One comparison (three-way) is counted for every probe of the middle element.
int binarySearch(int key, int &comparisons)
{
    int low = 0, high = n - 1, mid;
```

```
    comparisons = 0;
    while (low <= high) {
        mid = low + (high - low) / 2;
        comparisons++;
        if (rec[mid].id == key) {
            return mid;
        } else if (rec[mid].id < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main()
{
    int idx, count = 0;
    int key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    sortRecords();
    cout << "\nRecords after sorting by Employee ID:\n";
    printAll();
    cout << "Enter Employee ID to search: ";
    cin >> key;
    idx = binarySearch(key, count);
    if (idx == -1) {
        cout << "\nEmployee not found\n";
    } else {
        cout << "\nRecord found at index " << idx << ":\n";
        printRecord(idx);
    }
    cout << "Number of steps taken = " << count << "\n";
    return 0;
}
```

Set 60 – Hotel rooms (sorted input) - binary search by Room No

a. Read N rooms sorted by Room No
b. Search a room using binary search
c. Display the room details and rate if found
d. Display "Room does not exist" otherwise

Program (C++)

```
/* Set 60 - Hotel rooms (sorted input) - binary search by Room No */
// Compile: g++ -std=c++17 -Wall -Wextra set60.cpp -o set60
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct Room {
    int room;      // Room No
    string type;   // Type
    float rate;    // Rate
};

Room rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
```

```
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Room No : ";
        cin >> rec[i].room;
        cout << " Enter Type : ";
        cin >> ws;
        getline(cin, rec[i].type);
        cout << " Enter Rate : ";
        cin >> rec[i].rate;
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Room No" << ": " << rec[i].room << "\n";
    cout << " " << left << setw(16) << "Type" << ": " << rec[i].type << "\n";
    cout << " " << left << setw(16) << "Rate" << ": " << rec[i].rate << "\n";
}

// the records must be entered in ascending order of Room No
bool isSorted()
{
    for (int i = 1; i < n; i++) {
        if (rec[i].room < rec[i - 1].room)
            return false;
    }
    return true;
}

// iterative binary search : returns index or -1.
// One comparison (three-way) is counted for every probe of the middle element.
int binarySearch(int key, int &comparisons)
{
    int low = 0, high = n - 1, mid;
    comparisons = 0;
    while (low <= high) {
        mid = low + (high - low) / 2;
        comparisons++;
        if (rec[mid].room == key) {
            return mid;
        } else if (rec[mid].room < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main()
{
    int idx, count = 0;
    int key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    if (!isSorted()) {
        cout << "Records are not in ascending order of Room No. Please enter them sorted.\n";
        return 1;
    }
    cout << "Enter Room No to search: ";
    cin >> key;
    idx = binarySearch(key, count);
    if (idx == -1) {
        cout << "\nRoom does not exist\n";
    } else {
        cout << "\nRecord found at index " << idx << ":\n";
        printRecord(idx);
    }
    return 0;
}
```

Set 61 – Pin code list - binary search by Pin Code

a. Read N entries and sort them by Pin Code
b. Search a Pin Code using binary search
c. Display the area and city
d. Display the number of comparisons made

Program (C++)

```
/* Set 61 - Pin code list - binary search by Pin Code */
// Compile: g++ -std=c++17 -Wall -Wextra set61.cpp -o set61
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 100;

struct PinCode {
    int pin;       // Pin Code
    string area;   // Area
    string city;   // City
};

PinCode rec[MAX];
int n;

void readRecords()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        cout << " Enter Pin Code : ";
        cin >> rec[i].pin;
        cout << " Enter Area : ";
        cin >> ws;
        getline(cin, rec[i].area);
        cout << " Enter City : ";
        cin >> ws;
        getline(cin, rec[i].city);
    }
}

void printRecord(int i)
{
    cout << " " << left << setw(16) << "Pin Code" << ": " << rec[i].pin << "\n";
    cout << " " << left << setw(16) << "Area" << ": " << rec[i].area << "\n";
    cout << " " << left << setw(16) << "City" << ": " << rec[i].city << "\n";
}

void printAll()
{
    for (int i = 0; i < n; i++) {
        cout << "Record " << i + 1 << "\n";
        printRecord(i);
    }
}

// insertion sort : ascending order of Pin Code
void sortRecords()
{
    for (int i = 1; i < n; i++) {
        PinCode temp = rec[i];
        int j = i - 1;
        while (j >= 0 && rec[j].pin > temp.pin) {
            rec[j + 1] = rec[j];
            j--;
        }
        rec[j + 1] = temp;
    }
}

// iterative binary search : returns index or -1.
```

```
// One comparison (three-way) is counted for every probe of the middle element.
int binarySearch(int key, int &comparisons)
{
    int low = 0, high = n - 1, mid;
    comparisons = 0;
    while (low <= high) {
        mid = low + (high - low) / 2;
        comparisons++;
        if (rec[mid].pin == key) {
            return mid;
        } else if (rec[mid].pin < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main()
{
    int idx, count = 0;
    int key;
    cout << fixed << setprecision(2);
    cout << "Enter number of records N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    readRecords();
    sortRecords();
    cout << "\nRecords after sorting by Pin Code:\n";
    printAll();
    cout << "Enter Pin Code to search: ";
    cin >> key;
    idx = binarySearch(key, count);
    if (idx == -1) {
        cout << "\nPin Code not found\n";
    } else {
        cout << "\nRecord found at index " << idx << ":\n";
        printRecord(idx);
    }
    cout << "Number of comparisons made = " << count << "\n";
    return 0;
}
```

Set 62 – Sorted student marks with duplicates - first / last occurrence by binary search

a. Read N marks and sort them in ascending order
b. Find the first occurrence of a given mark using binary search
c. Find the last occurrence of the same mark
d. Display how many students scored that mark

Program (C++)

```
/* Set 62 - Sorted student marks with duplicates - first / last occurrence by binary
   search */
// Compile: g++ -std=c++17 -Wall -Wextra set62.cpp -o set62
#include <iostream>
using namespace std;

const int MAX = 100;

int a[MAX], n;

// b. first occurrence : keep searching on the left half after a hit
int firstOccurrence(int key)
{
    int low = 0, high = n - 1, mid, res = -1;
    while (low <= high) {
        mid = low + (high - low) / 2;
        if (a[mid] == key) {
```

```
            res = mid;
            high = mid - 1;
        } else if (a[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return res;
}

// c. last occurrence : keep searching on the right half after a hit
int lastOccurrence(int key)
{
    int low = 0, high = n - 1, mid, res = -1;
    while (low <= high) {
        mid = low + (high - low) / 2;
        if (a[mid] == key) {
            res = mid;
            low = mid + 1;
        } else if (a[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return res;
}

int main()
{
    int key, first, last;
    cout << "Enter number of students N (max " << MAX << "): ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Invalid N.\n";
        return 1;
    }
    for (int i = 0; i < n; i++) {
        cout << "Enter marks of student " << i + 1 << ": ";
        cin >> a[i];
    }
    for (int i = 1; i < n; i++) {               // a. sort in ascending order
        int temp = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > temp) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = temp;
    }
    cout << "\nSorted marks : ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << "\n";
    cout << "\nEnter the mark to search: ";
    cin >> key;
    first = firstOccurrence(key);
    if (first == -1) {
        cout << "Mark " << key << " not found.\n";
        return 0;
    }
    last = lastOccurrence(key);
    cout << "First occurrence at index " << first << " (position " << first + 1 << ")\n";
    cout << "Last occurrence at index " << last << " (position " << last + 1 << ")\n";
    cout << "Number of students who scored " << key << " = " << last - first + 1 << "\n";
    return 0;
}
```

Set 63 – Ticket booking counter of a cinema (simple queue, array based)

a. Enqueue a Token No
b. Dequeue and display the item being processed
c. Display the front token and rear token Token Nos
d. Display the complete queue

Program (C++)

```
/* Set 63 - Ticket booking counter of a cinema (simple queue, array based) */
// Compile: g++ -std=c++17 -Wall -Wextra set63.cpp -o set63
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Token No
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Token No : ";
    cin >> x.id;
}

void printItem(const Item &x)
{
    cout << "Token No " << x.id;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
    cout << "Token No " << x.id << " added to the queue.\n";
}

// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;        // queue became empty - reset
    else
        front++;
    cout << "Customer served: ";
    printItem(x);
    cout << "\n";
}

void showFrontRear()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
```

```
        return;
    }
    cout << "Front token : ";
    printItem(q[front]);
    cout << "\nRear token : ";
    printItem(q[rear]);
    cout << "\n";
}

void display()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "FRONT -> ";
    for (int i = front; i <= rear; i++) {
        printItem(q[i]);
        if (i < rear)
            cout << " | ";
    }
    cout << " <- REAR\n";
}

int main()
{
    int choice;
    do {
        cout << "\n===== CINEMA TICKET COUNTER =====\n";
        cout << "1. Enqueue a Token No\n";
        cout << "2. Dequeue and display the item being processed\n";
        cout << "3. Display the front token and rear token Token Nos\n";
        cout << "4. Display the complete queue\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            showFrontRear();
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 64 – Teller counter of a bank (simple queue)

a. Enqueue a Token No
b. Dequeue and display the item being processed
c. Display the front token and rear token Token Nos
d. Display the number of items waiting

Program (C++)

```
/* Set 64 - Teller counter of a bank (simple queue) */
// Compile: g++ -std=c++17 -Wall -Wextra set64.cpp -o set64
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Token No
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Token No : ";
    cin >> x.id;
}

void printItem(const Item &x)
{
    cout << "Token No " << x.id;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
    cout << "Token No " << x.id << " added to the queue.\n";
}

// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;        // queue became empty - reset
    else
        front++;
    cout << "Customer being served: ";
    printItem(x);
    cout << "\n";
}

void showFrontRear()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
```

```
        return;
    }
    cout << "Front token : ";
    printItem(q[front]);
    cout << "\nRear token : ";
    printItem(q[rear]);
    cout << "\n";
}

int countWaiting()
{
    return isEmpty() ? 0 : rear - front + 1;
}

int main()
{
    int choice;
    do {
        cout << "\n===== BANK TELLER QUEUE =====\n";
        cout << "1. Enqueue a Token No\n";
        cout << "2. Dequeue and display the item being processed\n";
        cout << "3. Display the front token and rear token Token Nos\n";
        cout << "4. Display the number of items waiting\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            showFrontRear();
            break;
        case 4:
            cout << "Number waiting = " << countWaiting() << "\n";
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 65 – Print queue of a college lab (simple queue)

a. Add a print job to the queue
b. Remove and display the job being printed
c. Display the item at the front
d. Display all pending jobs and the total pages pending

Program (C++)

```
/* Set 65 - Print queue of a college lab (simple queue) */
// Compile: g++ -std=c++17 -Wall -Wextra set65.cpp -o set65
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Job ID
    int pages;                  // Number of Pages
};
Item q[MAX];
```

```
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Job ID : ";
    cin >> x.id;
    cout << " Enter Number of Pages : ";
    cin >> x.pages;
}

void printItem(const Item &x)
{
    cout << "Job ID " << x.id << " (" << x.pages << " pages)";
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
    cout << "Job ID " << x.id << " added to the queue.\n";
}

// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;         // queue became empty - reset
    else
        front++;
    cout << "Job being printed: ";
    printItem(x);
    cout << "\n";
}

void showFront()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "Job at front : ";
    printItem(q[front]);
    cout << "\n";
}

void displayPending()
{
    int total = 0;
    if (isEmpty()) {
        cout << "No pending jobs.\n";
        return;
    }
    cout << "\n" << left << setw(10) << "Job ID" << " " << "Pages" << "\n";
    cout << "----------------------\n";
    for (int i = front; i <= rear; i++) {
        cout << left << setw(10) << q[i].id << " " << q[i].pages << "\n";
        total += q[i].pages;
    }
    cout << "Total pages pending = " << total << "\n";
```

```
}

int main()
{
    int choice;
    do {
        cout << "\n===== LAB PRINT QUEUE =====\n";
        cout << "1. Add a print job to the queue\n";
        cout << "2. Remove and display the job being printed\n";
        cout << "3. Display the item at the front\n";
        cout << "4. Display all pending jobs and the total pages pending\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            showFront();
            break;
        case 4:
            displayPending();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 66 – OPD queue of a hospital (simple queue)

a. Enqueue a Token No
b. Dequeue and display the item being processed
c. Check whether the queue is empty or full
d. Display the complete queue

Program (C++)

```
/* Set 66 - OPD queue of a hospital (simple queue) */
// Compile: g++ -std=c++17 -Wall -Wextra set66.cpp -o set66
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Token No
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Token No : ";
    cin >> x.id;
}

void printItem(const Item &x)
```

```
{
    cout << "Token No " << x.id;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
    cout << "Token No " << x.id << " added to the queue.\n";
}

// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;         // queue became empty - reset
    else
        front++;
    cout << "Patient being called: ";
    printItem(x);
    cout << "\n";
}

void checkStatus()
{
    cout << "Queue is " << (isEmpty() ? "EMPTY" : "not empty") << ".\n";
    cout << "Queue is " << (isFull() ? "FULL" : "not full") << ".\n";
}

void display()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "FRONT -> ";
    for (int i = front; i <= rear; i++) {
        printItem(q[i]);
        if (i < rear)
            cout << " | ";
    }
    cout << " <- REAR\n";
}

int main()
{
    int choice;
    do {
        cout << "\n===== OPD QUEUE =====\n";
        cout << "1. Enqueue a Token No\n";
        cout << "2. Dequeue and display the item being processed\n";
        cout << "3. Check whether the queue is empty or full\n";
        cout << "4. Display the complete queue\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
```

```
            dequeue();
            break;
        case 3:
            checkStatus();
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 67 – Check-in counter of an airline (simple queue)

a. Enqueue (add) a new entry
b. Dequeue and display the item being processed
c. Display the first passenger and last passenger entries
d. Display the complete queue

Program (C++)

```
/* Set 67 - Check-in counter of an airline (simple queue) */
// Compile: g++ -std=c++17 -Wall -Wextra set67.cpp -o set67
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Ticket No
    string name;                   // Name
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Ticket No : ";
    cin >> x.id;
    cout << " Enter Name : ";
    cin >> ws;
    getline(cin, x.name);
}

void printItem(const Item &x)
{
    cout << "Ticket No " << x.id << ", " << x.name;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
```

```
    cout << "Ticket No " << x.id << " added to the queue.\n";
}

// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;         // queue became empty - reset
    else
        front++;
    cout << "Passenger being checked in: ";
    printItem(x);
    cout << "\n";
}

void showFrontRear()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "First passenger : ";
    printItem(q[front]);
    cout << "\nLast passenger : ";
    printItem(q[rear]);
    cout << "\n";
}

void display()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "FRONT -> ";
    for (int i = front; i <= rear; i++) {
        printItem(q[i]);
        if (i < rear)
            cout << " | ";
    }
    cout << " <- REAR\n";
}

int main()
{
    int choice;
    do {
        cout << "\n===== AIRLINE CHECK-IN QUEUE =====\n";
        cout << "1. Enqueue (add) a new entry\n";
        cout << "2. Dequeue and display the item being processed\n";
        cout << "3. Display the first passenger and last passenger entries\n";
        cout << "4. Display the complete queue\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            showFrontRear();
            break;
        case 4:
            display();
            break;
        case 5:
```

```
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 68 – Order processing of a food delivery app (simple queue)

a. Enqueue a Order No
b. Dequeue and display the item being processed
c. Display the front order and rear order Order Nos
d. Display the complete queue

Program (C++)

```
/* Set 68 - Order processing of a food delivery app (simple queue) */
// Compile: g++ -std=c++17 -Wall -Wextra set68.cpp -o set68
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Order No
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Order No : ";
    cin >> x.id;
}

void printItem(const Item &x)
{
    cout << "Order No " << x.id;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
    cout << "Order No " << x.id << " added to the queue.\n";
}

// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;        // queue became empty - reset
```

```
    else
        front++;
    cout << "Order being dispatched: ";
    printItem(x);
    cout << "\n";
}

void showFrontRear()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "Front order : ";
    printItem(q[front]);
    cout << "\nRear order : ";
    printItem(q[rear]);
    cout << "\n";
}

void display()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "FRONT -> ";
    for (int i = front; i <= rear; i++) {
        printItem(q[i]);
        if (i < rear)
            cout << " | ";
    }
    cout << " <- REAR\n";
}

int main()
{
    int choice;
    do {
        cout << "\n===== FOOD DELIVERY ORDER QUEUE =====\n";
        cout << "1. Enqueue an Order No\n";
        cout << "2. Dequeue and display the item being processed\n";
        cout << "3. Display the front order and rear order Order Nos\n";
        cout << "4. Display the complete queue\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            showFrontRear();
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 69 – Call queue of a customer care centre (simple queue)

a. Enqueue a Call ID
b. Dequeue and display the item being processed
c. Display the number of items waiting
d. Display the complete queue

Program (C++)

```
/* Set 69 - Call queue of a customer care centre (simple queue) */
// Compile: g++ -std=c++17 -Wall -Wextra set69.cpp -o set69
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Call ID
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Call ID : ";
    cin >> x.id;
}

void printItem(const Item &x)
{
    cout << "Call ID " << x.id;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
    cout << "Call ID " << x.id << " added to the queue.\n";
}

// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;         // queue became empty - reset
    else
        front++;
    cout << "Call being answered: ";
    printItem(x);
    cout << "\n";
}

void display()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
```

```
        return;
    }
    cout << "FRONT -> ";
    for (int i = front; i <= rear; i++) {
        printItem(q[i]);
        if (i < rear)
            cout << " | ";
    }
    cout << " <- REAR\n";
}

int countWaiting()
{
    return isEmpty() ? 0 : rear - front + 1;
}

int main()
{
    int choice;
    do {
        cout << "\n===== CUSTOMER CARE CALL QUEUE =====\n";
        cout << "1. Enqueue a Call ID\n";
        cout << "2. Dequeue and display the item being processed\n";
        cout << "3. Display the number of items waiting\n";
        cout << "4. Display the complete queue\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            cout << "Number waiting = " << countWaiting() << "\n";
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 70 – Support ticket system of a software company (simple queue)

a. Enqueue (add) a new entry
b. Dequeue and display the item being processed
c. Display the front ticket and rear ticket entries
d. Display the complete queue

Program (C++)

```
/* Set 70 - Support ticket system of a software company (simple queue) */
// Compile: g++ -std=c++17 -Wall -Wextra set70.cpp -o set70
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Ticket ID
    string issue;                  // Issue Type
```

```
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Ticket ID : ";
    cin >> x.id;
    cout << " Enter Issue Type : ";
    cin >> ws;
    getline(cin, x.issue);
}

void printItem(const Item &x)
{
    cout << "Ticket ID " << x.id << ", Issue: " << x.issue;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
    cout << "Ticket ID " << x.id << " added to the queue.\n";
}

// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;         // queue became empty - reset
    else
        front++;
    cout << "Ticket being resolved: ";
    printItem(x);
    cout << "\n";
}

void showFrontRear()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "Front ticket : ";
    printItem(q[front]);
    cout << "\nRear ticket : ";
    printItem(q[rear]);
    cout << "\n";
}

void display()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "FRONT -> ";
    for (int i = front; i <= rear; i++) {
```

```
        printItem(q[i]);
        if (i < rear)
            cout << " | ";
    }
    cout << " <- REAR\n";
}

int main()
{
    int choice;
    do {
        cout << "\n===== SUPPORT TICKET QUEUE =====\n";
        cout << "1. Enqueue (add) a new entry\n";
        cout << "2. Dequeue and display the item being processed\n";
        cout << "3. Display the front ticket and rear ticket entries\n";
        cout << "4. Display the complete queue\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            showFrontRear();
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 71 – Railway reservation counter (simple queue)

a. Enqueue a Token No
b. Dequeue and display the item being processed
c. Check whether the queue is empty or full
d. Display the complete queue

Program (C++)

```
/* Set 71 - Railway reservation counter (simple queue) */
// Compile: g++ -std=c++17 -Wall -Wextra set71.cpp -o set71
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Token No
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
```

```
    cout << " Enter Token No : ";
    cin >> x.id;
}

void printItem(const Item &x)
{
    cout << "Token No " << x.id;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
    q[++rear] = x;
    cout << "Token No " << x.id << " added to the queue.\n";
}

// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;         // queue became empty - reset
    else
        front++;
    cout << "Passenger being served: ";
    printItem(x);
    cout << "\n";
}

void checkStatus()
{
    cout << "Queue is " << (isEmpty() ? "EMPTY" : "not empty") << ".\n";
    cout << "Queue is " << (isFull() ? "FULL" : "not full") << ".\n";
}

void display()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "FRONT -> ";
    for (int i = front; i <= rear; i++) {
        printItem(q[i]);
        if (i < rear)
            cout << " | ";
    }
    cout << " <- REAR\n";
}

int main()
{
    int choice;
    do {
        cout << "\n===== RAILWAY RESERVATION COUNTER =====\n";
        cout << "1. Enqueue a Token No\n";
        cout << "2. Dequeue and display the item being processed\n";
        cout << "3. Check whether the queue is empty or full\n";
        cout << "4. Display the complete queue\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
```

```
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            checkStatus();
            break;
        case 4:
            display();
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```

Set 72 – Order packing line of an Amazon warehouse (simple queue)

a. Enqueue a Order No
b. Dequeue and display the item being processed
c. Display the front order and rear order Order Nos
d. Display the complete queue and the count of items waiting

Program (C++)

```
/* Set 72 - Order packing line of an Amazon warehouse (simple queue) */
// Compile: g++ -std=c++17 -Wall -Wextra set72.cpp -o set72
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

const int MAX = 5;                // capacity of the queue (change as required)

struct Item {
    int id;                       // Order No
};

Item q[MAX];
int front = -1, rear = -1;

bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }

void readItem(Item &x)
{
    cout << " Enter Order No : ";
    cin >> x.id;
}

void printItem(const Item &x)
{
    cout << "Order No " << x.id;
}

// Enqueue : insert at the rear
void enqueue()
{
    Item x;
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    readItem(x);
    if (isEmpty())
        front = 0;
```

```
    q[++rear] = x;
    cout << "Order No " << x.id << " added to the queue.\n";
}

// Dequeue : remove from the front
void dequeue()
{
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }
    Item x = q[front];
    if (front == rear)
        front = rear = -1;         // queue became empty - reset
    else
        front++;
    cout << "Order being packed: ";
    printItem(x);
    cout << "\n";
}

void showFrontRear()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "Front order : ";
    printItem(q[front]);
    cout << "\nRear order : ";
    printItem(q[rear]);
    cout << "\n";
}

void display()
{
    if (isEmpty()) {
        cout << "Queue is empty.\n";
        return;
    }
    cout << "FRONT -> ";
    for (int i = front; i <= rear; i++) {
        printItem(q[i]);
        if (i < rear)
            cout << " | ";
    }
    cout << " <- REAR\n";
}

int countWaiting()
{
    return isEmpty() ? 0 : rear - front + 1;
}

int main()
{
    int choice;
    do {
        cout << "\n===== ORDER PACKING LINE =====\n";
        cout << "1. Enqueue an Order No\n";
        cout << "2. Dequeue and display the item being processed\n";
        cout << "3. Display the front order and rear order Order Nos\n";
        cout << "4. Display the complete queue and the count of items waiting\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice))
            break;
        switch (choice) {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
```

```
            showFrontRear();
            break;
        case 4:
            display();
            cout << "Number waiting = " << countWaiting() << "\n";
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 5);
    return 0;
}
```